ДИСЦИПЛИНА **Построение и применение нейронных сетей и генетических алгоритмов**

(полное наименование дисциплины без сокращений)

ИНСТИТУТ **информационных технологий**

КАФЕДРА **прикладной математики**

(полное наименование кафедры)

ВИД УЧЕБНОГО МАТЕРИАЛА **Лекционные материалы**

(в соответствии с пп.1-11)

ПРЕПОДАВАТЕЛЬ **Сараев Павел Викторович**

(фамилия, имя, отчество)

СЕМЕСТР **3, 2026-2027**

(указать семестр обучения, учебный год)

Построение и применение нейронных сетей и генетических алгоритмов
Лекция 1
Основные понятия и история развития теории нейронных сетей

САРАЕВ ПАВЕЛ ВИКТОРОВИЧ

РТУ МИРЭА
Кафедра прикладной математики

Москва – 2026

ПЛАН ЛЕКЦИИ

1. Содержание курса
2. Понятие нейронных сетей (НС)
3. История развития теории нейронных сетей
4. Задачи, решаемые нейронными сетями
5. Искусственный нейрон
6. Простейший перцептрон
7. Понятие обучения
8. Обучение перцептрона
9. Метод Монте-Карло для обучения нейронных сетей
10. Метод имитации отжига для обучения нейронных сетей
11. Выводы
12. Литература

1 Содержание курса
2 Понятие нейронных сетей (НС)
3 История развития теории нейронных сетей
4 Задачи, решаемые нейронными сетями
5 Искусственный нейрон
6 Простейший перцептрон
7 Понятие обучения
8 Обучение перцептрона
9 Метод Монте-Карло для обучения нейронных сетей
10 Метод имитации отжига для обучения нейронных сетей
11 Выводы
12 Литература

Содержание курса I

N	Тема занятия
1.1	Основные понятия и история развития теории нейронных сетей. (Лек). Содержание курса. Понятие нейронных сетей. История развития теории нейронных сетей. Задачи, решаемые нейронными сетями. Искусственный нейрон. Простейший перцептрон. Понятие обучения. Обучение перцептрона. Метод Монте-Карло для обучения нейронных сетей.
1.2	Выполнение практических заданий (Пр). Простейший перцептрон. Обучение перцептрона.
1.3	Выполнение практических заданий (Пр). Простейшие нейронные сети с различными функциями активации. Метод Монте-Карло в обучении простейших нейронных сетей.

Содержание курса II

1.4	Многослойные перцептроны и процедура обратного распространения ошибки. (Лек). Обзор архитектур нейронных сетей. Многослойный персептрон (нейронные сети прямого распространения). Универсальные аппроксимационные способности нейронных сетей. Теорема Фунахаши. Архитектура нейронных сетей с радиальными базисными функциями. Методика применения нейронных сетей. Обучение нейронных сетей как задача оптимизации. Суперпозиционная структура нейронных сетей. Процедура обратного распространения ошибки.
1.5	Выполнение практических заданий (Пр). Многослойный перцептрон.
1.6	Выполнение практических заданий (Пр). Процедура обратного распространения ошибки.

Содержание курса III

1.7	Генетические алгоритмы (Лек). Обучение как задача глобальной оптимизации. Генетические алгоритмы для решения задач глобальной оптимизации. Особенности применения генетических алгоритмов.
1.8	Выполнение практических заданий (Пр). Обучение многослойных нейронных сетей с учетом многоэкстремального характера функционала качества.
1.9	Выполнение практических заданий (Пр). Применение генетических алгоритмов в обучении нейронных сетей.

Содержание курса IV

1.10	Интервальные алгоритмы для решения задач глобальной оптимизации и обучения нейронных сетей. (Лек). Функционалы качества обучения, отличные от квадратичного. Основы интервального анализа. Интервальные алгоритмы в обучении нейронных сетей. Интервальные генетические алгоритмы.
1.11	Выполнение практических заданий (Пр). Обучение нейронных сетей с помощью интервальных алгоритмов.
1.12	Выполнение практических заданий (Пр). Интервальные генетические алгоритмы в обучении нейронных сетей.

Содержание курса V

1.13	Самоорганизующиеся карты Кохонена и их обучение (Лек). Самоорганизующиеся карты Кохонена SOM. Обучение без учителя. Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает всё» и его модификации.
1.14	Выполнение практических заданий (Пр). Самоорганизующиеся карты Кохонена для решения задач кластеризации данных.
1.15	Выполнение практических заданий (Пр). Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает всё».
1.16	Нейронные сети с обратными связями. (Лек). Понятие нейронных сетей с обратными связями. Архитектура сетей Хопфилда. Архитектура сетей Хэмминга. Временная обработка с использованием многослойных перцептронов. Алгоритм обратного распространения во времени.
1.17	Выполнение практических заданий (Пр). Нейронные сети Хопфилда и Хэмминга.

Содержание курса VI

1.18	Выполнение практических заданий (Пр). Временная обработка с использованием многослойных перцептронов. Алгоритм обратного распространения во времени.
1.19	Свёрточные и глубокие нейронные сети. (Лек). Свёрточные нейронные сети ЛеКуна. Глубокие нейронные сети. Архитектура BERT. Особенности архитектур глубоких нейронных сетей. Глубокое обучение и его особенности. Метод стохастического градиента и mini-batch метод.
1.20	Выполнение практических заданий (Пр). Свёрточные нейронные сети.
1.21	Выполнение практических заданий (Пр). Глубокое обучение нейронных сетей.

Содержание курса VII

1.22	Современные архитектуры нейронных сетей. нейронные сети для решения задач компьютерного зрения и обработки текста. (Лек). Генеративные и диффузионные модели. Решение прикладных задач с помощью нейронных сетей. Нейронные сети для решения задач компьютерного зрения. Нейронные сети для решения задач обработки естественного языка.
1.23	Выполнение практических заданий (Пр). Нейронные сети для решения задач компьютерного зрения.
1.24	Выполнение практических заданий (Пр). Нейронные сети для решения задач обработки естественного языка.

Содержание курса VIII

Промежуточная аттестация – **зачёт**.

1 Содержание курса

2 Понятие нейронных сетей (НС)

3 История развития теории нейронных сетей

4 Задачи, решаемые нейронными сетями

5 Искусственный нейрон

6 Простейший перцептрон

7 Понятие обучения

8 Обучение перцептрона

9 Метод Монте-Карло для обучения нейронных сетей

10 Метод имитации отжига для обучения нейронных сетей

11 Выводы

12 Литература

Понятие нейронных сетей (НС) I

Нейросетевое моделирование – математическое моделирование с использованием моделей специального вида (нейронных сетей, НС).

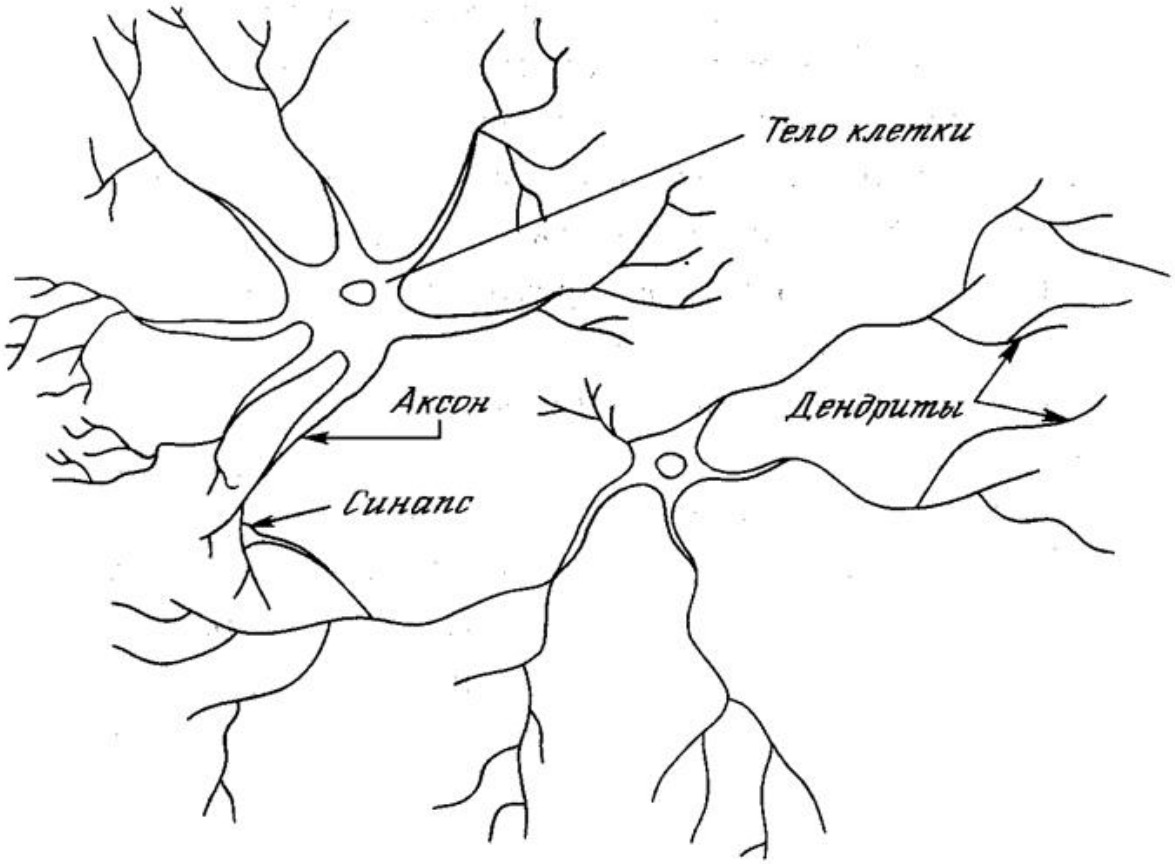


Рис. 1. Биологический нейрон (электрохимическая модель)

Понятие нейронных сетей (НС) II

НС – совокупность связанных нейронов.

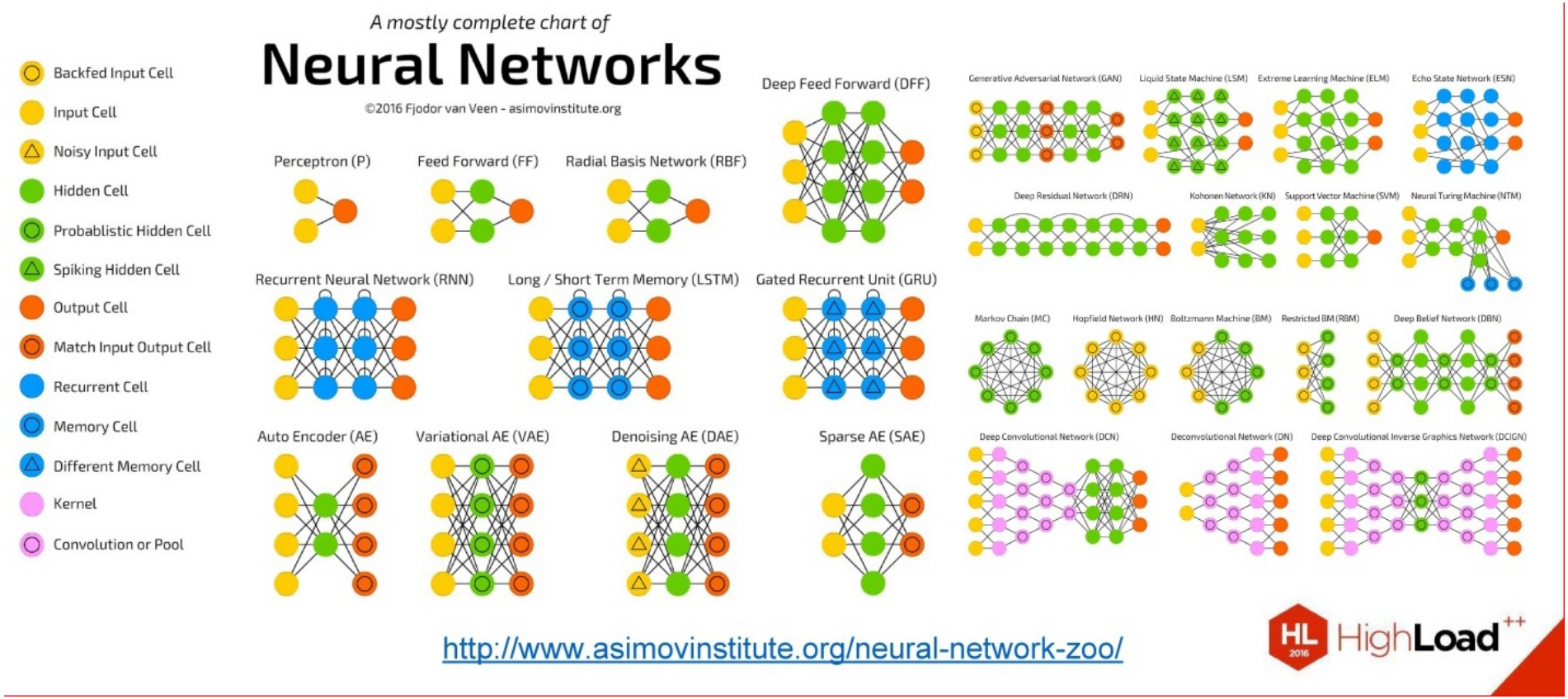
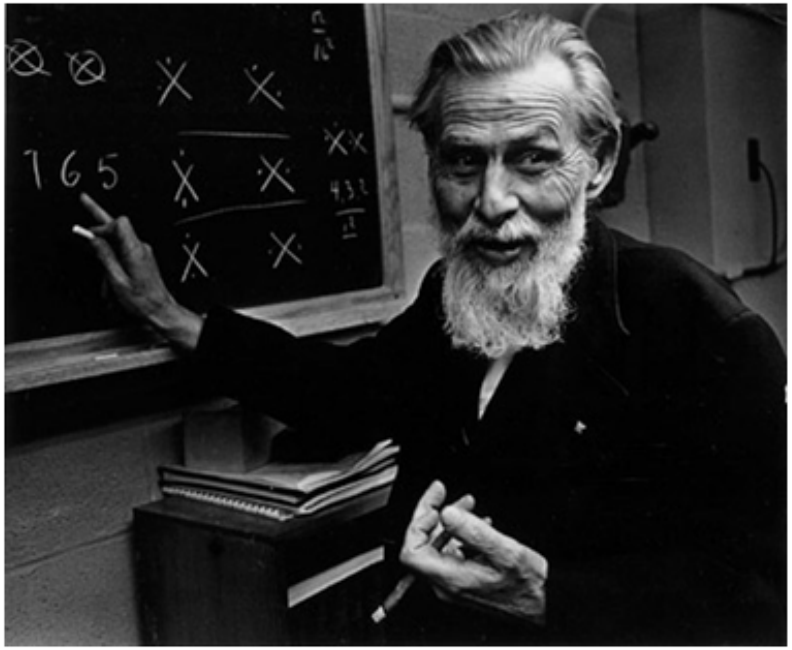


Рис. 2. Архитектуры НС

1 Содержание курса
2 Понятие нейронных сетей (НС)
3 История развития теории нейронных сетей
4 Задачи, решаемые нейронными сетями
5 Искусственный нейрон
6 Простейший перцептрон
7 Понятие обучения
8 Обучение перцептрона
9 Метод Монте-Карло для обучения нейронных сетей
10 Метод имитации отжига для обучения нейронных сетей
11 Выводы
12 Литература

История развития теории нейронных сетей I

- 1943 г. – МакКаллок (McCulloc) и Питтс (Pitts) «Логическое исчисление идей, относящихся к нервной деятельности»



Уоррен Маккаллок



Уолтер Питтс

Рис. 3. У. МакКаллок и У. Питс

- 1949 г. – Хебб (Hebb) «Организация поведения»;

История развития теории нейронных сетей II

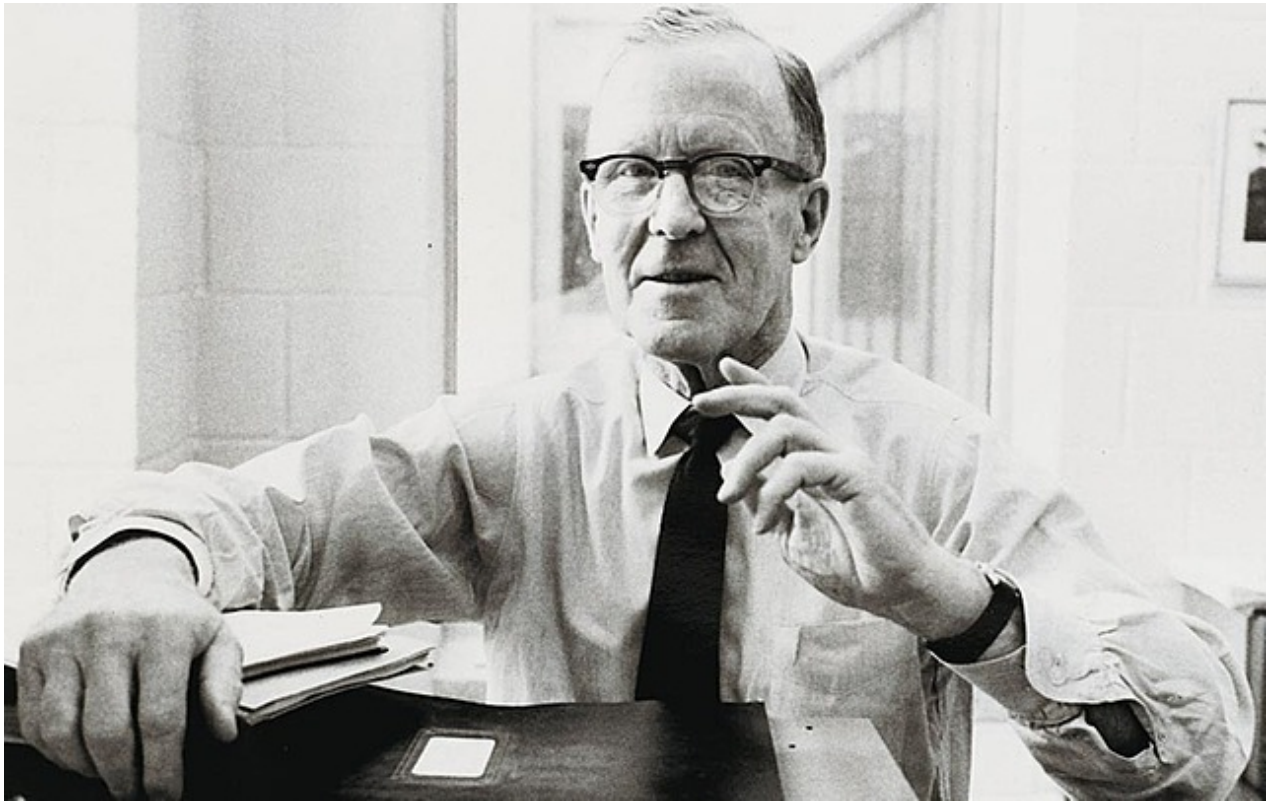


Рис. 4. Д. Хебб

- 1957 г. – Розенблатт – первый компьютер-нейронная сеть – перцептрон – обработка логических задач – линейная классификация;

История развития теории нейронных сетей III



Рис. 5. Ф. Розенблатт

- 1969 г. – Минский (Minsky) и Пейперт (Papert) «Перцептроны»: показана ограниченность возможностей НС (XOR-problem);

История развития теории нейронных сетей IV



Рис. 6. М. Минский

История развития теории нейронных сетей V



Рис. 7. С. Пейперт

История развития теории нейронных сетей VI

- 1982 г. – Хопфилд (Hopfield): работы по математическим основам динамики НС;



Рис. 8. Д. Хопфилд

История развития теории нейронных сетей VII

- 1984 г. – Кохонен (Kohonen): сети, обучающиеся без учителя;



Рис. 9. Т. Кохонен

История развития теории нейронных сетей VIII

- 1986 г. – Румельхарт (Rumelhart) и МакКлеланд (McCleland): алгоритм обратного распространения ошибки для обучения многослойных НС;



Рис. 10. Д. Румельхарт

История развития теории нейронных сетей IX

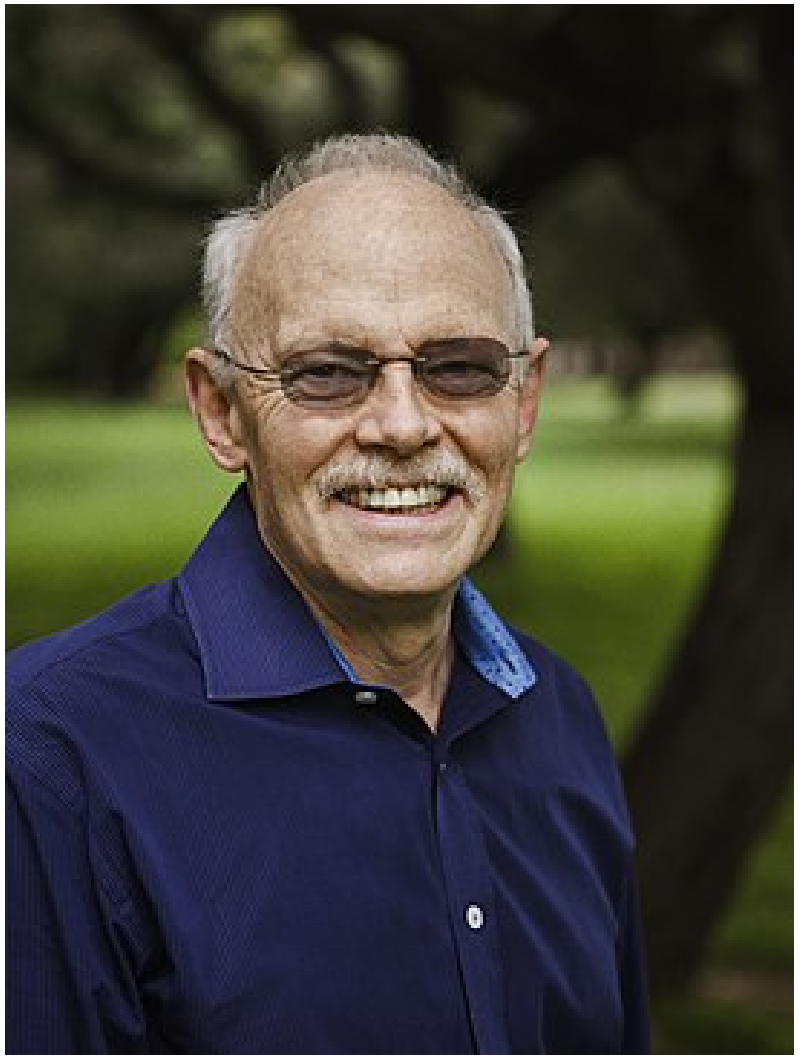


Рис. 11. Дж. МакКлеланд

История развития теории нейронных сетей X

- 1989 г. – ЛеКун (LeCun) – сверточные нейронные сети, распознавание рукописных индексов на конвертах (zip-кодов);
- 1998 г. – ЛеКун (LeCun) – распознавание чеков;



Рис. 12. Я. ЛеКун

История развития теории нейронных сетей XI

- 2006 г. – Хинтон (Hinton) – алгоритм обучения глубоких слоев в сетях – «умный» выбор начальных значений весов;



Рис. 13. Дж. Хинтон

История развития теории нейронных сетей XII

- 2010 г. – Глорот (Glorot) и Бенджио (Bengio) – анализ сложности обучения глубоких НС, критика дифференцируемых функций активации;



Рис. 14. Кс. Глорот

История развития теории нейронных сетей XIII



Рис. 15. Й. Бенджио

- 2012 г. – Хинтон (Hinton) – глубокие сверточные НС (deep convolutional NN) – отличные результаты в распознавании образов (ImageNet) – 15,3 % ошибки (26,2 % - ближайший конкурент).

1 Содержание курса
2 Понятие нейронных сетей (НС)
3 История развития теории нейронных сетей
4 Задачи, решаемые нейронными сетями
5 Искусственный нейрон
6 Простейший перцептрон
7 Понятие обучения
8 Обучение перцептрона
9 Метод Монте-Карло для обучения нейронных сетей
10 Метод имитации отжига для обучения нейронных сетей
11 Выводы
12 Литература

Задачи, решаемые нейронными сетями I

Решаемые задачи:

- Моделирование.
- Классификация.
- Кластеризация.
- Прогнозирование.
- Управление.

Области и примеры применения нейронных сетей:

- **Аэрокосмическая промышленность**: моделирование работы самолетов и их компонентов, траектории полета, определение неисправностей компонентов самолетов, системы управления самолетами, высокопроизводительное автопилотирование.
- **Автомобилестроение**: совершенствованные системы следования, разработка силовых агрегатов, виртуальных датчиков.
- **Электроника**: анализ отказов микросхем, компоновка микросхем, машинное зрение, нелинейное моделирование, прогнозирование кодовой последовательности, управление технологическим процессом.

Задачи, решаемые нейронными сетями II

- **Производство**: анализ химической продукции, динамическое моделирование систем химических процессов, управление технологическими процессами, диагностика процессов и оборудования, проектирование и анализ продукции, прогнозирование качества, планирование и управление, анализ качества компьютерных чипов, системы визуального контроля качества и анализ качества сварки.
- **Механика**: мониторинг состояния, моделирование систем и управление ими.
- **Робототехника**: роботы-погрузчики, контроллеры манипуляторов, управление траекторией и систем технического зрения.
- **Телекоммуникации**: управление сетью банкоматов, автоматизированные информационные службы, системы обработки платежей клиентов, сжатие данных, эквалайзеры, устранение неисправностей, распознавание рукописного текста, проектирование сетей, управление сетями, маршрутизация и контроль, мониторинг сети, перевод разговорной речи в режиме реального времени и распознавание образов (лиц, объектов, отпечатков пальцев, семантический анализ, орфография проверка, обработка сигналов и распознавание речи).

1 Содержание курса
2 Понятие нейронных сетей (НС)
3 История развития теории нейронных сетей
4 Задачи, решаемые нейронными сетями
5 Искусственный нейрон
6 Простейший перцептрон
7 Понятие обучения
8 Обучение перцептрона
9 Метод Монте-Карло для обучения нейронных сетей
10 Метод имитации отжига для обучения нейронных сетей
11 Выводы
12 Литература

Искусственный нейрон I

Искусственный нейрон: вход $x \in \mathbb{R}^n$, выход $y \in \mathbb{R}$.

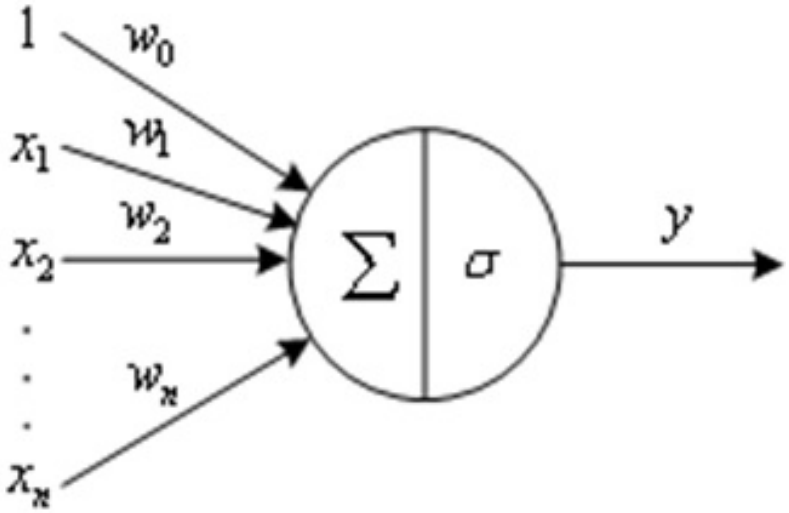


Рис. 16. (Искусственный) нейрон

Искусственный нейрон II

Вычисление выхода:

1. уровень активации (активности) нейрона net:

$$net = w_0 + \sum_{i=1}^{n} w_i x_i, \tag{1}$$

где $\begin{bmatrix} w_0 & w_1 & \dots & w_n \end{bmatrix}^T = w \in \mathbb{R}^{n+1}$ – вектор весов (весовых коэффициентов, параметров) нейрона;

2. функция активации (ФА):

$$y = \sigma(net),$$

где $\sigma(\cdot) : \mathbb{R} \to \mathbb{R}$ – некоторая, как правило, нелинейная функция.

Искусственный нейрон III

Основные ФА $\sigma(\cdot)$:

- пороговая (рис. 17)

$$\sigma(net) = \begin{cases} 1, & net \geqslant T, \\ 0, & net < T, \end{cases}$$

где T – значение порога (threshold level);

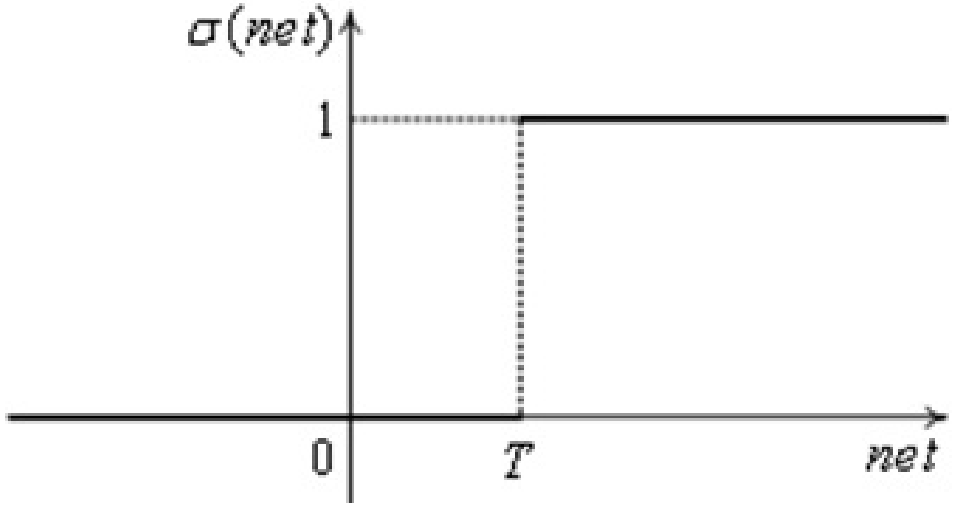


Рис. 17. Пороговая функция

Искусственный нейрон IV

- линейная пороговая (рис. 18)

$$\sigma(net) = \begin{cases} 1, & net \geqslant T_2, \\ \dfrac{net - T_1}{T_2 - T_1}, & T_1 < net < T_2, \\ 0, & net \leqslant T_1, \end{cases}$$

где T_1 и T_2 – пороговые константы;

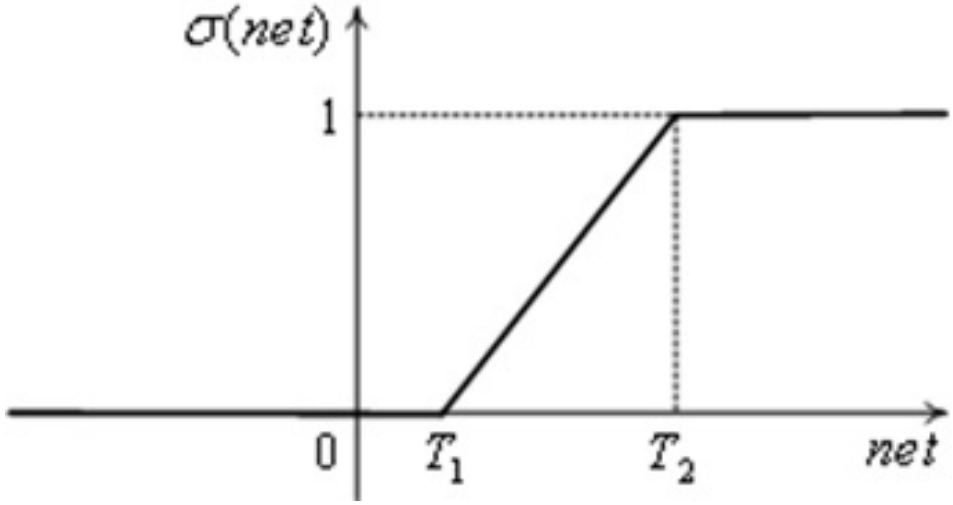


Рис. 18. Линейная пороговая функция

Искусственный нейрон V

- гиперболический тангенс (сигмоидная биполярная) (рис. 19)

$$\sigma(net) = th(net) = \frac{e^{net} - e^{-net}}{e^{net} + e^{-net}};$$

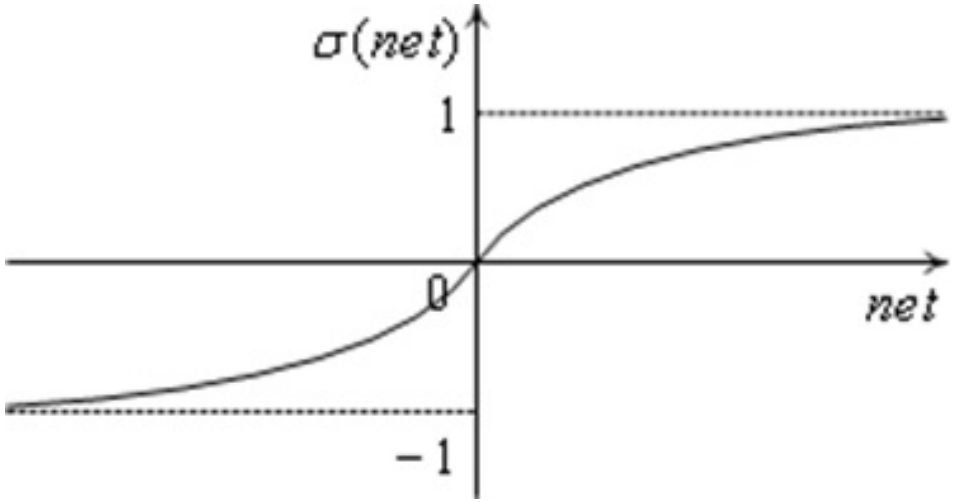


Рис. 19. Гиперболический тангенс

Искусственный нейрон VI

- сигмоидная логистическая (сигмоидная униполярная) (рис. 20)

$$\sigma(net) = \frac{1}{1 + e^{-net}}.$$

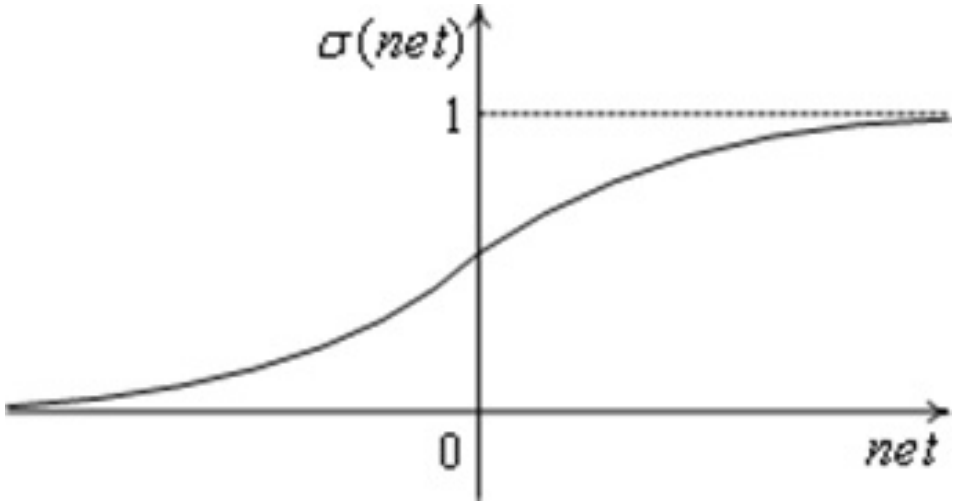


Рис. 20. Сигмоидная логистическая функция

1 Содержание курса
2 Понятие нейронных сетей (НС)
3 История развития теории нейронных сетей
4 Задачи, решаемые нейронными сетями
5 Искусственный нейрон
6 Простейший перцептрон
7 Понятие обучения
8 Обучение перцептрона
9 Метод Монте-Карло для обучения нейронных сетей
10 Метод имитации отжига для обучения нейронных сетей
11 Выводы
12 Литература

Простейший перцептрон I

Перцептрон*: ФА – пороговая

$$\sigma(net) = \begin{cases} 1, & w_1x_1 + w_2x_2 + \ldots + w_nx_n \geqslant T, \\ 0, & w_1x_1 + w_2x_2 + \ldots + w_nx_n < T \end{cases}$$

Пример. Булева функция.

$$y = f(x_1, x_2) = x_1 \vee x_2$$

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

Простейший перцептрон II

Задача: определить веса перцептрона w_1, w_2, порог T, чтобы выдавать правильный ответ для всех входных значений.

$$\sigma(net) = \begin{cases} 1, & w_1 x_1 + w_2 x_2 \geqslant T, \\ 0, & w_1 x_1 + w_2 x_2 < T \end{cases}$$

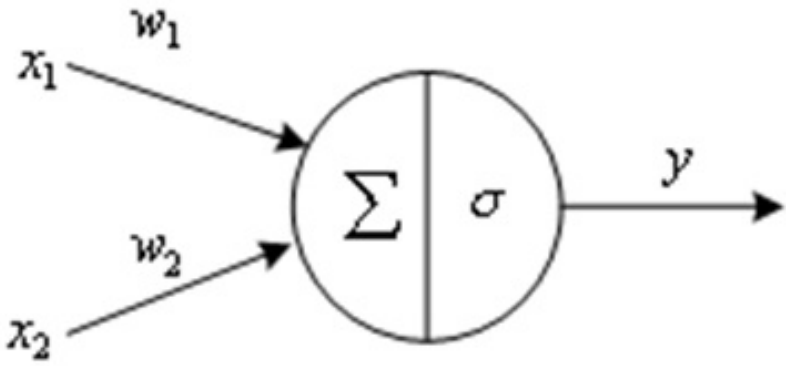


Рис. 21. Перцептрон

Простейший перцептрон III

Бесконечное множество решений.
Вариант решения:

$$w_1 = w_2 = 0,5; \quad T = 0,4$$

Пусть $x_1 = 1$, $x_2 = 0$:

$$net = 0,5 \cdot 1 + 0,5 \cdot 0 = 0,5$$

$$net = 0,5 \geqslant T = 0,4$$

$$y = 1$$

Две полуплоскости – граница:

$$0,5x_1 + 0,5x_2 = 0,4.$$

Простейший перцептрон IV

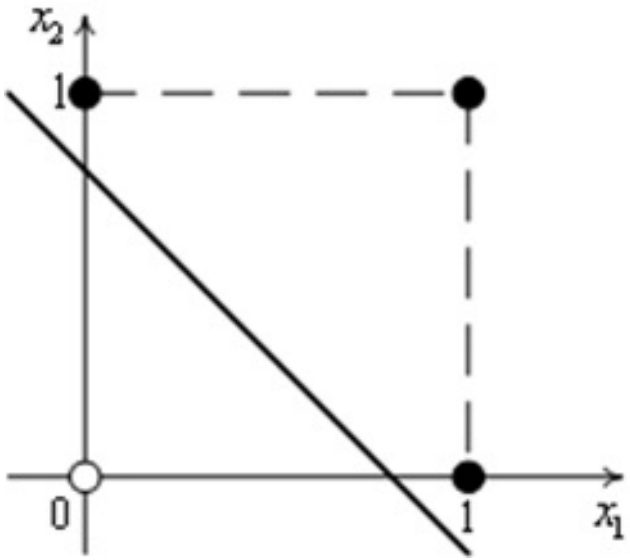


Рис. 22. Две полуплоскости

Унификация записи:

$$w_1 x_1 + w_2 x_2 + \ldots + w_n x_n \geqslant T;$$

$$-1 \cdot T + w_1 x_1 + w_2 x_2 + \ldots + w_n x_n = \sum_{i=0}^{n} w_i x_i = \langle w, x \rangle \geqslant 0;$$

$$x_0 = 1; w_0 = -T$$

Простейший перцептрон V

Задача – линейно разделимая, если существуют веса w_i, т.ч.

$$\langle w, x \rangle \geqslant 0 \quad \forall x \in C_1;$$

$$\langle w, x \rangle < 0 \quad \forall x \in C_2$$

1 Содержание курса
2 Понятие нейронных сетей (НС)
3 История развития теории нейронных сетей
4 Задачи, решаемые нейронными сетями
5 Искусственный нейрон
6 Простейший перцептрон
7 Понятие обучения
8 Обучение перцептрона
9 Метод Монте-Карло для обучения нейронных сетей
10 Метод имитации отжига для обучения нейронных сетей
11 Выводы
12 Литература

Понятие обучения

Обучение – процесс целенаправленного изменения весов НС, позволяющих решить исходную задачу.
Основа обучения – обучающее множество. Входная информация – обучающее множество:

$$\langle \tilde{X}, \tilde{Y} \rangle = \{\tilde{x}_i, \tilde{y}_i\}, i = 1, \ldots, k,$$

где $\tilde{x}_i \in \mathbb{R}^n$ – входной вектор из i-го примера, $\tilde{y}_i \in \mathbb{R}^r$ – вектор соответствующих указаний учителя.
Строки – **примеры**. Выходы – **указания учителя**.
Степень соответствия модели данным обучающего множества – функционал качества обучения:

$$Q = Q(w).$$

Цель обучения:

$$w^* = \arg \min_{w \in \mathbb{R}^s} Q(w).$$

Для линейно разделимых задач применим алгоритм обучения.

1 Содержание курса
2 Понятие нейронных сетей (НС)
3 История развития теории нейронных сетей
4 Задачи, решаемые нейронными сетями
5 Искусственный нейрон
6 Простейший перцептрон
7 Понятие обучения
8 Обучение перцептрона
9 Метод Монте-Карло для обучения нейронных сетей
10 Метод имитации отжига для обучения нейронных сетей
11 Выводы
12 Литература

Обучение перцептрона I

Алгоритм (обучение перцептрона):

1. Задание шага обучения $\eta > 0$, k – количество строк в таблице.
2. Инициализация весов w_i небольшими случайными значениями.
   Ошибка $Q := 0$.
   Номер строки таблицы: $m := 1$.
3. Начало этапа обучения.
   $$x^* := x_m, y^* := y_m$$
   Выход перцептрона:
   $$y = y(x^*).$$
4. Обновление весов:
   $$w := w - \eta(y^* - y)x^*.$$
5. Накопление ошибки:
   $$Q := Q + \|y^* - y\|^2.$$
6. Если $m < k$, то $m := m + 1$ и переход на шаг 3.

Обучение перцептрона II

7. Если $Q = 0$, то прекращение вычислений (решение найдено), иначе $Q := 0$, $m := 1$ и переход на шаг 2.

Существуют линейно неразделимые задачи.
Проблема «исключающего или» (XOR-problem) (Минский и Пейперт):

$$y = f(x_1, x_2) = x_1 \oplus x_2$$

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Обучение перцептрона III

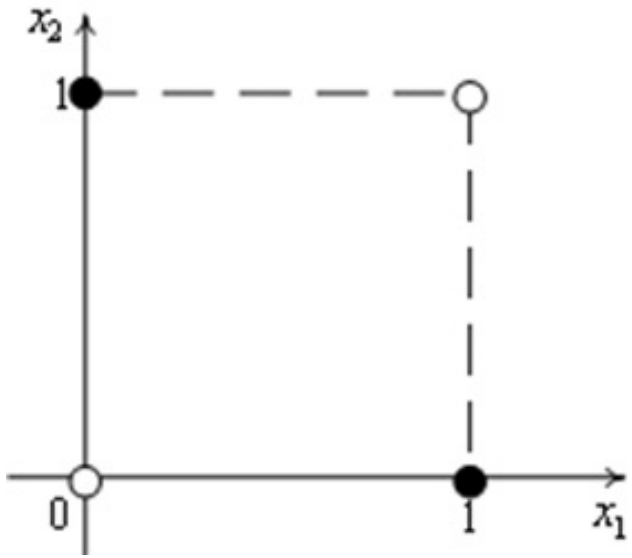


Рис. 23. Проблема «исключающего ИЛИ»

1 Содержание курса
2 Понятие нейронных сетей (НС)
3 История развития теории нейронных сетей
4 Задачи, решаемые нейронными сетями
5 Искусственный нейрон
6 Простейший перцептрон
7 Понятие обучения
8 Обучение перцептрона
9 Метод Монте-Карло для обучения нейронных сетей
10 Метод имитации отжига для обучения нейронных сетей
11 Выводы
12 Литература

Метод Монте-Карло для обучения нейронных сетей I

Алгоритмы стохастической глобальной оптимизации – бесконечные во времени процессы, для которых вероятность нахождения оптимального значения приближается к 1 при стремлении количества итераций к бесконечности. Базовый алгоритм стохастической оптимизации – метод Монте-Карло.

Алгоритм.

Инициализация: $f_{min} := \infty, x_{min} := NaN$ (Not A Number – неопределенность типа (0/0)), N – очень большое целое число – число генерируемых точек, 0. $i := 0$

1. Цикл: повторять пока $i < N$
   1.1. $x_i =$ случайная точка
   1.2. Если $f(x_i) < f_{min}$, то $x_{min} = x_i$, $f_{min} = f(x_i)$
   1.3. $i := i + 1$ и перейти на шаг 1.1.

Если значения функции вычисляются в точках, полученных на основе равномерного распределения области X, наименьшее значение функции сходится к глобальному минимуму с вероятностью 1.

1 Содержание курса
2 Понятие нейронных сетей (НС)
3 История развития теории нейронных сетей
4 Задачи, решаемые нейронными сетями
5 Искусственный нейрон
6 Простейший перцептрон
7 Понятие обучения
8 Обучение перцептрона
9 Метод Монте-Карло для обучения нейронных сетей
10 Метод имитации отжига для обучения нейронных сетей
11 Выводы
12 Литература

Метод имитации отжига для обучения нейронных сетей I

Алгоритм имитации отжига (Simulated Annealing Method) основан на идее, заимствованной из статистической механики. Он отражает поведение расплавленного материала при отвердевании с применением процедуры отжига (управляемого охлаждения) при температуре, последовательно понижаемой до нуля.

В процессе медленного управляемого охлаждения, называемого отжигом, кристаллизация расплава сопровождается глобальным уменьшением его энергии, однако допускаются ситуации, в которых она может на какое-то время возрастать (в частности, при подогреве расплава для предотвращения слишком быстрого его остывания). Благодаря допустимости кратковременного повышения энергетического уровня, возможен выход из ловушек локальных минимумов энергии, которые возникают при реализации процесса. Только понижение температуры до абсолютного нуля делает невозможным какое-либо самостоятельное повышение энергетического уровня расплава.

Метод имитации отжига для обучения нейронных сетей II

Алгоритм

Инициализация:

$T := T_{max} > 0$ – максимальная температура (большое вещественное число)

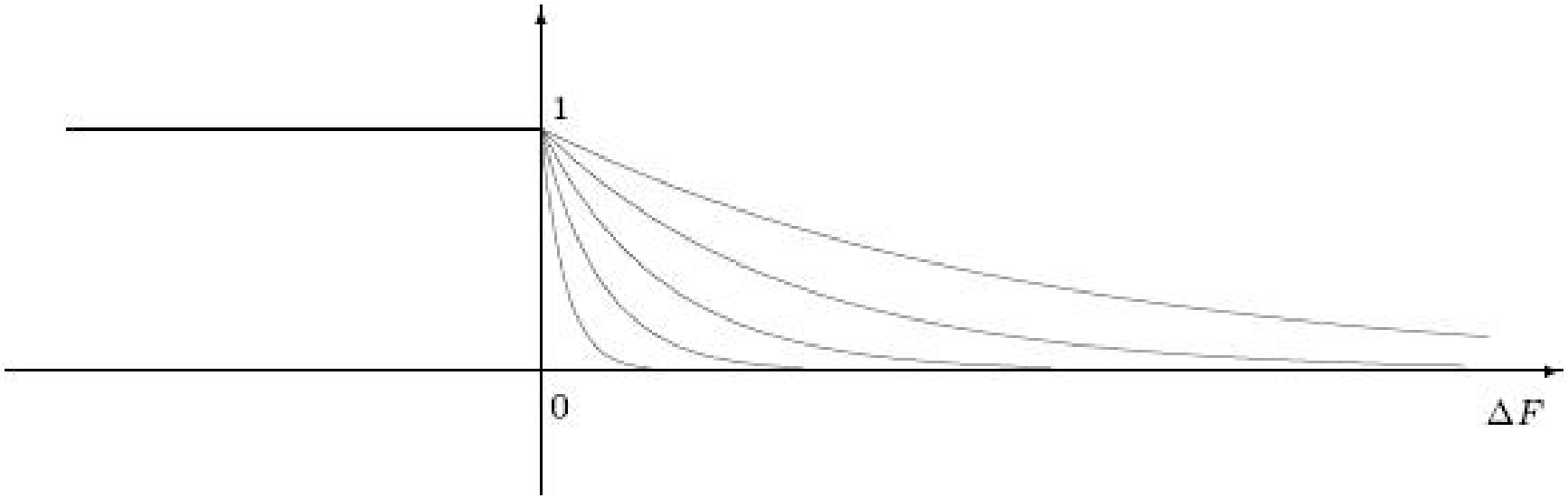
L – количество циклов для каждой температуры (целое число)

r из интервала $(0; 1)$ – параметр снижения температуры (вещественное число)

$\epsilon > 0$ – малое вещественное число (например, 10^{-10})

1. Выбрать случайную точку x
2. Пока $T > 0$ повторять L раз следующие действия:
   2.1. Выбрать новую точку x' из ε-окрестности точки x
   2.2. Рассчитать изменение целевой функции $\Delta = f(x') - f(x)$
      Если $\Delta \leq 0$, то $x := x'$ иначе
      Если $e^{-\Delta/T} >$ случайного числа, р/р на интервале $(0; 1)$, то $x := x'$
3. Уменьшить температуру $T := rT$. Вернуться к пункту 2.
4. Провести оптимизацию любым методом локальной оптимизации.

Метод имитации отжига для обучения нейронных сетей III



1 Содержание курса
2 Понятие нейронных сетей (НС)
3 История развития теории нейронных сетей
4 Задачи, решаемые нейронными сетями
5 Искусственный нейрон
6 Простейший перцептрон
7 Понятие обучения
8 Обучение перцептрона
9 Метод Монте-Карло для обучения нейронных сетей
10 Метод имитации отжига для обучения нейронных сетей
11 Выводы
12 Литература

ВЫВОДЫ (ОСНОВНЫЕ МОМЕНТЫ ЛЕКЦИИ)

1. Нейронные сети – вид математических моделей определенной структуры.
2. Искусственный нейрон – аналог биологического нейрона. Нейрон – основной элемент нейронных сетей.
3. Преобразование нейрона осуществляется в два этапа: вычисляется уровень активности, затем применяется функция активации.
4. Обучение – целенаправленная настройка весов НС.
5. Существуют линейно разделимые и линейно неразделимые задачи. Для линейно разделимых задач применим алгоритм обучения перцептрона.

1 Содержание курса
2 Понятие нейронных сетей (НС)
3 История развития теории нейронных сетей
4 Задачи, решаемые нейронными сетями
5 Искусственный нейрон
6 Простейший перцептрон
7 Понятие обучения
8 Обучение перцептрона
9 Метод Монте-Карло для обучения нейронных сетей
10 Метод имитации отжига для обучения нейронных сетей
11 Выводы
12 Литература

ЛИТЕРАТУРА

1. Кириченко А.А. Основы теории искусственных нейронных сетей: Издательская система Ridero, 2020.– 231 с.
2. Блюмин С.Л., Шуйкова И.А., Сараев П.В., Черпаков И.В. Нечеткая логика: алгебраические основы и приложения. Липецк: ЛЭГИ, 2002.– 107с.
3. Осовский С. Нейронные сети для обработки информации.– М.: Финансы и статистика, 2004.– 344 с.
4. Сараев П.В. Идентификация нейросетевых моделей: Монография / Липецк: ЛГТУ, 2011.– 94 с.
5. Хайкин С. Нейронные сети: полный курс.– М.: Вильямс, 2006.– 1104 с.
6. Fuller R. Introduction to Neuro-Fuzzy Systems.– Berlin/Heildelberg, Springer-Verlag, 2000.– 289 p.

1 Содержание курса
2 Понятие нейронных сетей (НС)
3 История развития теории нейронных сетей
4 Задачи, решаемые нейронными сетями
5 Искусственный нейрон
6 Простейший перцептрон
7 Понятие обучения
8 Обучение перцептрона
9 Метод Монте-Карло для обучения нейронных сетей
10 Метод имитации отжига для обучения нейронных сетей
11 Выводы
12 Литература

СПАСИБО ЗА ВНИМАНИЕ!

Построение и применение нейронных сетей и генетических алгоритмов
Лекция 2
Многослойные перцептроны и процедура обратного распространения ошибки

САРАЕВ ПАВЕЛ ВИКТОРОВИЧ

РТУ МИРЭА
Кафедра прикладной математики

Москва – 2026

ПЛАН ЛЕКЦИИ

1. Многослойный перцептрон (нейронные сети прямого распространения)
2. Универсальные аппроксимационные способности нейронных сетей
3. Архитектура нейронных сетей с радиальными базисными функциями
4. Методика применения нейронных сетей
5. Обучение нейронных сетей как задача оптимизации
6. Процедура обратного распространения ошибки
7. Поиск глобального минимума при обучении
8. Определение оптимальной структуры многослойного перцептрона
9. Выводы
10. Литература

1 Многослойный перцептрон (нейронные сети прямого распространения)
2 Универсальные аппроксимационные способности нейронных сетей
3 Архитектура нейронных сетей с радиальными базисными функциями
4 Методика применения нейронных сетей
5 Обучение нейронных сетей как задача оптимизации
6 Процедура обратного распространения ошибки
7 Поиск глобального минимума при обучении
8 Определение оптимальной структуры многослойного перцептрона
9 Выводы
10 Литература

Многослойный перцептрон I

Многослойный перцептрон = Нейронная сеть прямого распространения (НС ПР)

- Входной слой
- Скрытые (промежуточные) слои
- Выходной слой

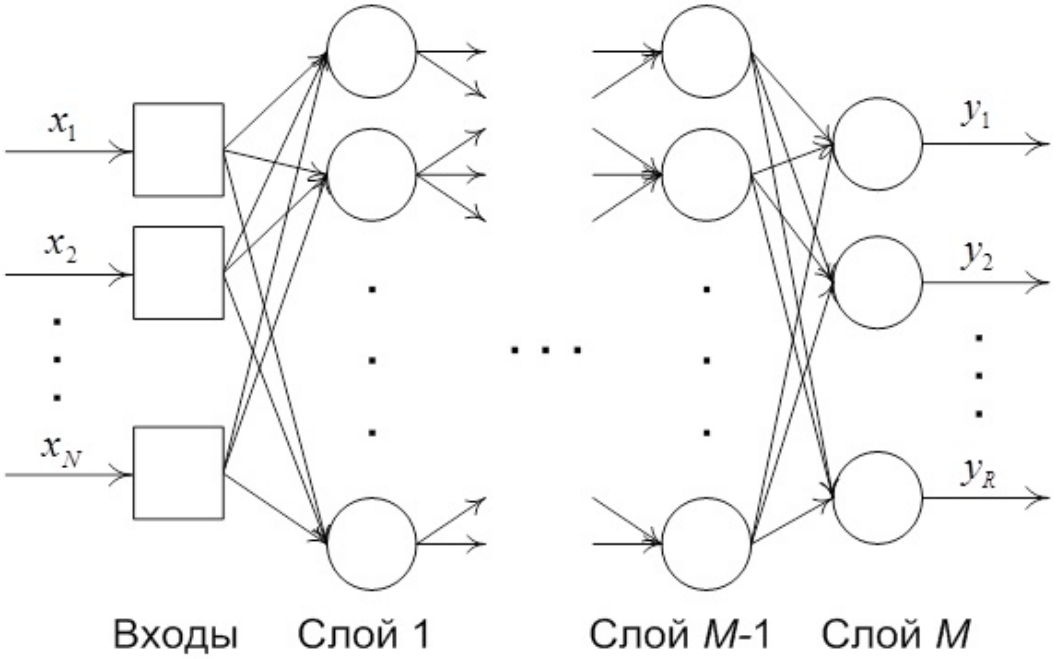


Рис. 1. Структура многослойного перцептрона

Многослойный перцептрон II

Выход q-го нейрона m-го слоя:

$$y^{(m,q)} = \sigma^{(m,q)}\left(net^{(m,q)}\right) = \sigma^{(m,q)}\left(\sum_{i=0}^{n} w_i^{(m,q)} y^{(m-1,i)}\right), \qquad (1)$$

где $y^{(m-1,0)} = 1$, $m = 1, \ldots, M$, – единичные входы нейронов; $y^{(0,i)} = x_i$, $i = 1, \ldots, n$, – входы НС; $\sigma^{(m,q)}$ – ФА q-го нейрона m-го слоя.

Выход нейронной сети:

$$y(x; w) = y^{(M)} = W^{(M)}\Phi^{(M-1)}\left(W^{(M-1)}\Phi^{(M-2)}\left(\ldots\Phi^{(1)}\left(W^{(1)}x\right)\ldots\right)\right) -$$

суперпозиционная линейно-нелинейная по параметрам структура.
Здесь:

- $W^{(i)}$ – матрица, составленная из векторов весов нейронов i-го слоя, записанных построчно;
- $\Phi^{(i)}$ – вектор функций нейронов i-го слоя – векторная функция векторного аргумента (ВФВА).

Многослойный перцептрон III

Пример.

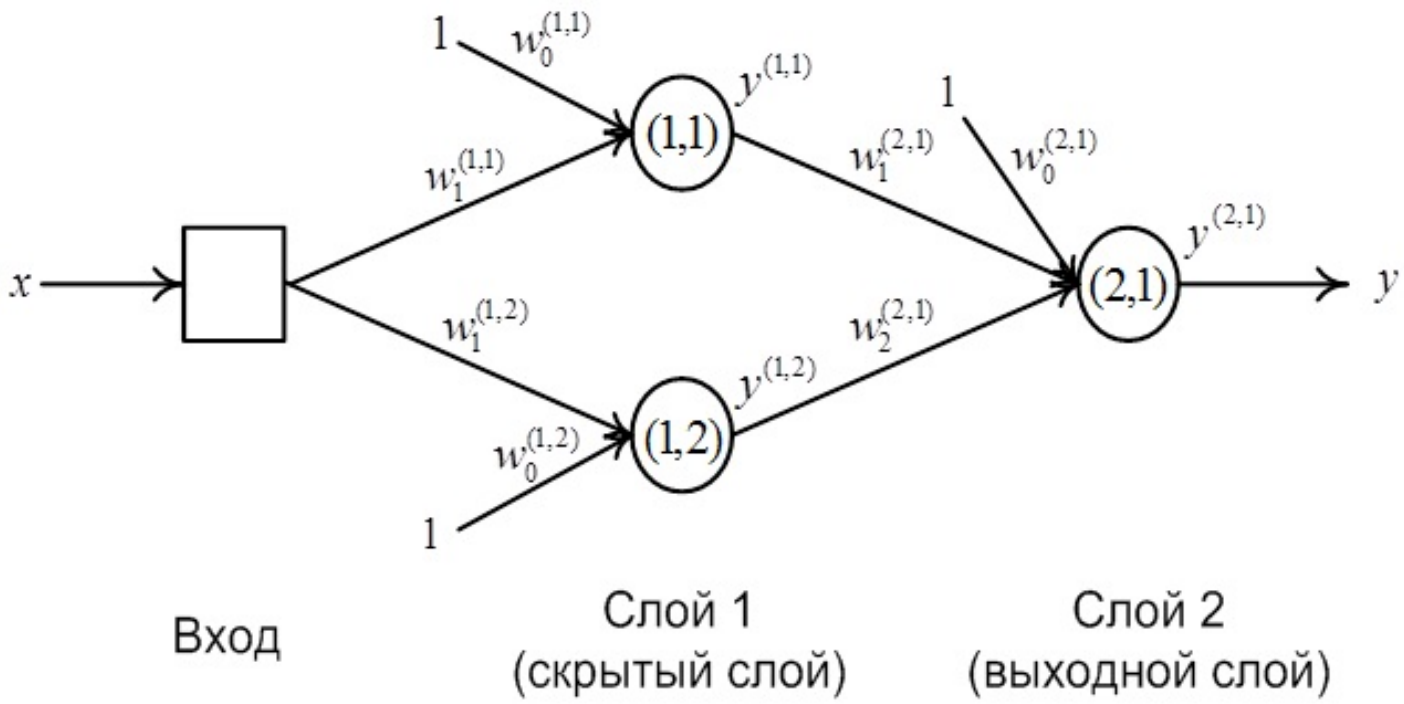


Рис. 2. Пример многослойного перцептрона

Многослойный перцептрон IV

Простейшая НС:

$$y = w_0 + \sum_{i=1}^{n} w_i x_i$$

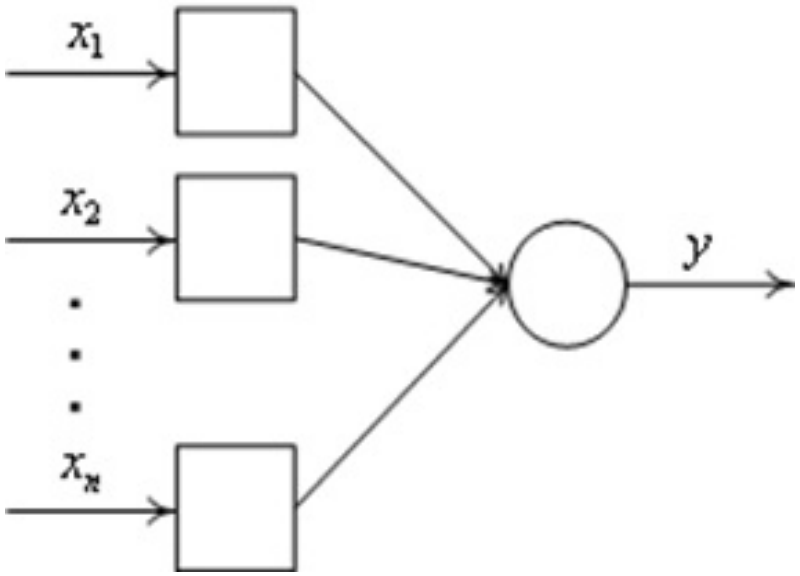


Рис. 3. Простейший многослойный перцептрон

Многослойный перцептрон V

Двухслойный перцептрон (стандартная НС ПР, полутораслойный предиктор):

$$y = \sum_{i=1}^{q} w_i \sigma \left(\sum_{j=1}^{n} w_{ij} x_j \right)$$

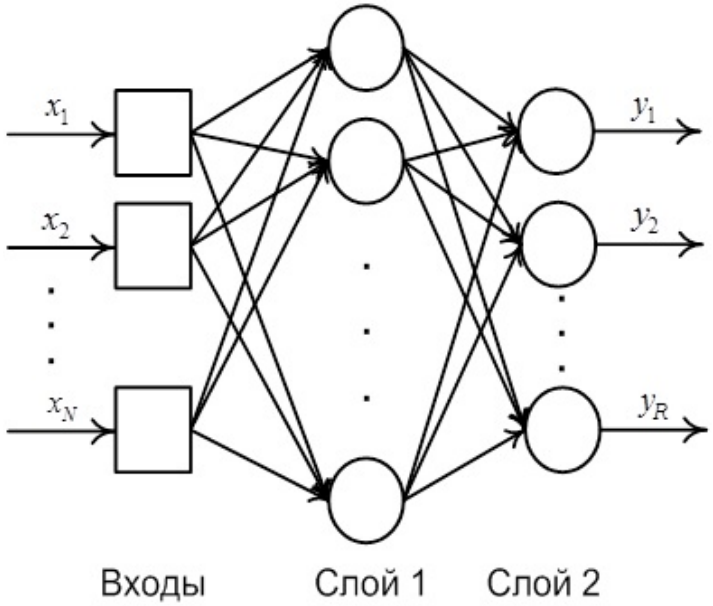


Рис. 4. Двухслойный перцептрон

1 Многослойный перцептрон (нейронные сети прямого распространения)

2 Универсальные аппроксимационные способности нейронных сетей

3 Архитектура нейронных сетей с радиальными базисными функциями

4 Методика применения нейронных сетей

5 Обучение нейронных сетей как задача оптимизации

6 Процедура обратного распространения ошибки

7 Поиск глобального минимума при обучении

8 Определение оптимальной структуры многослойного перцептрона

9 Выводы

10 Литература

Универсальные аппроксимационные способности НС

- Теорема Вейерштрасса.
- Теорема Стоуна-Вейерштрасса.
- Теоремы А.Н. Колмогорова и В.И. Арнольда.
- Теоремы об универсальных аппроксимационных способностях НС.

Теорема (Фунахаши) об универсальных аппроксимационных способностях многослойных перцептронов.

Пусть $f(x_1, x_2, \ldots, x_n)$ – непрерывная функция, определенная на компактном множестве, и $\varepsilon > 0$ – точность аппроксимации. Существует натуральное число q и набор действительных чисел w_{ij}, w_i, что функция

$$y = \sum_{i=1}^{q} w_i \sigma \left(\sum_{j=1}^{n} w_{ij} x_j \right),$$

где σ – непостоянная ограниченная монотонно возрастающая непрерывная функция (например, сигмоидная логистическая), приближает исходную функцию с погрешностью, не превышающей ε на всей области определения, т.е.

$$\sup_{(x_1, x_2, \ldots, x_n) \in \mathbb{R}^n} |f(x_1, x_2, \ldots, x_n) - y| \leqslant \varepsilon.$$

1 Многослойный перцептрон (нейронные сети прямого распространения)

2 Универсальные аппроксимационные способности нейронных сетей

3 Архитектура нейронных сетей с радиальными базисными функциями

4 Методика применения нейронных сетей

5 Обучение нейронных сетей как задача оптимизации

6 Процедура обратного распространения ошибки

7 Поиск глобального минимума при обучении

8 Определение оптимальной структуры многослойного перцептрона

9 Выводы

10 Литература

Архитектура нейронных сетей с радиальными базисными функциями

Один из классов НС – НС с радиальными базисными функциями (RBF-сети). Это многослойные перцептроны с 1 скрытым слоем.
Функция активации в нейронах скрытого слоя –функции гауссовского типа:

$$\sigma(net) = e^{-\frac{(net-m)^2}{2s^2}}, \tag{2}$$

где m и s – параметры функции.
Обычно в качестве (2) –

$$\sigma(net) = e^{-net^2}.$$

К недостаткам моделей данного типа относят то, что для их применения в моделировании требуется большое количество нейронов скрытого слоя.

1 Многослойный перцептрон (нейронные сети прямого распространения)

2 Универсальные аппроксимационные способности нейронных сетей

3 Архитектура нейронных сетей с радиальными базисными функциями

4 Методика применения нейронных сетей

5 Обучение нейронных сетей как задача оптимизации

6 Процедура обратного распространения ошибки

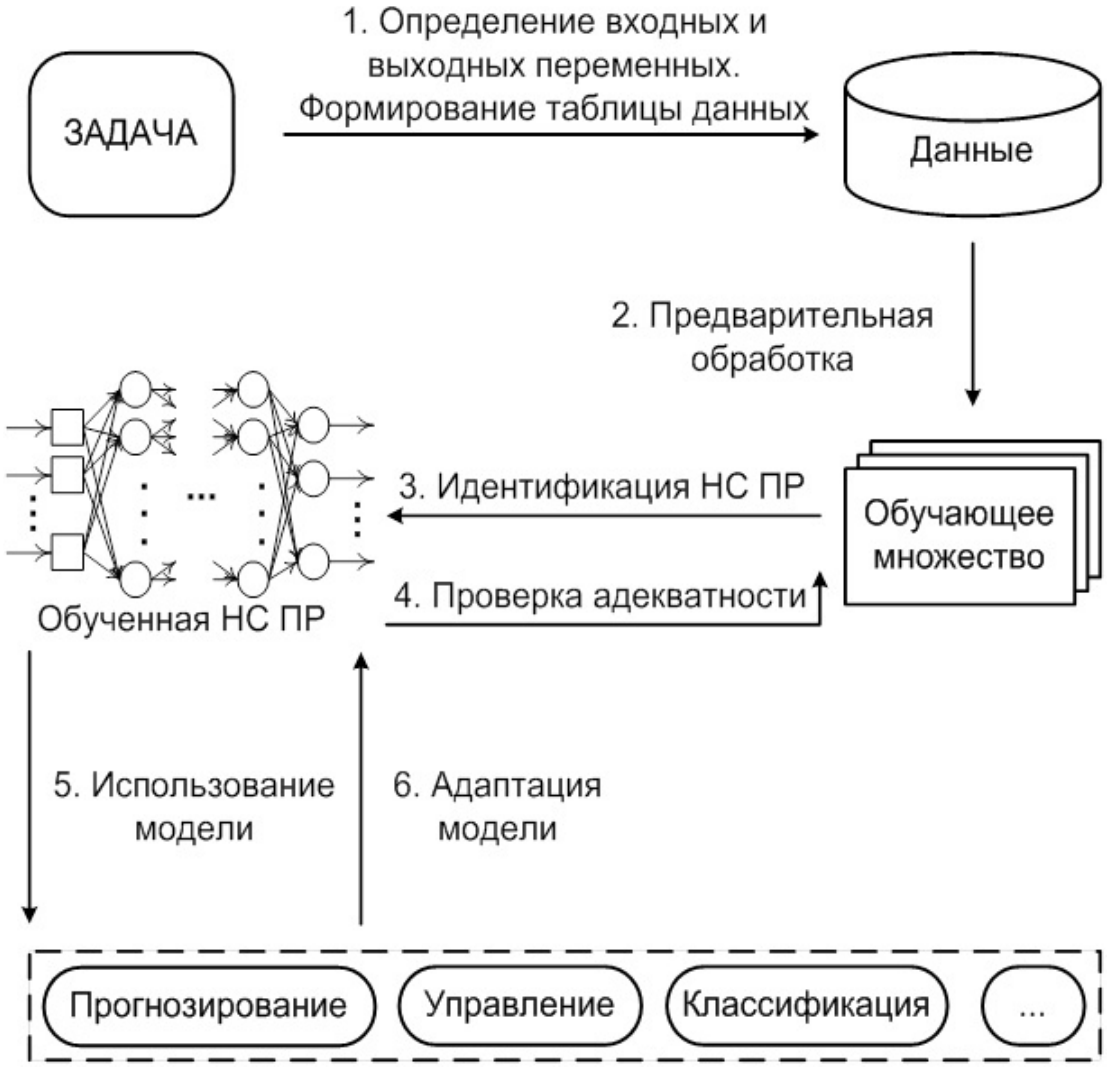
7 Поиск глобального минимума при обучении

8 Определение оптимальной структуры многослойного перцептрона

9 Выводы

10 Литература

Методика применения нейронных сетей I



Методика применения нейронных сетей II

Этап 1. Информативное множество входных x и выходных y величин выбирается, исходя из поставленной задачи.

Этап 2. Преобразование данных, заданных в качественном, к числовому виду.
Восстановление отсутствующих данных в обучающем множестве (восстановление пробелов).
Удаление неестественных, ошибочных, сильно искаженных данных, не отражающих реального поведения объекта.
Переход от абсолютных значений к относительным.
Нормировка значений переменных в диапазон $[0;1]$ или $[-1;1]$.

Этап 3. Идентификация (построение) нейросетевой модели – центральный этап.

Этап 4. Основной подход: выделение из исходной таблицы вход-выходных данных тестового множества (около 20%). Остальная часть – обучающее множество. Тестовое множество состоит из данных, не использующихся в процессе обучения. Более адекватна модель, которая показывает меньшую ошибку на тестовом множестве.

Методика применения нейронных сетей III

Этап 5. Прогнозирование - получение выходных величин для заданных входных значений.
Управление – формирование управляющих сигналов с целью получения желаемых выходов управляемой системы.
Классификация – соотнесение входного вектора, описывающего объект, к определенному классу объектов.

Этап 6. При использовании модели с течением времени встает необходимость учета новой информации о предметной области, рассматриваемой в задаче, например, связанная с поступлением новой вход-выходной информации.

1 Многослойный перцептрон (нейронные сети прямого распространения)

2 Универсальные аппроксимационные способности нейронных сетей

3 Архитектура нейронных сетей с радиальными базисными функциями

4 Методика применения нейронных сетей

5 Обучение нейронных сетей как задача оптимизации

6 Процедура обратного распространения ошибки

7 Поиск глобального минимума при обучении

8 Определение оптимальной структуры многослойного перцептрона

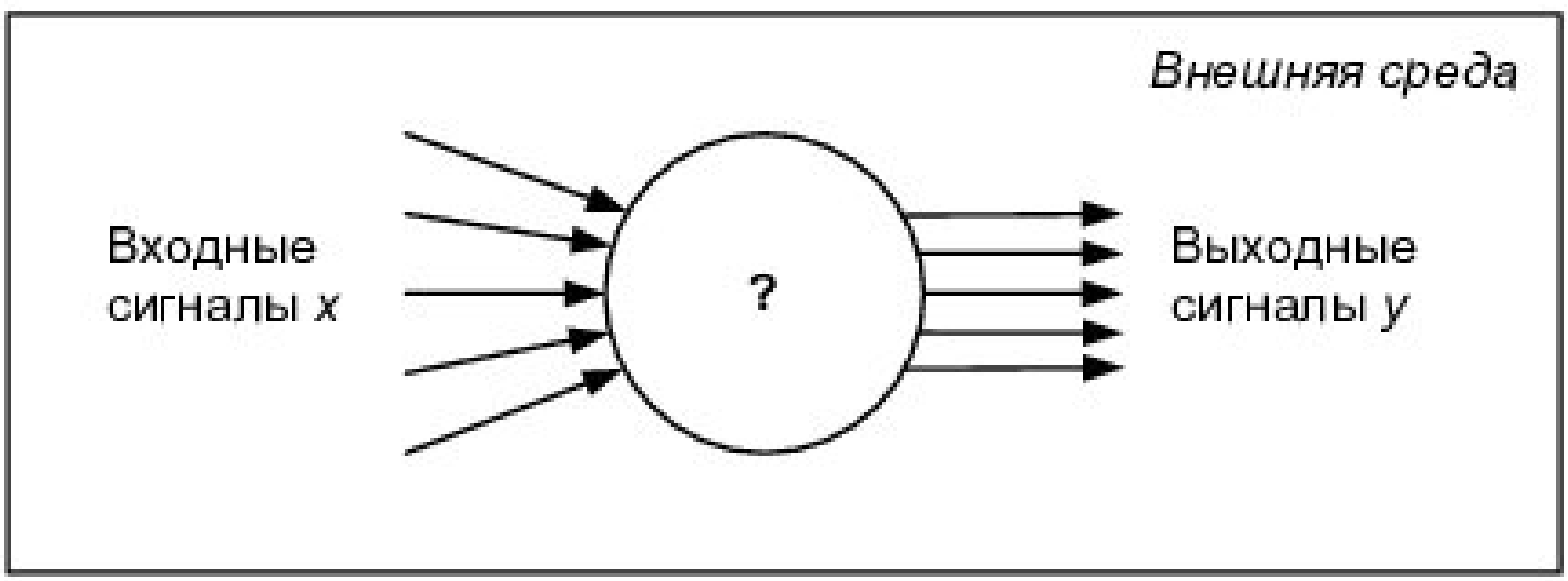
9 Выводы

10 Литература

Обучение нейронных сетей как задача оптимизации I

«Черный ящик» (black box) –

система, преобразующая входные сигналы в выходные по неизвестному закону:



Входная информация – обучающее множество:

$$\left[\begin{array}{cccc|cccc} \tilde{x}_{11} & \tilde{x}_{12} & \dots & \tilde{x}_{1n} & \tilde{y}_{11} & \tilde{y}_{12} & \dots & \tilde{y}_{1r} \\ \tilde{x}_{21} & \tilde{x}_{22} & \dots & \tilde{x}_{2n} & \tilde{y}_{21} & \tilde{y}_{22} & \dots & \tilde{y}_{2r} \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ \tilde{x}_{k1} & \tilde{x}_{k2} & \dots & \tilde{x}_{kn} & \tilde{y}_{k1} & \tilde{y}_{k2} & \dots & \tilde{y}_{kr} \end{array}\right]$$

Обучение нейронных сетей как задача оптимизации II

Здесь k – количество примеров, n и r – количество входов и выходов соответственно.

Краткая запись:

$$\langle \tilde{X}, \tilde{Y} \rangle = \{\tilde{x}_i, \tilde{y}_i\}, i = 1, \ldots, k,$$

где $\tilde{x}_i \in \mathbb{R}^n$ – входной вектор из i-го примера, $\tilde{y}_i \in \mathbb{R}^r$ – вектор соответствующих указаний учителя.

Строки – **примеры**. Выходы – **указания учителя**. Далее считаем: $r = 1$.
В данном контексте: **обучение с учителем**.

Этапы построения (**идентификации**) формальных математических моделей:

- структурная (определяется многообразие/класс моделей, зависящих от параметров w);
- параметрическая (в терминологии машинного обучения и нейронных сетей: **обучение**).

Обучение нейронных сетей как задача оптимизации III

Отклонение выхода модели сети y_i на i-м примере и соответствующего указания учителя $\tilde{y}_i$ при текущем входном векторе $x_i \in \mathbb{R}^n$ (**невязка, residual**):

$$\varepsilon_i(w) = y_i(w) - \tilde{y}_i.$$

Вектор невязок:

$$\varepsilon = \varepsilon(w) = y(w) - \tilde{y}.$$

Пусть известные значения вектора параметров $w \in \mathbb{R}^s$. Мгновенный функционал качества для примера i:

$$Q_i = Q_i(w) = Q(\varepsilon_i(w)).$$

Степень соответствия модели данным обучающего множества: интегральный функционал качества обучения:

$$Q = Q(w) = \sum_{i=1}^{k} Q_i(w)$$

Цель обучения:

$$w^* = \arg\min_{w \in \mathbb{R}^s} Q(w).$$

Обучение нейронных сетей как задача оптимизации IV

Обучение – задача параметрической идентификации – **оптимизационная задача**.
Q – обычно норма вектора невязок ε:

$$Q = \|\varepsilon\|.$$

Наиболее известно семейство норм Гёльдера l_p, которые определяются как

$$\|\varepsilon\|_p = \left(\sum_{i=1}^{k} |\varepsilon_i|^p \right)^{\frac{1}{p}}.$$

Формула может применяться для $0 < p \leqslant \infty$. Наиболее часто используются:

- l_1-норма (робастная, не чувствительная к «выбросам»):

$$\|\varepsilon\|_1 = \left(\sum_{i=1}^{k} |\varepsilon_i| \right),$$

Обучение нейронных сетей как задача оптимизации V

- l_2-норма (традиционная):

$$\|\varepsilon\|_2 = \left(\sum_{i=1}^{k} |\varepsilon_i|^2\right)^{\frac{1}{2}},$$

- l_∞-норма (равномерная, все решения в δ-трубке):

$$\|\varepsilon\|_\infty = \max_{i=1}^{k} |\varepsilon_i|.$$

В общем случае, с ростом номера p норма вектора может только уменьшиться. l_∞-норма – минимальная из норм.

Обучение нейронных сетей как задача оптимизации VI

Взвешенные нормы: $v_i > 0$ – веса, отражающие значимость примеров.

- l_1-норма (робастная, не чувствительная к «выбросам»):

$$\|\varepsilon\|_1 = \left(\sum_{i=1}^{k} v_i|\varepsilon_i|\right),$$

- l_2-норма (традиционная):

$$\|\varepsilon\|_2 = \left(\sum_{i=1}^{k} v_i|\varepsilon_i|^2\right)^{\frac{1}{2}},$$

- l_∞-норма (равномерная, все решения в δ-трубке):

$$\|\varepsilon\|_\infty = \max_{i=1}^{k} v_i|\varepsilon_i|.$$

Обучение нейронных сетей как задача оптимизации VII

Норма, независимо от вида, необходима для **возможности сравнения** различных вариантов решений задачи параметрической идентификации. Лучший вариант из двух векторов w_1 и w_2 – обеспечивающий меньшее значение нормы векторов невязок: $Q(w_1)$ или $Q(w_2)$.

Замечание:

вектор, обеспечивающий минимум одной нормы, в общем случае может не приводить к минимуму при использовании других норм.

Далее: 2-норма

Обучение нейронных сетей как задача оптимизации VIII

Оптимизация (минимизация) функции одной переменной:

$$y = f(x) \to \min$$

Необходимое условие оптимума (НУО):

$$y' = f'(x) = 0$$

Достаточное условие оптимума (ДУО):

$$y'' = f''(x) > 0(\min)$$

$$y'' = f''(x) < 0(\max)$$

Обучение нейронных сетей как задача оптимизации IX

Пример

$$y = x^2$$

НУО:

$$y' = 2x = 0 \Rightarrow x^* = 0$$

ДУО:

$$y'' = 2 > 0 \Rightarrow x^* = 0 \Rightarrow \min$$

Оптимизация (минимизация) функции нескольких переменных

$$y = f(x_1, x_2, \ldots, x_n)$$

Необходимое условие оптимума (НУО):

$$\nabla f(x) = grad \quad f(x) = \begin{bmatrix} f'_{x_1} \\ f'_{x_2} \\ \vdots \\ f'_{x_n} \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} = 0$$

Обучение нейронных сетей как задача оптимизации X

Достаточное условие оптимума (ДУО):

$$\nabla^2 f(x) = Hessian \quad f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & & & \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

- положительно определен – т. min;
- отрицательно определен – т. max.

Обучение нейронных сетей как задача оптимизации XI

Пример

$$y = x_1^2 + 3x_2^2$$

НУО:

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 \\ 6x_2 \end{bmatrix} = 0$$

$$x_1^* = 0, \quad x_2^* = 0$$

ДУО:

$$\nabla^2 f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} \end{bmatrix} |_{x_1=0, \quad x_2=0} = \begin{bmatrix} 2 & 0 \\ 0 & 6 \end{bmatrix}$$

Матрица положительно определена (по критерию Сильвестра: $2 > 0$, $2 * 6 - 0 = 12 > 0$) – т. min

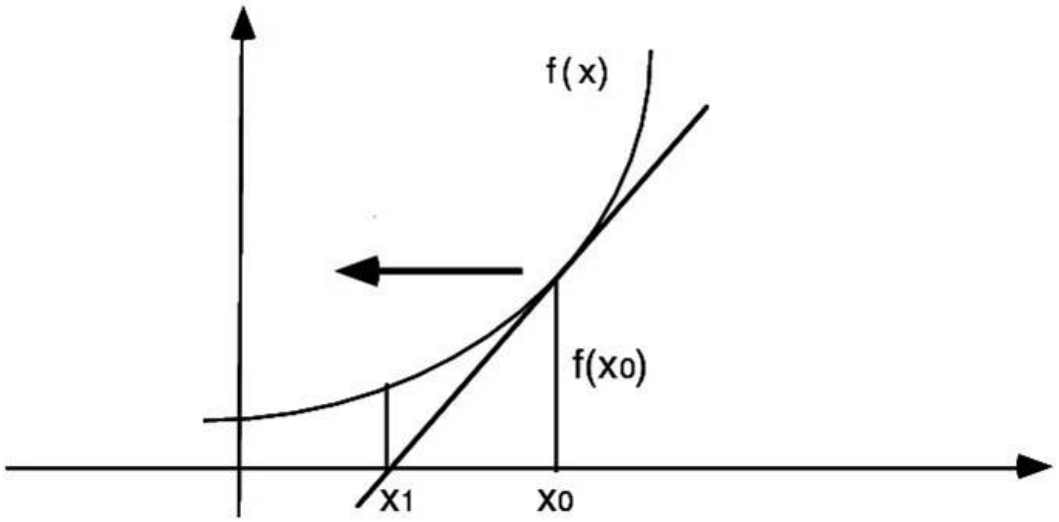
Обучение нейронных сетей как задача оптимизации XII

Если для функции одной переменной

$$f'(x) = g(x) = 0$$

не решается аналитически, применяются численные методы решения нелинейных уравнений (метод Ньютона и др.):

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}, \quad k = 1, 2, \ldots$$



Обучение нейронных сетей как задача оптимизации XIII

Это типичный итерационный метод, идея которого – переход к x_{k+1}:

$$f(x_{k+1}) < f(x_k), \quad k = 1, 2, \ldots$$

Метод наискорейшего спуска (производная – направление возрастания функции):

$$x_{k+1} = x_k - \eta f'(x_k), \quad \eta > 0,$$

где η – шаг оптимизации – достаточно мал.
В частности:

$$\eta = \frac{1}{f''(x_k)}, \quad f''(x_k) > 0$$

Обучение нейронных сетей как задача оптимизации XIV

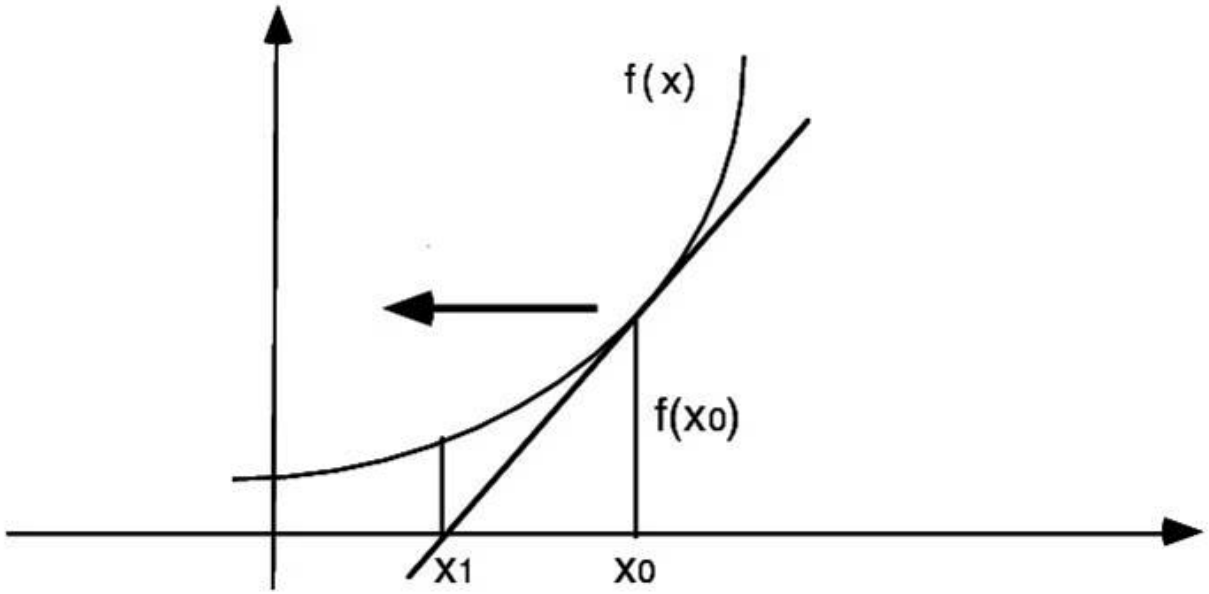


Рис. 5. Метод наискорейшего спуска

Обучение нейронных сетей как задача оптимизации XV

Функция нескольких переменных: направление возрастания функции – градиент.
Метод наискорейшего спуска:

$$x_{k+1} = x_k - \eta \nabla f(x_k), \quad \eta > 0,$$

Критерий останова:

$$\|f(x_{k+1}) - f(x_k)\| < \varepsilon,$$

где $\varepsilon > 0$ – малое число – точность.

1 Многослойный перцептрон (нейронные сети прямого распространения)
2 Универсальные аппроксимационные способности нейронных сетей
3 Архитектура нейронных сетей с радиальными базисными функциями
4 Методика применения нейронных сетей
5 Обучение нейронных сетей как задача оптимизации
6 Процедура обратного распространения ошибки
7 Поиск глобального минимума при обучении
8 Определение оптимальной структуры многослойного перцептрона
9 Выводы
10 Литература

Процедура обратного распространения ошибки I

Эффективный способ нахождения градиента – процедура **обратного распространения ошибки** (ОРО) (**error backpropagation**).

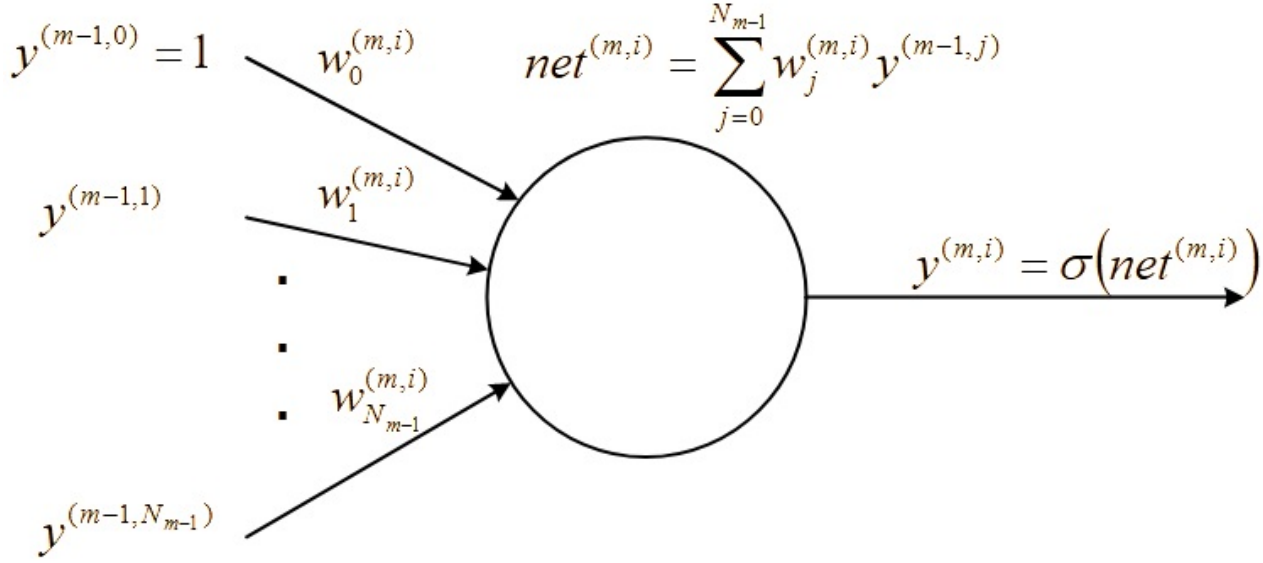
Вследствие аддитивности производной:

$$\nabla_w Q(w) = \nabla_w \left(\sum_{i=1}^{k} Q_i(w) \right) = \sum_{i=1}^{k} \nabla_w Q_i(w)$$

достаточно вывести формулу вычисления градиента для мгновенного функционала качества.

Процедура обратного распространения ошибки II

i-й нейрон m-го слоя:



$$\nabla_w Q(w) = \begin{bmatrix} \vdots \\ \dfrac{\partial Q_i(w)}{\partial w_j^{(m,i)}} \\ \vdots \end{bmatrix}$$

Процедура обратного распространения ошибки III

Производная суперпозиции функций:

$$[f(g(x))]'_x = f'_g(g(x)) \cdot g'(x)$$

$$[f(g_1(x), g_2(x), \ldots, g_n(x))]'_x = \sum_{i=1}^{n} \frac{\partial f}{\partial g_i(x)} \cdot g'_i(x)$$

Процедура обратного распространения ошибки IV

Обозначим $Q_i(w)$ через $Q(w)$.

$$\frac{\partial Q(w)}{\partial w_j^{(m,i)}} = \frac{\partial Q(w)}{\partial y^{(m,i)}} \cdot \frac{\partial y^{(m,i)}}{\partial net^{(m,i)}} \cdot \frac{\partial net^{(m,i)}}{\partial w_j^{(m,i)}}, \tag{3}$$

где последние два множителя определяются легко:

$$\frac{\partial y^{(m,i)}}{\partial net^{(m,i)}} = \sigma'_{net}(net^{(m,i)}), \tag{4}$$

$$\frac{\partial net^{(m,i)}}{\partial w_j^{(m,i)}} = y^{(m-1,j)}. \tag{5}$$

Для нахождения первого множителя формулы обозначим его как

$$s^{(m,i)} = \frac{\partial Q(w)}{\partial y^{(m,i)}}.$$

Процедура обратного распространения ошибки V

Нахождение $s^{(m,i)}$ опирается на рекуррентную процедуру, которая получается из формулы производной сложной функции нескольких переменных:

$$s^{(m,i)} = \sum_{j=1}^{N_{m+1}} \frac{\partial Q}{\partial y^{(m+1,j)}} \frac{\partial y^{(m+1,j)}}{\partial y^{(m,i)}} = \sum_{j=1}^{N_{m+1}} s^{(m+1,j)} \sigma'_{net}(net^{(m+1,j)}) w_i^{(m+1,j)}. \quad (6)$$

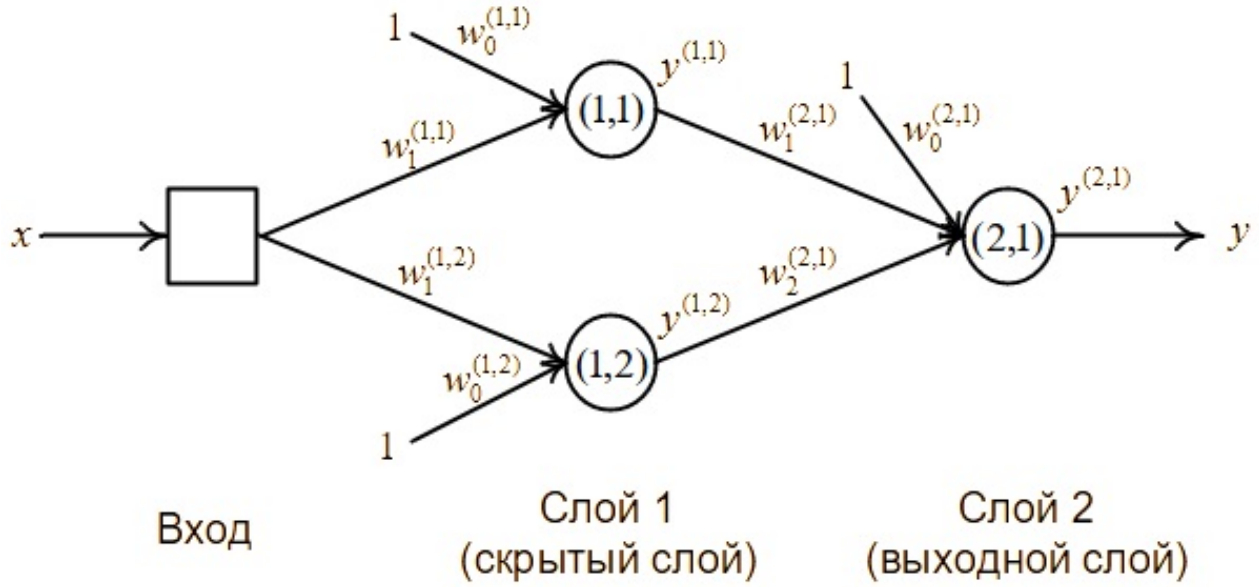
Для вычислений по формуле (6) необходимо начальное условие:

$$s^{(M,1)} = \frac{\partial Q(w)}{\partial y^{(M,1)}} = \varepsilon(w) = y(w) - \tilde{y}, \quad (7)$$

где $\tilde{y}$ – указание учителя текущего примера. Ошибка работы сети $\varepsilon(w)$ словно распространяется в направлении, обратном функционированию сети, что и обусловило такое название алгоритма нахождения градиента.

Процедура обратного распространения ошибки VI

Пример



$$w = \begin{bmatrix} w_0^{(1,1)} \\ w_1^{(1,1)} \\ w_0^{(1,2)} \\ w_1^{(1,2)} \\ w_0^{(2,1)} \\ w_1^{(2,1)} \\ w_2^{(2,1)} \end{bmatrix}$$

Процедура обратного распространения ошибки VII

$$y = y^{(2,1)} = w_0^{(2,1)} + w_1^{(2,1)} y^{(1,1)} + w_0^{(2,1)} y^{(1,1)}$$

Для последнего слоя:

$$s^{(2,1)} = \frac{\partial Q}{\partial y^{(2,1)}} = y(w) - \tilde{y}$$

Для скрытого слоя:

$$s^{(1,1)} = s^{(2,1)} \frac{\partial y^{(2,1)}}{\partial y^{(1,1)}} = s^{(2,1)} w_1^{(2,1)}$$

$$s^{(1,2)} = s^{(2,1)} \frac{\partial y^{(2,1)}}{\partial y^{(1,2)}} = s^{(2,1)} w_2^{(2,1)}$$

Далее:

$$\frac{\partial Q(w)}{\partial w_0^{(1,1)}} = s^{(1,1)} \cdot \frac{\partial y^{(1,i)}}{\partial net^{(1,i)}} \cdot \frac{\partial net^{(1,i)}}{\partial w_0^{(1,i)}}$$

Процедура обратного распространения ошибки VIII

Результирующий градиент:

$$\nabla_w Q(w) = \begin{bmatrix} s^{(1,1)} y^{(1,1)} (1 - y^{(1,1)}) \\ s^{(1,1)} y^{(1,1)} (1 - y^{(1,1)}) x \\ s^{(1,2)} y^{(1,2)} (1 - y^{(1,2)}) \\ s^{(1,2)} y^{(1,2)} (1 - y^{(1,2)}) x \\ s^{(2,1)} y^{(2,1)} (1 - y^{(2,1)}) \\ s^{(2,1)} y^{(2,1)} (1 - y^{(2,1)}) y^{(1,1)} \\ s^{(2,1)} y^{(2,1)} (1 - y^{(2,1)}) y^{(1,2)} \end{bmatrix}$$

1 Многослойный перцептрон (нейронные сети прямого распространения)
2 Универсальные аппроксимационные способности нейронных сетей
3 Архитектура нейронных сетей с радиальными базисными функциями
4 Методика применения нейронных сетей
5 Обучение нейронных сетей как задача оптимизации
6 Процедура обратного распространения ошибки
7 Поиск глобального минимума при обучении
8 Определение оптимальной структуры многослойного перцептрона
9 Выводы
10 Литература

Поиск глобального минимума при обучении

Функционал качества, который обучается,– многоэкстремален.
Решение – добавление к ОРО **стохастичности** (на основе метода Монте-Карло):

- метод **мультистарта**:
  1. из множества X случайно или детерминировано выбирается некоторое подмножество из N точек;
  2. последовательно из каждой точки запускается алгоритм локального спуска, и из полученного множества локальных решений выбирается наилучшее;
- метод **встряски весов** (jog of weights) = метод туннелирования – каждая итерация из двух фаз:
  1. фаза минимизации улучшает значение текущего рекорда;
  2. фаза туннелирования находит точку из допустимого множества, отличную от последней, где найдено значение минимума. Полученная на фазе туннелирования точка рассматривается как стартовая для очередного цикла.

1 Многослойный перцептрон (нейронные сети прямого распространения)
2 Универсальные аппроксимационные способности нейронных сетей
3 Архитектура нейронных сетей с радиальными базисными функциями
4 Методика применения нейронных сетей
5 Обучение нейронных сетей как задача оптимизации
6 Процедура обратного распространения ошибки
7 Поиск глобального минимума при обучении
8 Определение оптимальной структуры многослойного перцептрона
9 Выводы
10 Литература

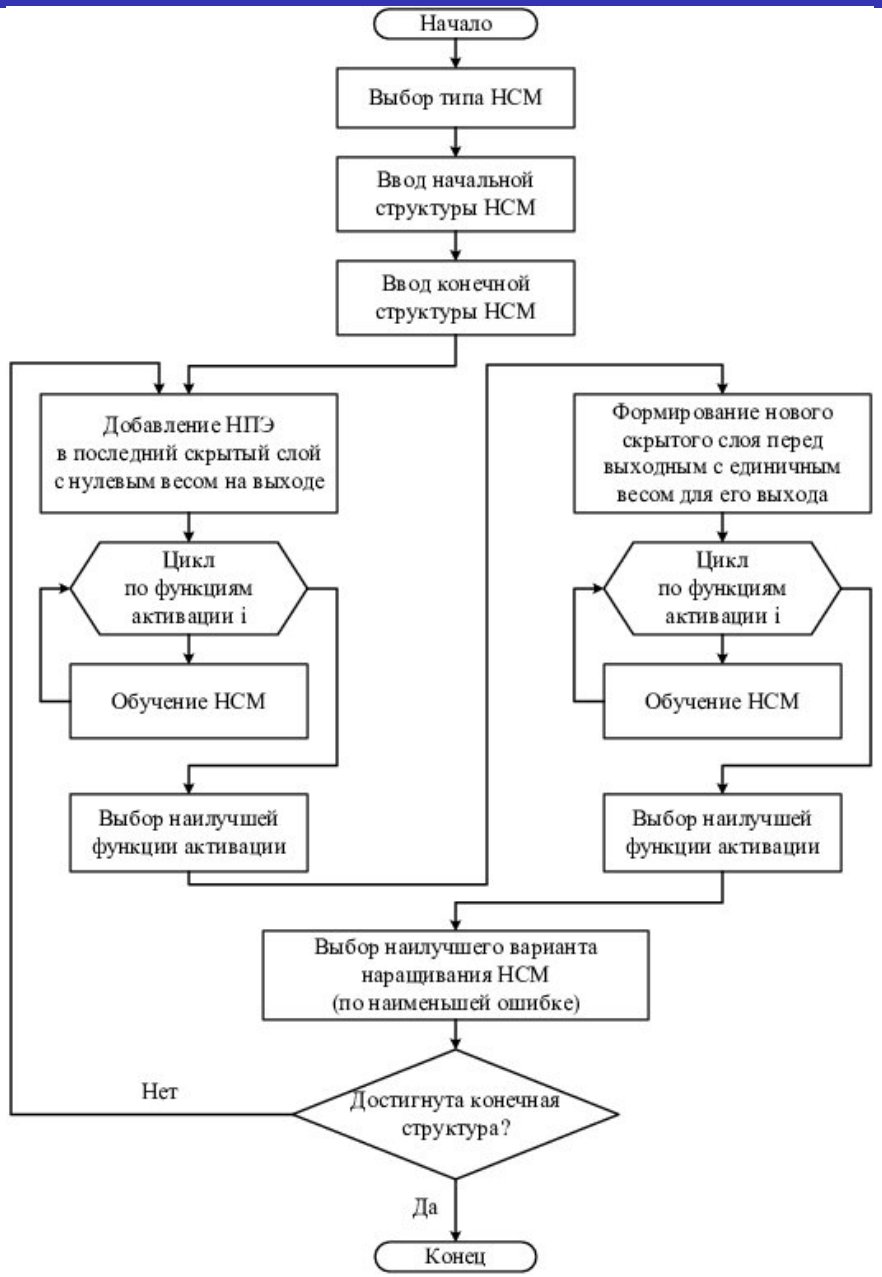
Определение оптимальной структуры I

Стратегии выбора структуры многослойного перцептрона:

- **генерация** набора многослойных перцептронов, не зависимых друг от друга, и дальнейший выбор лучшей из них;
- **контрастирование** многослойного перцептрона (редукция, network pruning), заключающееся в оценке значимости нейронов в текущей НС и удалении наименее значимых весов или нейронов;
- **конструктивный подход** (constructive, network growing), заключающийся в последовательном наращивании количества слоев и/или нейронов в скрытых слоях, начиная с сети минимальной структуры.

Один из подходов к конструированию многослойных перцептронов, обеспечивающих невозрастание ошибки обучения.

Определение оптимальной структуры II



1 Многослойный перцептрон (нейронные сети прямого распространения)
2 Универсальные аппроксимационные способности нейронных сетей
3 Архитектура нейронных сетей с радиальными базисными функциями
4 Методика применения нейронных сетей
5 Обучение нейронных сетей как задача оптимизации
6 Процедура обратного распространения ошибки
7 Поиск глобального минимума при обучении
8 Определение оптимальной структуры многослойного перцептрона
9 Выводы
10 Литература

ВЫВОДЫ (ОСНОВНЫЕ МОМЕНТЫ ЛЕКЦИИ)

1. Многослойные перцептроны – наиболее распространенная архитектура НС.
2. Теоретическая основа применения многослойных перцептронов – их универсальные аппроксимационные способности.
3. Обучение многослойных перцептронов формализуется как задача оптимизации.
4. Эффективный способ вычисления градиента функционала качества по вектору весов – процедура обратного распространения ошибки.
5. Существуют разные подходы для решения задачи определения оптимальной структуры нейронной сети.

1 Многослойный перцептрон (нейронные сети прямого распространения)
2 Универсальные аппроксимационные способности нейронных сетей
3 Архитектура нейронных сетей с радиальными базисными функциями
4 Методика применения нейронных сетей
5 Обучение нейронных сетей как задача оптимизации
6 Процедура обратного распространения ошибки
7 Поиск глобального минимума при обучении
8 Определение оптимальной структуры многослойного перцептрона
9 Выводы
10 Литература

ЛИТЕРАТУРА

1. Кириченко А.А. Основы теории искусственных нейронных сетей: Издательская система Ridero, 2020.– 231 с.
2. J. Kaplan, S. McCandlish, T. Henighan, et al. Scaling Laws for Neural Language Models. CoRR, abs/2001.08361, 2020.– https://arxiv.org/pdf/2001.08361
3. J. Sevilla, L. Heim, A. Ho, et al. Compute Trends Across Three Eras of Machine Learning. CoRR, abs/2202.05924, 2022.– https://arxiv.org/pdf/2202.05924
4. J. Zbontar, L. Jing, I. Misra, et al. Barlow Twins: Self-Supervised Learning via Redundancy Reduction. CoRR, abs/2103.03230, 2021.– https://arxiv.org/pdf/2103.03230

1 Многослойный перцептрон (нейронные сети прямого распространения)
2 Универсальные аппроксимационные способности нейронных сетей
3 Архитектура нейронных сетей с радиальными базисными функциями
4 Методика применения нейронных сетей
5 Обучение нейронных сетей как задача оптимизации
6 Процедура обратного распространения ошибки
7 Поиск глобального минимума при обучении
8 Определение оптимальной структуры многослойного перцептрона
9 Выводы
10 Литература

СПАСИБО ЗА ВНИМАНИЕ!

Построение и применение нейронных сетей и генетических алгоритмов
Лекция 3
Генетические алгоритмы

САРАЕВ ПАВЕЛ ВИКТОРОВИЧ

РТУ МИРЭА
Кафедра прикладной математики

Москва – 2026

ПЛАН ЛЕКЦИИ

1. Обучение как задача глобальной оптимизации
2. Генетические алгоритмы для решения задач глобальной оптимизации
3. Особенности применения генетических алгоритмов
4. Схемы кодирования
5. Применение генетических алгоритмов в обучении нейронных сетей
6. Выводы
7. Литература

1 Обучение как задача глобальной оптимизации

2 Генетические алгоритмы для решения задач глобальной оптимизации

3 Особенности применения генетических алгоритмов

4 Схемы кодирования

5 Применение генетических алгоритмов в обучении нейронных сетей

6 Выводы

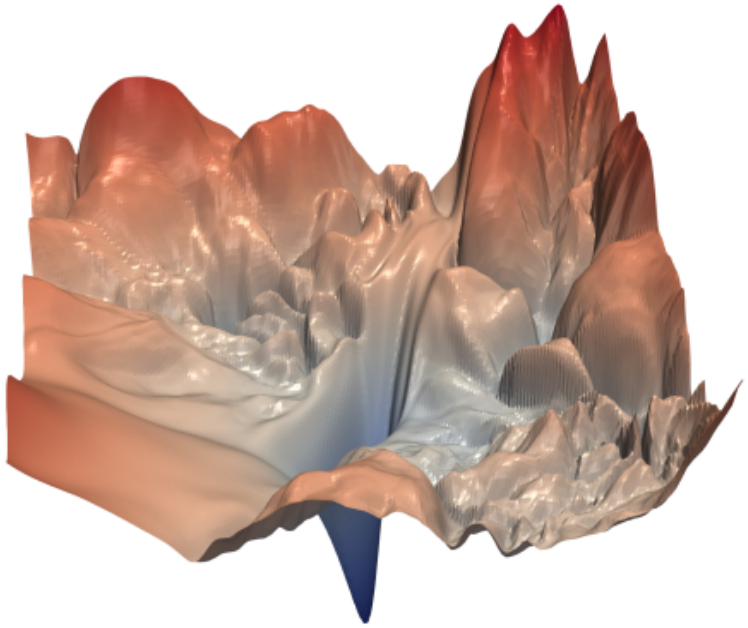
7 Литература

Обучение как задача глобальной оптимизации

Квадратичный функционал качества (ошибка обучения):

$$Q = \sum_{i=1}^{k} \left(W^{(M)} \Phi^{(M-1)} \left(W^{(M-2)} \Phi^{(M-1)} \left(\ldots \Phi^{(1)} \left(W^{(1)} x \right) \ldots \right) \right) - \tilde{y} \right)^2 -$$

многоэкстремальный характер вследствие нейлинейности и многомерности функциональной зависимости, реализуемой НС.



1 Обучение как задача глобальной оптимизации

2 Генетические алгоритмы для решения задач глобальной оптимизации

3 Особенности применения генетических алгоритмов

4 Схемы кодирования

5 Применение генетических алгоритмов в обучении нейронных сетей

6 Выводы

7 Литература

Генетические алгоритмы для решения задач глобальной оптимизации I

Во многих ситуациях непосредственный случайный поиск **малоэффективен**. Значительное улучшение эффективности – внесение элементов детерминированности.

Один из типов детерминировано-стохастических методов – эволюционные вычисления (ЭВ) – общее описание алгоритмов оптимизации, основанных на формализованных принципах естественного эволюционного процесса. Основное преимущество ЭВ – возможность решения **многоэкстремальных задач большой размерности**.

Наиболее распространённая парадигма ЭВ – генетические алгоритмы (ГА).

Парадигма ГА предложена Джоном Холландом в начале 60-х годов. Всеобщее признание теория ГА получила после выхода в 1975г. его работы «Адаптация в естественных и искусственных системах».

Генетические алгоритмы для решения задач глобальной оптимизации II



Рис. 1. Джон Холланд (John Holland), 02.02.1929–09.08.2015

Джон Холланд – американский ученый, профессор психологии, профессор электроинженерии и компьютерных наук. Закончил MIT.

Генетические алгоритмы для решения задач глобальной оптимизации III

Особенности ГА:

1. Представление любой альтернативы решения в виде битовой строки фиксированной длины, манипуляции с которой производятся в отсутствие всякой связи с её смысловой интерпретацией.
2. Для поиска ГА использует несколько точек пространства одновременно, а не переходит от точки к точке, как это делается в традиционных методах оптимизации. Это позволяет снизить вероятность попадания целевой функции в локальный минимум.
3. Для работы ГА требуется только информация о целевой функции и области допустимых значений. Выполнение условий дифференцируемости и даже непрерывности оптимизируемой функции не требуется. Т.о., ГА могут применяться как для оптимизации непрерывных, так и для оптимизации дискретных, а также дискретно-непрерывных функций.
4. Для перехода от одних точек к другим используются как вероятностные, так и детерминированные правила перехода.

Генетические алгоритмы для решения задач глобальной оптимизации IV

Основные понятия:

Оптимизируемая функция $f(x)$ наз. фитнесс-функцией (функцией приспособляемости).

Точка пространства, описываемая с помощью битовой строки,

$$x = (x_1 x_2 x_3 \dots x_n),$$

где x_i – бит – хромосома (особь, индивид).

Бит строки x_i – ген. (В некоторых алгоритмах 1 ген представляет собой некоторую десятичную цифру) В дальнейшем будет рассматриваться только бинарный способ кодирования.

Совокупность особей на каждом шаге работы алгоритма, образующих множество точек решений,

$$X = \{x_i, i = 1, \dots, k\} -$$

популяция.

Количество особей в популяции k – размер популяции.

Генетические алгоритмы для решения задач глобальной оптимизации V

Основной параметр работы – качество особи, которое определяется значением фитнесс-функции в точке пространства поиска $f(x)$, описываемой этой особью.

На каждом шаге работы ГА обновляет популяцию путём создания новых особей и уничтожения старых особей, являющихся «бесперспективными» («**естественный отбор**»).

Каждое обновление интерпретируется как смена поколений. Поколения идентифицируются по номеру. Популяция, полученная после первого шага работы алгоритма – первое поколение X_1, после следующего – второе X_2 и т.д.

Генерация новых поколений происходит на основе моделирования размножения. Порождающие особи – родители, а порождённые – потомки. Обычно родительская пара порождает пару потомков.
Непосредственная генерация новых особей происходит за счёт работы оператора скрещивания (вероятностно-детерминированного обмена) – кроссинговера. Часть пар может переходить в популяцию следующего поколения без изменений. Выбор

пар родителей для порождения потомков производит оператор отбора, а выбор особей для уничтожения – оператор редукции.

Также генерация новых точек моделируется оператором мутации, который приводит к изменению некоторых генов особей.

Т.о., **генетические операторы**:

- оператор отбора;
- оператор скрещивания (кроссинговера);
- оператор мутации;
- оператор редукции.

Критерии останова работы ГА:

- Сформировано заданное число поколений.
- Достигнута популяция заданного предела качества (например, при превышении качества каждой особи некоторого порога).
- Достигнут некоторый уровень сходимости, т.е. наблюдается слабое улучшение качества особей.

Генетические алгоритмы для решения задач глобальной оптимизации VII

Алгоритм (схема ГА)

1. Создание исходной популяции.
2. Выбор родителей для процесса размножения (оператор отбора).
3. Создание потомков выбранных пар родителей (оператор скрещивания).
4. Мутация новых особей (оператор мутации).
5. Сокращение расширенной популяции до исходного размера (оператор редукции).
6. Проверка выполнения критерия останова. Если не выполнен, то переход на шаг 2.
7. Выбор лучшей достигнутой особи в конечной популяции в качестве решения.

Генетические алгоритмы для решения задач глобальной оптимизации VIII

1. Создание исходной популяции.

Задается размер исходной популяции – целое число k (20-100).

Выбор исходной популяции заданного размера происходит с использованием случайного закона распределения, как правило, равномерного.

Исходная популяция может быть результатом работы какого-либо другого алгоритма оптимизации.

2. Выбор родителей для процесса размножения (оператор отбора).

В основе оператора отбора лежит принцип «**выживает сильнейший**». Вероятность участия i-й особи, $i = 1, \ldots, k$, в процессе размножения при *максимизации* при $f(x_i) > 0$:

$$p_i = \frac{f(x_i)}{\sum_{i=1}^{k} f(x_i)}$$

Генетические алгоритмы для решения задач глобальной оптимизации IX

Вероятность участия i-й особи, $i = 1, \ldots, k$, в процессе размножения при минимизации в общем случае *может* быть такой:

$$p_i = \frac{f_{max} - f(x_i) + 1}{\sum_{i=1}^{k}(f_{max} - f(x_i) + 1)} = \frac{f_{max} - f(x_i) + 1}{k(f_{max} + 1) - \sum_{i=1}^{k} f(x_i)}$$

Здесь $f_{max} = \max_{i=1}^{k} f(x_i)$

- Может участвовать любая особь.
- Одна особь может быть задействована в нескольких родительских парах.

Пример.

$f(x) = x^2$

$x_1 = 0,5; x_2 = 1; x_3 = -1$
$k = 3; f_{max} = 1$

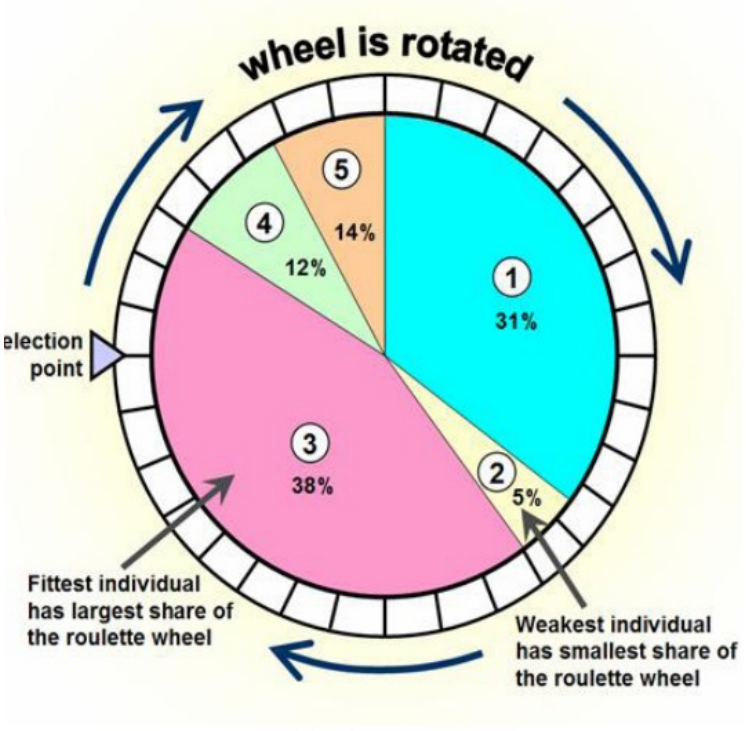
$$p_1 = \frac{1 - 0,25 + 1}{3(1+1) - \sum_{i=1}^{k} f(x_i)} = \frac{1,75}{6 - 2,25} = \frac{1,75}{3,25} \approx 0,46$$

$$p_2 = \frac{1 - 1 + 1}{3(1+1) - \sum_{i=1}^{k} f(x_i)} = \frac{1}{6 - 2,25} = \frac{1}{3,25} \approx 0,27$$

$$p_3 = \frac{1 - 1 + 1}{3(1+1) - \sum_{i=1}^{k} f(x_i)} = \frac{1}{6 - 2,25} = \frac{1}{3,25} \approx 0,27$$

Генетические алгоритмы для решения задач глобальной оптимизации XI

Pratibha Bajpai et al. / Indian Journal of Computer Science and Engineering
Vol 1 No 3 199-206



No.	Chromosome	$Value_{10}$	X	Fitness f(x)	% of Total
1	0001101011	107	1.05	6.82	31
2	1111011000	984	9.62	1.11	5
3	0100000101	261	2.55	8.48	38
4	1110100000	928	9.07	2.57	12
5	1110001011	907	8.87	3.08	14
Totals				22.05	100

Example population of 5 for: $f(x) = -\frac{1}{4}x^2 + 2x + 5$

Value10: Value of chromosome to the base 10

X = Value10normalized between 0 to 10 range

Рис. 2. «Рулетка» для выбора хромосом

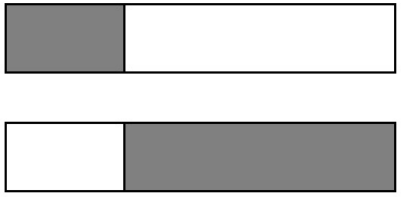
3. Создание потомков выбранных пар родителей (оператор скрещивания).

- *Одноточечный*

Для двух родителей x и y на основе равномерного закона выбирается случайное натуральное число l от 1 до $n-1$ – точка разбиения (разрыва). Векторы x, y разделяются на 2 сегмента (подстроки) по этой точке. Далее x и y обмениваются своими сегментами:

$$x_1 x_2 \ldots x_l y_{l+1} \ldots y_n;$$

$$y_1 y_2 \ldots y_l x_{l+1} \ldots x_n.$$



Пример.

Родители:	Потомки:
01110010	11010010
↕	
110**10100**	**01110100**

Точка разбиения: $l = 3$

- *Двухточечный*

Выбираются 2 точки разрыва, и родительские особи обмениваются сегментом, находящимся между этими двумя точками.



- *Равномерный*

Каждый бит первого родителя наследуется первым потомком с заданной вероятностью; в противном случае этот бит передаётся второму потомку.

Вероятность применения оператора скрещивания выбирается достаточно большой, от 0,9 до 1, чтобы обеспечить постоянное появление новых особей, расширяющих пространство поиска.

4. Мутация новых особей (оператор мутации).

Оператор скрещивания расширяет область пространства поиска, но использует ограниченный набор значений. Оператор мутации (случайных изменений) позволяет значительно расширить область поиска, что приводит к сокращению времени работы алгоритма и улучшению качества работы.

Мутация – случайное изменение одного гена текущей особи, т.е. инверсия битов. Вероятность мутации является достаточно малой. Обычно она не превышает 0,1. Векторы, полученные в ходе мутации, могут значительно отличаться друг от друга.

Пример.

$$10110 \longrightarrow 11110.$$

5. Сокращение расширенной популяции до исходного размера (оператор редукции).

Оператор редукции уничтожает особи с наихудшим уровнем приспособляемости, оставляя k лучших особей.

1 Обучение как задача глобальной оптимизации

2 Генетические алгоритмы для решения задач глобальной оптимизации

3 Особенности применения генетических алгоритмов

4 Схемы кодирования

5 Применение генетических алгоритмов в обучении нейронных сетей

6 Выводы

7 Литература

Особенности применения генетических алгоритмов I

Особенности работы ГА:

1. Интенсивная мутация снижает скорость алгоритма и может серьезно повлиять на сходимость.
2. Слишком малая вероятность мутации затрудняет выход алгоритма из локальных максимумов.
3. Большие популяции сходятся дольше.
4. При малом размере популяции возрастает вероятность остановки в локальном максимуме.
5. Битовое кодирование имеет недостатки.

Особенности применения генетических алгоритмов II

Основной недостаток - соседние числа отличаются в значениях нескольких битов, так например числа 7 и 8 в битовом представлении

$$7 = 0111$$

$$8 = 1000$$

различаются в 4-х(!) позициях.

Это затрудняет функционирование генетического алгоритма и увеличивает время, необходимое для его сходимости. Чтобы избежать этого, используется кодирование, при котором соседние числа отличаются меньшим количеством позиций, в идеале - значением одного бита.

Таким кодом является код Грея, который целесообразно использовать в реализации генетического алгоритма.

Особенности применения генетических алгоритмов III

Таблица кодировки Грея:

Десятичный код	Битовая кодировка	Код Грея	Десятичный код	Битовая кодировка	Код Грея
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

Особенности применения генетических алгоритмов IV

Алгоритм (преобразование битовой кодировки в кодировку Грея-1)

1. Старший единичный бит оставляется.
2. Младшие биты меняются на противоположные, если слева находится единица в исходном представлении. В противном случае биты не меняются.

Алгоритм (преобразование битовой кодировки в кодировку Грея-2)

i-й бит кода Грея G_i выражается через биты двоичного кода B_i следующим образом:

$$G_i = B_i \oplus B_{i+1},$$

биты нумеруются справа налево, начиная с младшего.
Удобная форма для компьютерной реализации:

$$G = BXOR(B >> 1),$$

где $>>$ – операция нециклического сдвига вправо.

Особенности применения генетических алгоритмов V

Пример.

$$0101 = 011\,1\ \mathrm{G}$$
$$1010 = 111\,1\ \mathrm{G}$$

Особенности применения генетических алгоритмов VI

Алгоритм (преобразование кодировки Грея в битовую-1)

1. Старший единичный бит оставляется.
2. Младшие биты меняются на противоположные, если слева находится нечетное количество единиц. В противном случае биты не меняются.

Алгоритм (преобразование кодировки Грея в битовую-2)

Рекуррентная процедура:

$$B_i = G_i \oplus B_{i+1},$$

старший бит сохраняется.
Удобная форма для компьютерной реализации:

$$B = \oplus_{i=0}^{n-1}(G >> i),$$

где n – количество бит.

Особенности применения генетических алгоритмов VII

Пример.

$$0111\ \mathrm{G} = 0101$$
$$1111\ \mathrm{G} = 1010$$

6. Кодирование признаков, которым соответствуют числа с плавающей точкой.

Интервал допустимых значений признака разбивается на участки с требуемой точностью. Значение гена – неотрицательное целое число, определяющее номер интервала (используется код Грея). Как правило, количество интервалов принимается равным степени 2.

Значение параметра - середина интервала.

Пример.
Пусть значения признака лежат в [0;1]. Разобьем отрезок на $256 = 2^8$ интервалов. Для кодирования номера нужно 8 бит. Пусть значение гена:

$$\mathbf{00100101}\ \mathrm{G}.$$

Особенности применения генетических алгоритмов VIII

Найдем соответствующий номер интервала:

$$0010\ 0101\ \mathrm{G} = 0011\ 0110 = \mathbf{54}$$

Ему соответствует интервал:

$$[54/256;\ (54+1)/256] = [0{,}210937500;\ 0{,}214843750]$$

Значение параметра:

$$(0{,}210937500 + 0{,}214843750)\ /\ 2 = 0{,}212890625$$

При увеличении числа переменных пространство поиска становится огромным.

Пусть для 100 переменных, изменяющихся в интервале [-500; 500], требуется найти экстремум с точностью до шестого знака после запятой. При использовании ГА с двоичным кодированием длина строки составит 3000 элементов, а пространство поиска – около 10^{1000} хромосом.

Особенности применения генетических алгоритмов IX

7. Часто используется принцип элитизма - переход в следующую популяцию элиты, т.е. лучших особей текущей популяции, без всяких изменений.

8. Иногда используется оператор инверсии: особь делится на 2 части, которые меняются друг с другом местами.

Пример.

$$111\ \mathbf{00000} \longrightarrow \mathbf{00000}\ 111.$$

9. Исследования показывают следующие результаты:

Особенности применения генетических алгоритмов X



Рис. 3.)))

1 Обучение как задача глобальной оптимизации

2 Генетические алгоритмы для решения задач глобальной оптимизации

3 Особенности применения генетических алгоритмов

4 Схемы кодирования

5 Применение генетических алгоритмов в обучении нейронных сетей

6 Выводы

7 Литература

Схемы кодирования I

Возможные варианты кодирования (иногда относят к одному из видов генетических операторов):

- двоичное;
- восьмеричное;
- шестнадцатеричное;
- перестановочное;
- основанное на значениях;
- древовидное.

Двоичное кодирование:

- самый распространённый;
- быстрые мутация и скрещивание;
- дополнительные затраты на преобразование данных в/из двоичную.

Схемы кодирования II

Восьмеричное и шестнадцатеричное кодирование:

представление в виде 8- или 16-ричной системы.

Перестановочное кодирование:

обычно используется в задачах упорядочивания: ген/хромосома представлены строкой чисел, которая представляет позицию в последовательности.

Основанное на значениях (value-based):

- ген/хромосома представлены строкой вещественных, целых или символьных значений;
- полезна при решении задач, в которых используются более сложные комбинации значений;
- в основном используется в **нейронных сетях** для обучения.

Схемы кодирования III

Древовидное кодирование:

ген/хромосома представлены деревом функций или команд (могут быть связаны с любым языком программирования);

1 Обучение как задача глобальной оптимизации

2 Генетические алгоритмы для решения задач глобальной оптимизации

3 Особенности применения генетических алгоритмов

4 Схемы кодирования

5 Применение генетических алгоритмов в обучении нейронных сетей

6 Выводы

7 Литература

Применение генетических алгоритмов в обучении нейронных сетей I

Представление хромосомы в виде последовательности **вещественных** чисел – весов НС

Montana, David J. and Lawrence Davis. "Training Feedforward Neural Networks Using Genetic Algorithms." International Joint Conference on Artificial Intelligence (1989)

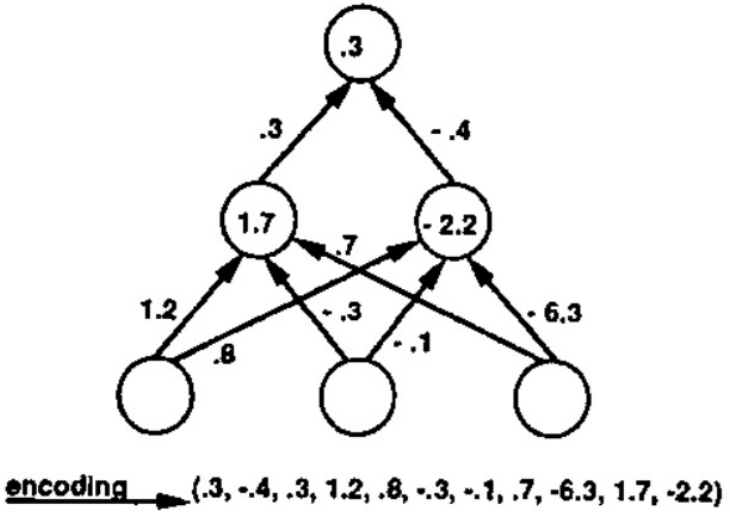


Figure 1: Encoding a Network on a Chromosome

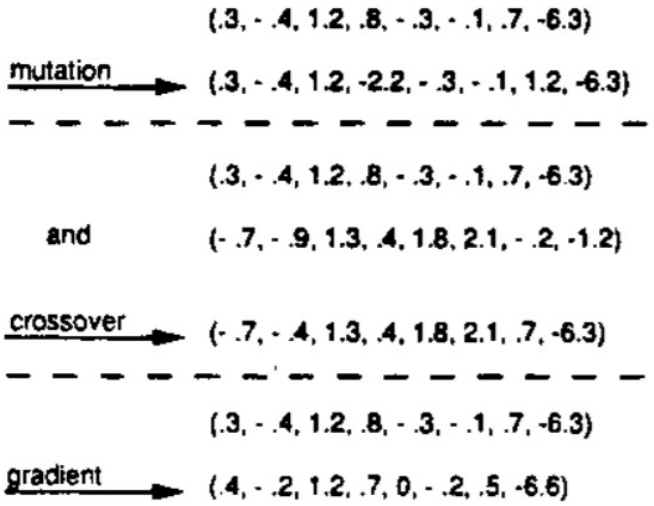


Figure 2: Operation of the Operators

Применение генетических алгоритмов в обучении нейронных сетей II

Выбор оптимальной структуры – представление хромосомы на основе структуры НС

Delima A.J., Arroyo J. C. (2022). An Optimized Neural Network Using Genetic Algorithm for Cardiovascular Disease Prediction. Journal of Advances in Information Technology. 13. 95-99. 10.12720/jait.13.1.95-99.

Parameters	Neurons	Layers	Activation	Optimizer
Chromosome X	32	3	sigmoid	sgd

Parameters	Neurons	Layers	Activation	Optimizer
Chromosome Y	64	2	softmax	adam

Figure 2. Crossover mating scheme.

1 Обучение как задача глобальной оптимизации

2 Генетические алгоритмы для решения задач глобальной оптимизации

3 Особенности применения генетических алгоритмов

4 Схемы кодирования

5 Применение генетических алгоритмов в обучении нейронных сетей

6 Выводы

7 Литература

ВЫВОДЫ (ОСНОВНЫЕ МОМЕНТЫ ЛЕКЦИИ)

1. Генетические алгоритмы – комбинация детерминированного и стохастического подхода.
2. Применение генетических алгоритмов не зависит от решаемой задачи. Основная проблема – кодировка оптимизируемых переменных битовыми строками.
3. Генетические алгоритмы – смена поколений на основе операторов отбора, скрещивания, мутации, редукции.
4. Для кодирования переменных вместо традиционной кодировки необходимо использовать кодировку Грея.
5. Для представления вещественных чисел используется разбиение исходной области на интервалы и кодирование их целым числом – порядковым номером интервала.

1 Обучение как задача глобальной оптимизации

2 Генетические алгоритмы для решения задач глобальной оптимизации

3 Особенности применения генетических алгоритмов

4 Схемы кодирования

5 Применение генетических алгоритмов в обучении нейронных сетей

6 Выводы

7 Литература

ЛИТЕРАТУРА

1. Вирсански Э. Генетические алгоритмы на Python / пер. с англ. А. А. Слинкина.– М.: ДМК Пресс, 2020.– 286 с.
2. Ghosh, S., Chatterjee, A. & Fiondella, L. An active learning framework for adversarial training of deep neural networks. Neural Comput & Applic 37, 6849–6876 (2025).– https://doi.org/10.1007/s00521-024-10851-6
3. Charilogis, V.; Tsoulos, I.G. Introducing a Parallel Genetic Algorithm for Global Optimization Problems. AppliedMath 2024, 4, 709-730.– https://doi.org/10.3390/appliedmath4020038

1 Обучение как задача глобальной оптимизации

2 Генетические алгоритмы для решения задач глобальной оптимизации

3 Особенности применения генетических алгоритмов

4 Схемы кодирования

5 Применение генетических алгоритмов в обучении нейронных сетей

6 Выводы

7 Литература

СПАСИБО ЗА ВНИМАНИЕ!

Построение и применение нейронных сетей и генетических алгоритмов
Лекция 4
Интервальные алгоритмы для решения задач глобальной оптимизации и обучения нейронных сетей

САРАЕВ ПАВЕЛ ВИКТОРОВИЧ

РТУ МИРЭА
Кафедра прикладной математики

Москва – 2026

ПЛАН ЛЕКЦИИ

1. Функционалы качества обучения, отличные от квадратичного
2. Основы интервального анализа
3. Интервальные алгоритмы в обучении нейронных сетей
4. Интервальные генетические алгоритмы
5. Заключение
6. Литература

1 Функционалы качества обучения, отличные от квадратичного

2 Основы интервального анализа

3 Интервальные алгоритмы в обучении нейронных сетей

4 Интервальные генетические алгоритмы

5 Заключение

6 Литература

Функционалы качества обучения, отличные от квадратичного

Вектор весов:

$$w.$$

Обучающее множество:

$$\langle \tilde{X}, \tilde{Y} \rangle = \{\tilde{x}_i, \tilde{y}_i\}, i = 1, \ldots, k.$$

Вектор невязок:

$$\varepsilon(w) = y(w) - \tilde{y}.$$

Q – норма вектора невязок ε:

$$Q(w) = \|\varepsilon(w)\|.$$

l_∞-норма (равномерная):

$$Q(w) = \|\varepsilon(w)\|_\infty = \max_{i=1}^{k} |\varepsilon_i|.$$

1 Функционалы качества обучения, отличные от квадратичного

2 Основы интервального анализа

3 Интервальные алгоритмы в обучении нейронных сетей

4 Интервальные генетические алгоритмы

5 Заключение

6 Литература

Основы интервального анализа I

Вычислительные парадоксы (КАТАСТРОФА!)

Пример 1. Perl

http://xpoint.ru/know-how/Articles/FloatingPointNumbers?comments)

```
$a = 1.1 - 1;
$b = 0.1;
if ($a == $b)
{
  print "$a равно $b";
}
else
{
  print "$a не равно $b";
}
```

Результат: «0,1 не равно 0,1».

Основы интервального анализа II

Объяснение:

$0,1$ представляется периодическим двоичным числом:

$$1.10011001100110011001100110011001100110011001100110011010b \times 2^{-4}.$$

Обратный перевод:

$$0,10000000000000000555111512.$$

С другой стороны,

$$1,1 - 1 = 0,10000000000000008881784197.$$

Если в вычислении участвует **хотя бы одна** переменная или константа с плавающей запятой, все другие числа тоже преобразуются к этому типу.
Проблемы при сравнении чисел с плавающей запятой. Варианты решения:

1. использовать сравнение модуля разности с малым числом;
2. использовать числа с фиксированной запятой (см. программы для бухгалтерии).

Основы интервального анализа III

Пример 2 (Rump)

Функция:

$$f(x,y) = (333,75 - x^2)y^6 + x^2(11x^2y^2 - 121y^4 - 2) +$$
$$+5,5y^8 + \frac{x}{2y}$$

Значения точек: $x_0 = 77617$, $y_0 = 33096$.
Точность 32-бит: $f(x_0, y_0) = 1,172604$
Точность 64-бит: $f(x_0, y_0) = 1,1726039400531786$
Точность 128-бит: $f(x_0, y_0) = 1,172603940053178631858834904904520 1838$
На самом деле (при этих значениях $x^2 = 5,5y^2 + 1$):

$$f(x_0, y_0) = \frac{x_0}{2y_0} - 2 = -0,82739660...$$

Основы интервального анализа IV

Пример 3.

Детерминант – критерий невырожденности матрицы!? Матрица 30×30:

$$A = \begin{bmatrix} 1 & 2 & 0 & & 0 \\ 0 & 1 & 2 & & \\ & \ddots & \ddots & \ddots & \\ & & & 1 & 2 \\ 0 & & & 0 & 1 \end{bmatrix} \qquad det(A) = 1$$

Малое возмущение:

$$B = \begin{bmatrix} 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & \dots & \vdots & \vdots \\ 0 & 0 & \dots & 0 & 0 \\ \varepsilon & 0 & \dots & 0 & 0 \end{bmatrix} \qquad \varepsilon = 2^{-29} \approx 1,8 \cdot 10^{-9}$$

$$det(A + B) = 1 \cdot 1 \cdot \ldots \cdot 1 - 2 \cdot 2 \cdot \ldots \cdot 2 \cdot 2^{-29} = 1 - 1 = 0!!!$$

Основы интервального анализа V

Пример 4. Собственные значения матрицы C 7×7

$$C = \begin{bmatrix} 289 & 2064 & 336 & 128 & 80 & 32 & 16 \\ 1152 & 30 & 1312 & 512 & 288 & 128 & 32 \\ -29 & -2000 & 756 & 384 & 1008 & 224 & 48 \\ 512 & 128 & 640 & 0 & 640 & 512 & 128 \\ 1053 & 2256 & -504 & -284 & -756 & 800 & 208 \\ -287 & -16 & 1712 & -128 & 1968 & -30 & 2032 \\ -2176 & -287 & -1565 & -512 & -541 & -1152 & -289 \end{bmatrix}$$

MATLAB (точность 10^{-16}):

$$\lambda_1(C) = 6,624$$

$$\lambda_{2,3}(C) = 4,065 \pm 4,381i$$

$$\lambda_{4,5}(C) = -1,472 \pm 5,449i$$

$$\lambda_{6,7}(C) = -5,906 \pm 2,371i$$

Основы интервального анализа VI

Ортогональная матрица P ($P^T = P^{-1}$):

$$P = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Вычислим матрицы (перестановка строк и столбцов):

$$D = P^{-1}CP \qquad F = P^{-1}DP = P^{-1}(P^{-1}CP)P$$

Спектры C, D, F должны совпадать:

Основы интервального анализа VII

MATLAB (точность 10^{-16}):

$$\lambda_1(D) = 6,005$$

$$\lambda_{2,3}(D) = 3,651 \pm 3,784i$$

$$\lambda_{4,5}(D) = -1,331 \pm 4,699i$$

$$\lambda_{6,7}(D) = -5,332 \pm 2,014i$$

MATLAB (точность 10^{-16}):

$$\lambda_1(F) = 8,095$$

$$\lambda_{2,3}(F) = 4,984 \pm 5,709i$$

$$\lambda_{4,5}(F) = -1,825 \pm 7,052i$$

$$\lambda_{6,7}(F) = -7,207 \pm 3,090i$$

Основы интервального анализа VIII

Точный спектр матриц:

$$\lambda_1 = 0$$

$$\lambda_{2,3} = \pm 1$$

$$\lambda_{4,5} = \pm 2$$

$$\lambda_{6,7} = \pm 4$$

Основы интервального анализа IX

Вещественное интервальное число (**интервал**) –

односвязное подмножество R:

$$[x] = [\underline{x}, \overline{x}] = [x \in R | \underline{x} \leqslant x \leqslant \overline{x}]$$

$\underline{x} = lb([x])$ - **нижняя граница** интервала
$\overline{x} = ub([x])$ - **верхняя граница** интервала

Интервал $[x]$ – **точечный (вырожденный)**,

если $\underline{x} = \overline{x}$

Множество (замкнутых) интервалов: IR.

Содержит элементы: $\emptyset$ (пустое множество); $[-\infty, a]$, $[a, +\infty]$, $[-\infty, +\infty]$.

Замечание. $[-\infty, -\infty]$, $[+\infty, +\infty]$, $[a, b]$, $a > b$ – не интервалы

Основы интервального анализа X

Ширина интервала $[x]$:

$$w([x]) = wid([x]) = \overline{x} - \underline{x}$$

Середина (средняя точка, центр) интервала $[x]$:

$$m([x]) = mid([x]) = \frac{\underline{x} + \overline{x}}{2}$$

Пример

$$[x] = [1; 1,5]$$

$$w([x]) = 1,5 - 1 = 0,5$$

$$m([x]) = (1 + 1,5)/2 = 1,25$$

Основы интервального анализа XI

Теоретико-множественные операции:

- пересечение – всегда интервал

$$[x] \cap [y] = \{z \in R | z \in [x] \text{ и } z \in [y]\} =$$

$$= \begin{cases} [\max\{\underline{x}, \underline{y}\}, \min\{\overline{x}, \overline{y}\}], & \text{если } \max\{\underline{x}, \underline{y}\} \leqslant \min\{\overline{x}, \overline{y}\} \\ 0, & \text{иначе} \end{cases}$$

- объединение – *интервальная оболочка*

$$[x] \coprod [y] = [[x] \cup [y]] = \{z \in R | z \in [x] \text{ и } z \in [y]\} = [\min\{\underline{x}, \underline{y}\} \leqslant \min\{\overline{x}, \overline{y}\}]$$

- разность – *интервальная оболочка*

$$[x][\backslash][y] = [[x] \setminus [y]]$$

Основы интервального анализа XII

Пример.

$$[1;4] \cap [2;7] = [2;4]$$

$$[1;4] \coprod [7;9] = [[1;4] \cup [7;9]] = [1;9]$$

$$[1;9][\backslash][2;6] = [[1;2) \cup (6;9]] = [1;9]$$

Арифметические операции.
Для любого бинарного оператора $*$:

$$[x][*][y] = [\min\{x * y | x \in [x], y \in [y]\}, \max\{x * y | x \in [x], y \in [y]\}]$$

1. Умножение на скаляр:
$$a[x] = \begin{cases} [a\underline{x}, a\overline{x}], a \geqslant 0; \\ [a\overline{x}, a\underline{x}], a < 0 \end{cases}$$
2. Сумма:
$$[x] + [y] = [\underline{x} + \underline{y}, \overline{x} + \overline{y}]$$

Основы интервального анализа XIII

3. Разность:

$$[x] - [y] = [\underline{x} - \overline{y}, \overline{x} - \underline{y}]$$

4. Умножение:

$$[x][y] = [\min\{\underline{x}\underline{y}, \underline{x}\overline{y}, \overline{x}\underline{y}, \overline{x}\overline{y}, \}, \max\{\underline{x}\underline{y}, \underline{x}\overline{y}, \overline{x}\underline{y}, \overline{x}\overline{y}\}]$$

5. Деление:

$$\frac{1}{[y]} = \begin{cases} 0, [y] = [0, 0] \\ [\frac{1}{\overline{y}}, \frac{1}{\underline{y}}], 0 \notin [y] \\ [\frac{1}{\overline{y}}, +\infty], \underline{y} = 0, \overline{y} > 0 \\ [-\infty, \frac{1}{\underline{y}}], \underline{y} < 0, \overline{y} = 0 \\ [-\infty, +\infty], \underline{y} < 0, \overline{y} > 0 \end{cases}$$

$$\frac{[x]}{[y]} = [x][\frac{1}{\overline{y}}, \frac{1}{\underline{y}}]$$

Основы интервального анализа XIV

Пример.

$$3 \cdot [1;4] = [3;12]$$

$$[1;4] + [-2;4] = [-1;8]$$

$$[1;4] - [-2;4] = [-3;6]$$

$$[1;4] \cdot [-2;4] = [-8;16]$$

$$\frac{[1,4]}{[1,2]} = [1/2;4]$$

Основы интервального анализа XV

Монотонность арифметических операций по включению:

$$([x] \subset [x\prime], [y] \subset [y\prime]) \Rightarrow ([x] * [y] \subset [x\prime] * [y\prime]) \qquad * \in \{+, -, \cdot, /\}$$

Особенности алгебраической структуры:

- $$([a] + [b]) - [b] \neq [a]$$
- $$([a] \cdot [b])/[b] \neq [a]$$

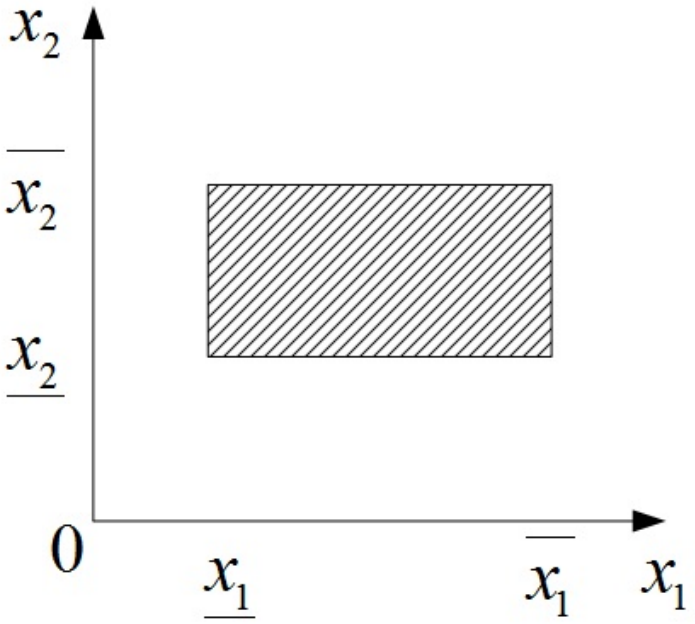
Интервальный вектор (брус, параллелотоп) $[x]$ –

подмножество R^n, – прямое произведение n интервалов:

$$[x] = [x_1] \times [x_2] \times \ldots \times [x_n]$$

Основы интервального анализа XVI

Множество всех n-мерных брусов - IR^n – n-мерные параллелепипеды, ориентированные по соответствующим осям декартовой системы координат в n-мерном пространстве.



Основы интервального анализа XVII

Ширина $w([x])$ бруса $[x]$:

$$w([x]) = \max_{1 \leqslant i \leqslant n} w([x_i])$$

Интервальная матрица размера $m \times n$ –

прямое произведение mn интервалов:

$$[A] = \begin{bmatrix} [a_{11}] & & \\ & \ddots & \\ & & [a_{ij}] \end{bmatrix} = [a_{11}] \times a_{12} \times \ldots \times [a_{mn}]$$

Множество всех $m \times n$-мерных интервальных матриц – $IR^{m \times n}$.

Основы интервального анализа XVIII

Особенности произведения матриц:

- не ассоциативно:

$$([A][B])[C] <> [A]([B][C])$$

- не коммутативно по отношению к скалярам:

$$[\alpha]([A][x]) <> [A]([\alpha][x]).$$

Основы интервального анализа XIX

Интервальные функции.

Интервальное расширение функций:

$$f([x]) = \{f(x)|x \in [x]\}$$

Интервальное расширение многих элементарных функций можно выразить в терминах границ.
Для монотонно возрастающих функций:

$$f([x]) = [f(\underline{x}), f(\overline{x})]$$

Для монотонно убывающих функций аналогично.

Пример.

$$e^{[x]} = [e^{\underline{x}}, e^{\overline{x}}]$$

$$e^{[0;1]} = [e^0, e^1] = [1, e]$$

Для ряда немонотонных функций образ можно выразить через границы:

$$f(x) = x^2$$

$$([x])^2 = \begin{cases} [0, \max\{\underline{x^2}, \overline{x^2}\}], 0 \in [\underline{x}, \overline{x}], \\ [\underline{x^2}, \overline{x^2}], x > 0, \\ [\overline{x^2}, \underline{x^2}], x < 0, \end{cases}$$

Пример.

$$[-1; 2]^2 = [0; 4] \neq [(-1)^2; 2^2] = [1; 4]$$

Основы интервального анализа XXI

$f(x) = \sin x$

Алгоритм интервальных вычислений синуса:
Вход: $[x]$, выход: $[r]$.

$$\underline{r} = \begin{cases} -1, \text{ если } \exists k \in Z : (2k\pi - \frac{\pi}{2}) \in [x] \\ \min\{\sin(\underline{x}), \sin(\overline{x})\}, \text{ иначе} \end{cases}$$

$$\overline{r} = \begin{cases} 1, \text{ если } \exists k \in Z : (2k\pi + \frac{\pi}{2}) \in [x] \\ \max\{\sin(\underline{x}), \sin(\overline{x})\}, \text{ иначе} \end{cases}$$

Результат вычисления интервальных функций зависит от формального выражения, т. е. от способа представления выражения. Равенство достигается в редких случаях.
В общем случае точно оценить образ функции **невозможно**!

Основы интервального анализа XXII

Пример.

$$f(x) = (f^{(1)}(x) = \frac{1}{2-x} + \frac{1}{2+x}) = (f^{(2)}(x) = \frac{4}{4 - x \cdot x}) = (f^{(3)}(x) = \frac{4}{4 - x^2})$$

$$[x] = [-\frac{1}{2}, \frac{3}{2}]$$

$$[f^{(1)}([x])] = [-\frac{24}{35}, \frac{8}{3}] \qquad [f^{(2)}([x])] = [-\frac{16}{19}, \frac{16}{7}]$$

$$[f^{(3)}([x])] = [1, \frac{16}{7}] = f([x])$$

Для получения более точного результата необходимо максимально уменьшить число появлений переменных в формальном выражении.

1 Функционалы качества обучения, отличные от квадратичного

2 Основы интервального анализа

3 Интервальные алгоритмы в обучении нейронных сетей

4 Интервальные генетические алгоритмы

5 Заключение

6 Литература

Интервальные алгоритмы в обучении нейронных сетей I

Функции включения

Важная задача интервального анализа – получение минимального включающего интервала для функции f, т.е. в заключении множества значений данной вещественной функции в максимально узкий интервал.

Функция включения

Пусть

$$f : R^n \to R^m.$$

Интервальная функция

$$[f] : IR^n \to IR^m -$$

функция включения для f, если

$$f([x]) \subset [f]([x]) \qquad \forall [x] \in IR^n$$

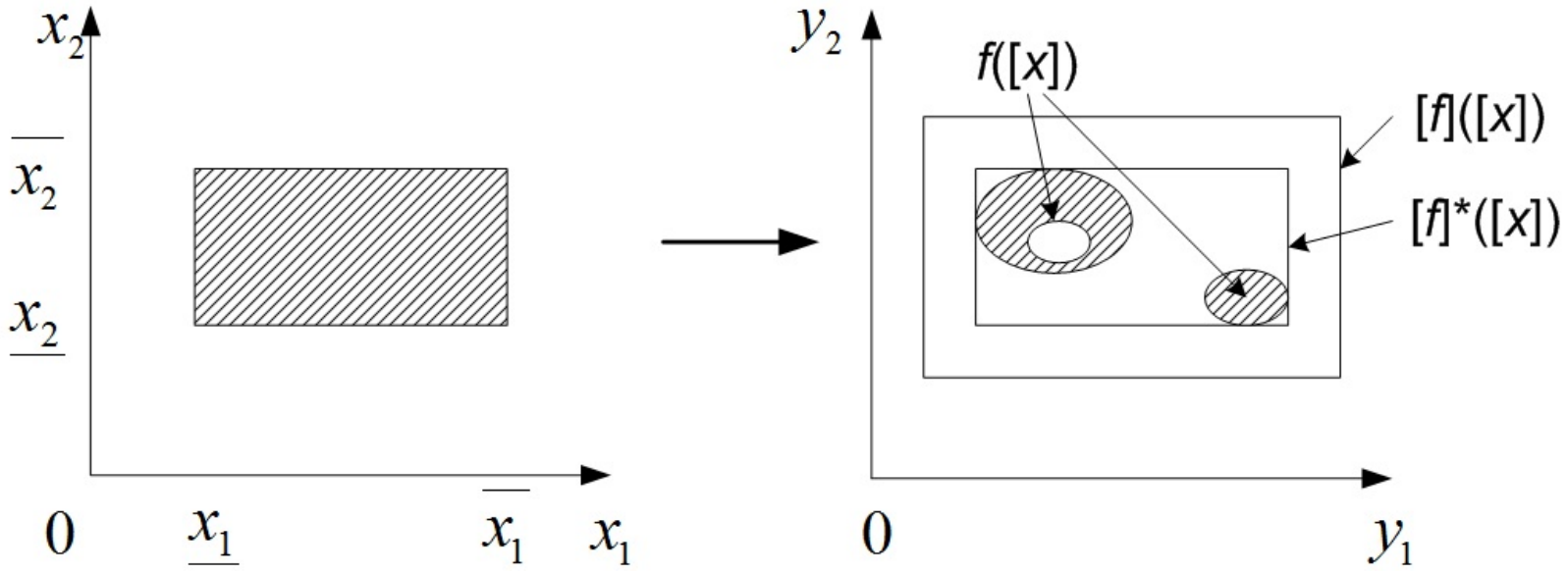
Интервальные алгоритмы в обучении нейронных сетей II

Пример.

$$[f]([x]) = R^m -$$

функция включения для всех функций f.

$$f : R^2 \to R^2$$



Интервальные алгоритмы в обучении нейронных сетей III

$f([x])$ – образ функции,
$[f]([x])$, $[f]^*([x])$ – функции включения,
$[f]^*([x])$ – минимальная функция включения

Функции включения:

- естественная;
- центрированная;
- тейлоровская 2-го порядка.

Функция

$$[f]_n([x]) : IR^n \to IR-$$

естественная для f, если в f заменены:

- каждая вещественная переменная x_i - интервалом $[x_i]$;
- каждый оператор или функция - интервальными аналогами.

Интервальные алгоритмы в обучении нейронных сетей IV

Теорема (о среднем).

Пусть

$$f : R^n \to R-$$

дифференцируема на $[x]$ и $m = mid([x])$ – середина бруса $[x]$. Тогда

$$\forall x \in [x] \exists z \in [x] : f(x) = f(m) + g^T(z)(x - m)$$

где g – градиент f.

Т.о.,

$$\forall x \in [x] : f(x) \in f(m) + [g^T]([x])(x - m)$$

где $[g^T]$ – функция включения для g^T.

Интервальные алгоритмы в обучении нейронных сетей V

Пусть f – дифференцируема на брусе $[x]$.

Центрированная функция включения:

$$[f]_c([x]) = f(m) + [g^T]([x])([x] - m)$$

Для центрированной функции включения (в общем случае неверно для естественной):

$$\lim_{k\to\infty} w([x]_k) = 0 \Rightarrow \lim_{k\to\infty} \frac{w[f]_c([x]_k)}{w(f([x]_k))} = 1$$

Интервальные алгоритмы в обучении нейронных сетей VI

Пример.

$$f(x) = x^2 + \sin x \qquad [x] = \left[\frac{2\pi}{3}, \frac{4\pi}{3}\right] \qquad [y] = \left[\frac{99\pi}{100}, \frac{101\pi}{100}\right]$$

Функции включения:

$$[f]_n([x]) = [x]^2 + sin([x])$$

$$[f]_c([x]) = f(\pi) + [f']([x])([x] - \pi)$$

$f(x)$ – возрастающая на заданных интервалах, минимальная функция включения:

$$[f]^*([x]) = [\underline{x}^2 + \sin(\underline{x}), \overline{x}^2 + \sin(\overline{x})]$$

	$[x] = [2\pi/3; 4\pi/3]$		$[y] = [99\pi/100; 101\pi/100]$	
$[f]$	$[f]([x])$	$\Delta([f]([x]))$	$[f]([y])$	$\Delta([f]([y]))$
$[f]_n$	$[3.52046; 18.41199]$	3.46410	$[9.64178; 10.09940]$	0.12564
$[f]_c$	$[1.62022; 18.11899]$	5.07134	$[9.70163; 10.03758]$	0.00397
$[f]^*$	$[5.25251; 16.67994]$	0	$[9.70641; 10.03658]$	0

Интервальные алгоритмы в обучении нейронных сетей VII

Бисекция бруса $[x]$ по i-й координате –

его разделение на 2 части: $[x^1]$ и $[x^2]$, где

$$[x_i^1] = [\underline{x_i}, mid([x_i])]$$

$$[x_i^1] = [mid([x_i]), \overline{x_i}]$$

остальные координаты-интервалы $(i \neq j)$:

$$[x_j^1] = [x_j^2] = [x_j]$$

Интервальные алгоритмы в обучении нейронных сетей VIII

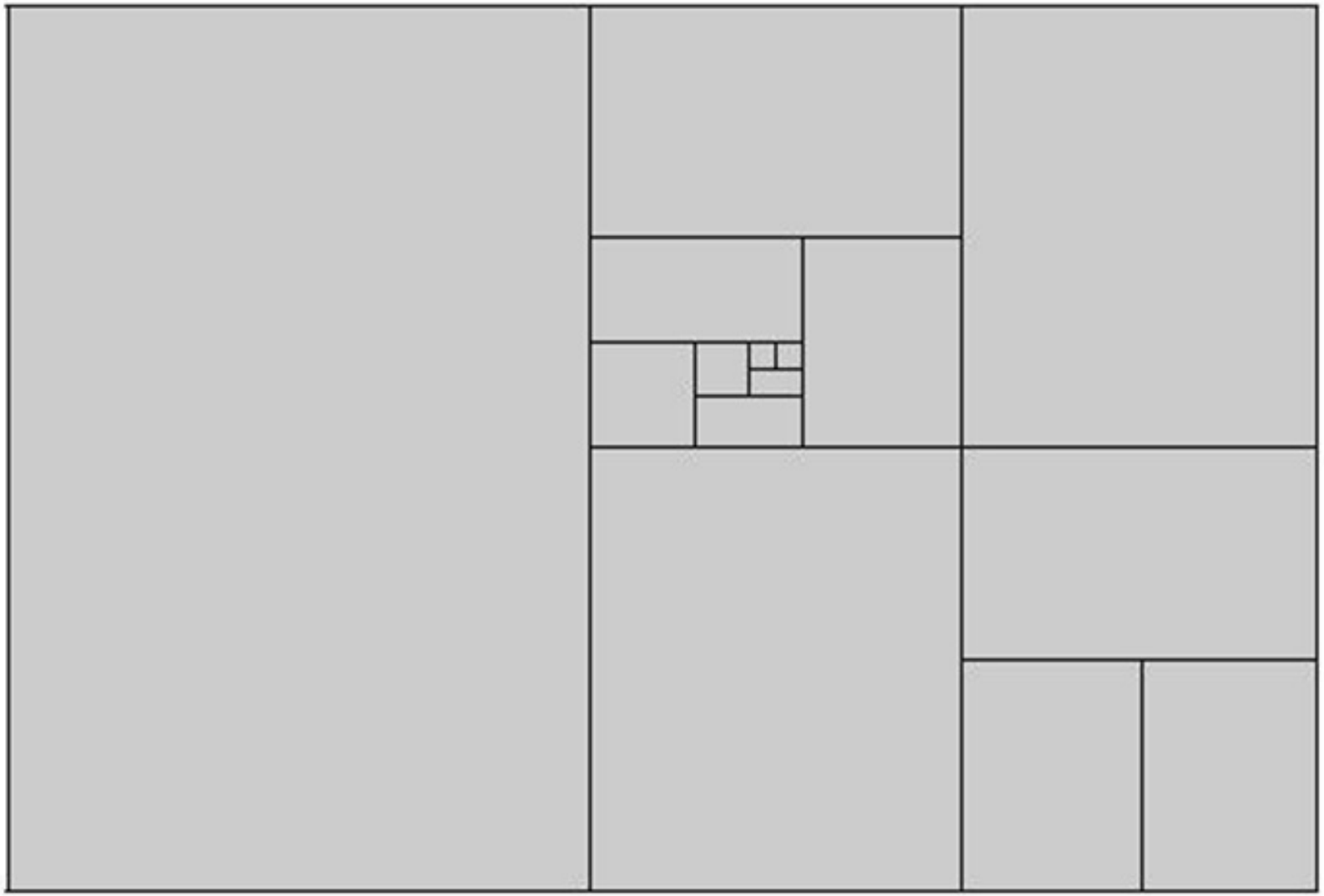
Алгоритм (простейший алгоритм поиска одного глобального оптимума GlobOpt)

Вход: Функция $f(x), x \in R^n$; $[x]$ – начальный брус; минимальная ширина бруса $\varepsilon > 0$

Выход: $[p]$ – брус, содержащий точку глобального минимума; f^* - глобальный минимум

1. Инициализация: $[p] := [x]$
2. Оценка нижней границы минимума: $f^* := [\underline{f}([p])]$
3. Инициализация списка: $L := \{([p], f^*)\}$
4. Цикл: Пока $wid([p]) \geqslant \varepsilon$
   - 4.1. Выбираем компоненту l, по которой брус $[p]$ имеет наибольшую длину: $l := \arg\max wid([p_i])$
   - 4.2. Бисекция $[p]$ по l-й координате на $[p']$ и $[p'']$
   - 4.3. Вычисление: $f' := [\underline{f}]([p'])$ $\quad$ $f'' :== [\underline{f}]([p''])$
   - 4.4. Удаление из L элемента $([p], f^*)$
   - 4.5. Помещение в список L в порядке возрастания 2-го поля
   - 4.6. Обозначаем 1-ю запись через $([p], f^*)$

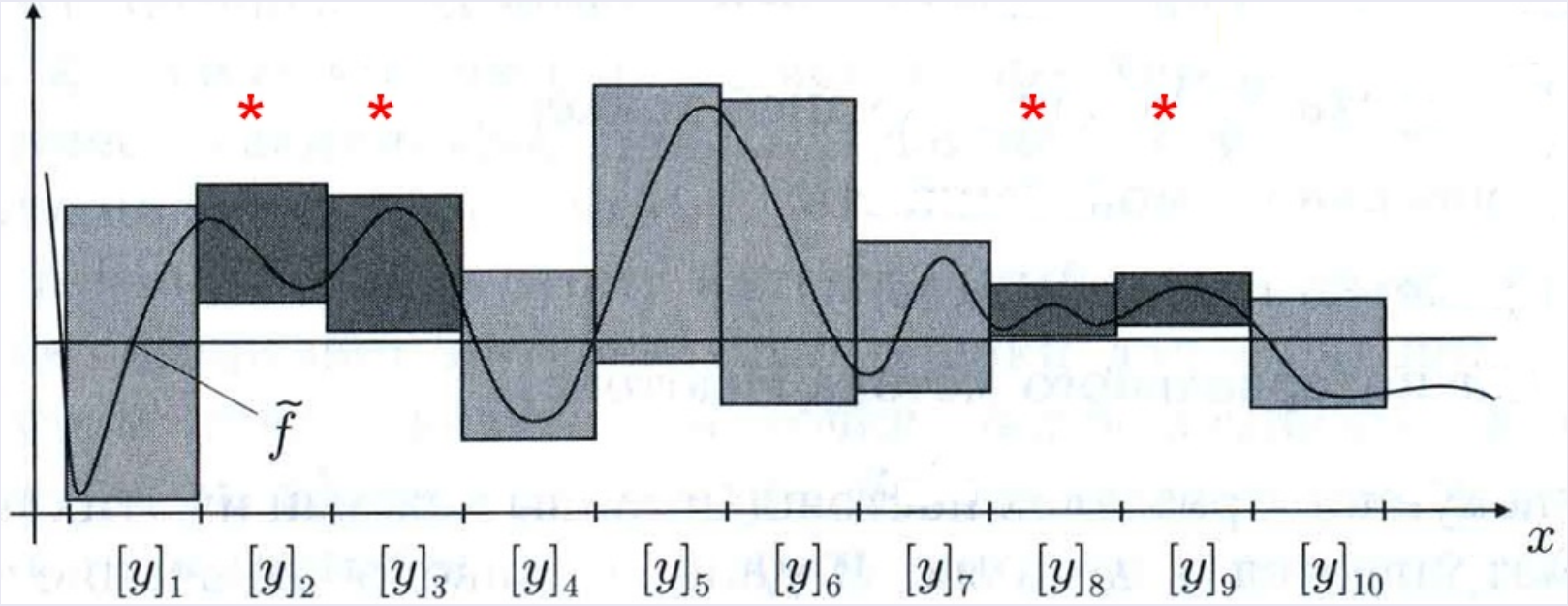
Интервальные алгоритмы в обучении нейронных сетей IX



Интервальные алгоритмы в обучении нейронных сетей X

Тест на значение в средней точке:

неперспективными являются брусы $[x]$ из списка L, если для текущего бруса $[p]$: $[\underline{f}]([x]) > \tilde{f} = f(mid[p])$



Замечание. Вместо средней точки внутри бруса можно найти другую точку (точки) с помощью локальных методов оптимизации.

Интервальные алгоритмы в обучении нейронных сетей XI

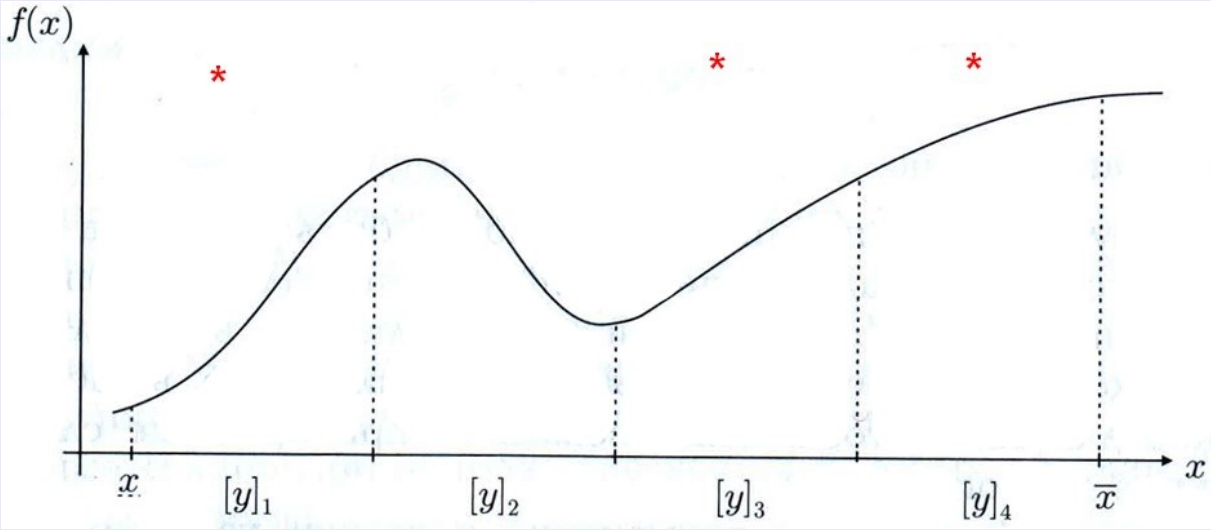
Тест на необходимое условие оптимума (на монотонность):

неперспективными являются брусы $[x]$ из списка L, если функции включения градиентов g не содержат 0:

$$0 \notin g([x])$$

или:

$$\underline{g}([x]) > 0, \qquad \overline{g}([x]) < 0$$



Интервальные алгоритмы в обучении нейронных сетей XII

Алгоритм SIVIA (Set Invertor Via Interval Analyis, Luc Jaulin)

Инвертор $INV(f,[\mathbf{s}],[l],\varepsilon)$ - функция, которая по заданному интервалу $[l]$, функции f и параметру точности ε находит в области поиска $[s]$ множество параллелотопов $\mathrm{P}\mathcal{P}$ такое, что $\forall[\mathbf{x}]\in\mathrm{P}$ справедливо: $[\mathbf{x}]\subseteq[\mathbf{s}],\omega([\mathbf{x}])\geq\varepsilon,[f]([\mathbf{x}])\subseteq[l]$ или $[\mathbf{x}]\subseteq[\mathbf{s}],\omega([\mathbf{x}])<\varepsilon,[f]([\mathbf{x}])\cap[l]\neq\varnothing$.

Шаг 1. Задать функцию f, область поиска $[\mathbf{s}]$, интервал $[l]$ и параметр точности ε. Создать пустое множество параллелотопов P и множество $\mathrm{X}=\{[\mathbf{s}]\}$.

Шаг 2. Для каждого параллелотопа $[\mathbf{x}]\in\mathrm{X}$ найти $[f]([\mathbf{x}])$. Проверить выполнение следующих условий (сам параллелотоп $[\mathbf{x}]$ после проверки условий удаляется из X):

a) если $[f]([\mathbf{x}])\cap[l]=\varnothing$, то параллелотоп $[\mathbf{x}]$ не добавляется в множество P;

b) если $[f]([\mathbf{x}])\subseteq[l]$ или $[f]([\mathbf{x}])\cap[l]\neq\varnothing,\omega([\mathbf{x}])<\varepsilon$, добавить параллелотоп $[\mathbf{x}]$ в множество P;

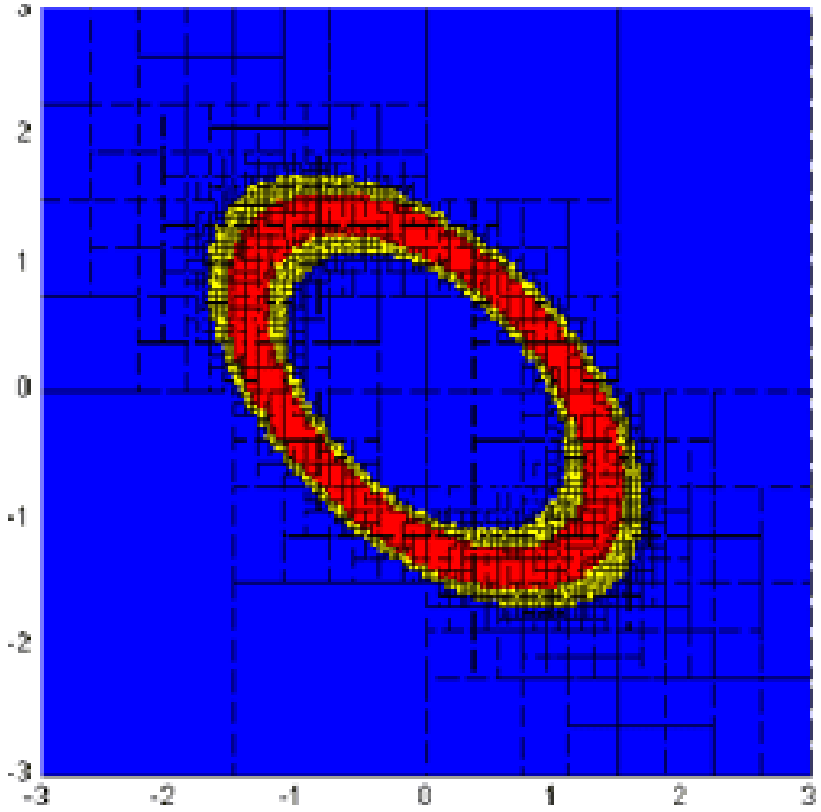
c) в противном случае задать параллелотопы

$$[\mathbf{x}]'=[x_1]_i\times\ldots\times\left[mid([x_k]_i);\overline{[x_k]_i}\right]\times\ldots\times[x_n]_i \quad \text{и} \quad [\mathbf{x}]''=[x_1]_i\times\ldots\times\left[\underline{[x_k]_i};mid([x_k]_i)\right]\times\ldots\times[x_n]_i,$$

где k – номер компоненты параллелотопа $[\mathbf{x}]_i$ с наибольшей шириной, и добавить их в множество X.

Шаг 3. Если $\mathrm{X}\neq\varnothing$, то перейти к шагу 2, в противном случае – к шагу 4.

Шаг 4. Вернуть P.


3
2
1
0
-1
-2
-3
-3
-2
-1
0
1
2
3

1 Функционалы качества обучения, отличные от квадратичного

2 Основы интервального анализа

3 Интервальные алгоритмы в обучении нейронных сетей

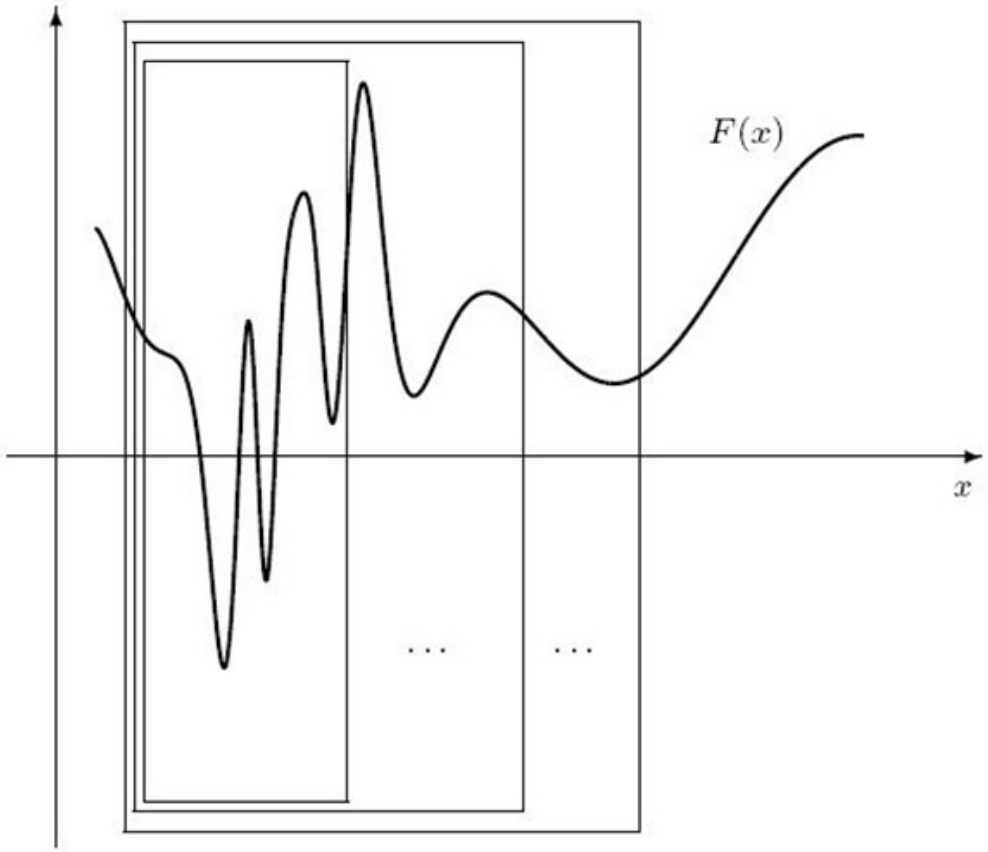
4 Интервальные генетические алгоритмы

5 Заключение

6 Литература

Интервальные генетические алгоритмы I

Проблемы при применении интервальных методов – застаивание оценки



Интервальные генетические алгоритмы II

Алгоритм(случайного интервального дробления). Часть 1

Вход

Брус $\boldsymbol{X} \subset \mathbb{R}^n$. Количество испытаний N.

Интервальное расширение целевой функции $\boldsymbol{F} : \mathbb{I}\boldsymbol{X} \to \mathbb{IR}$.

Выход

Оценка F^* глобального минимума функции F на $\boldsymbol{X}$.

Алгоритм

$\boldsymbol{Y} \leftarrow \boldsymbol{X}$;

вычисляем $\boldsymbol{F}(\boldsymbol{Y})$ и присваиваем $F^* \leftarrow \underline{\boldsymbol{F}(\boldsymbol{Y})}$;

инициализируем рабочий список $\mathcal{L}$ записью $\{\boldsymbol{Y}\}$;

DO FOR $j = 1$ TO N

случайным образом выбираем из списка $\mathcal{L}$ брус $\boldsymbol{Z}$;

рассекаем брус $\boldsymbol{Z}$ по самой длинной компоненте пополам на брусы-потомки $\boldsymbol{Z}'$ и $\boldsymbol{Z}''$;

удаляем из списка $\mathcal{L}$ брус $\boldsymbol{Z}$, помещаем в $\mathcal{L}$ брусы $\boldsymbol{Z}'$ и $\boldsymbol{Z}''$;

Интервальные генетические алгоритмы III

Часть 2:

вычисляем оценки $\phi' \leftarrow \underline{\boldsymbol{F}(\boldsymbol{Z}')}$ и $\phi'' \leftarrow \underline{\boldsymbol{F}(\boldsymbol{Z}'')}$;

IF ($Z = Y$) THEN

присваиваем $F^* \leftarrow \min\{\phi', \phi''\}$;

ELSE

присваиваем $F^* \leftarrow \min\{F^*, \phi', \phi''\}$

END IF

если F^* изменилась, то через $\boldsymbol{Y}$ обозначаем тот брус — Z' или Z'', который обеспечил новую оценку F^*;

END DO

Можно динамически назначать более высокую вероятность выбора брусов с меньшим значением функции (**алгоритм случайного интервального дробления с приоритетом**).

Интервальные генетические алгоритмы IV

Фитнес-функция:

$$\mathbf{F(b)} = \sum_{i=1}^{n} (\alpha_i \cdot \operatorname{wid} \mathbf{b_i}) + \beta \cdot \underline{\mathbf{f(b)}} + \gamma \cdot \operatorname{wid} \mathbf{f(b)}$$

$\mathbf{f(b)}$ – интервальная оценка целевой функции f на брусе $\mathbf{b}$;
$\underline{\mathbf{f(b)}}$ – нижняя граница интервальной оценки (оценка снизу);
$\overline{\operatorname{wid}}\ \mathbf{f(b)}$ – ширина (точность) интервальной оценки;
$\operatorname{wid} \mathbf{b_i}$ – ширина i-той стороны бруса;
α_i, β, γ – весовые коэффициенты.

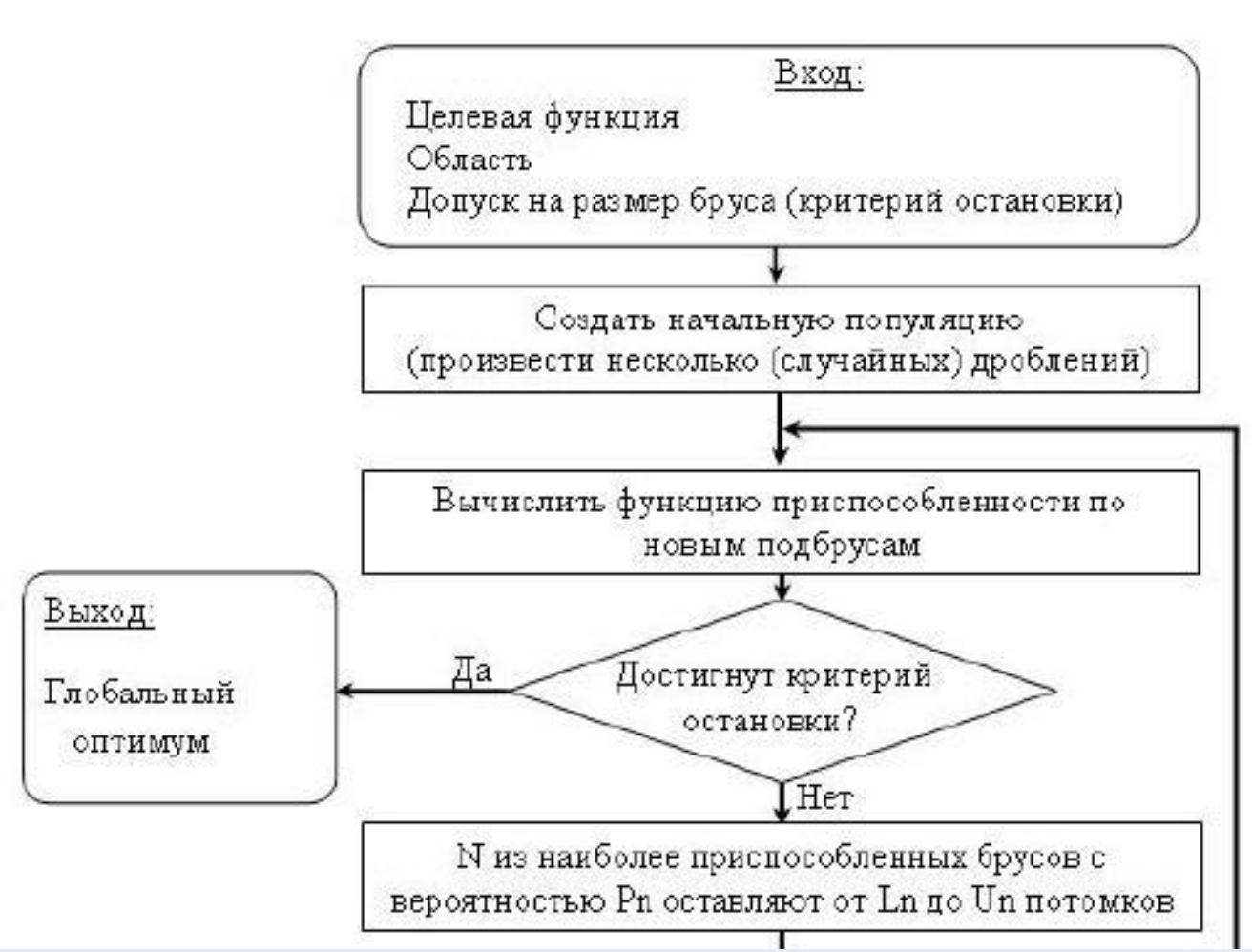
Интервальные генетические алгоритмы V

Параметры:

- вероятности P_n и P_m;
- минимальные количества потомков L_n и L_m;
- максимальное количество потомков U_n и U_m;
- величина N – сколько объектов, начиная с самого приспособленного, могут оставить потомство;
- брусы дробятся на равные части, или разбиение происходит в случайной пропорции, «разновесные» дети;
- количество особей в начальной популяции.

Интервальные генетические алгоритмы VI

Интервальный генетический алгоритм. Часть 1:



Интервальные генетические алгоритмы VII

Часть 2:



1 Функционалы качества обучения, отличные от квадратичного

2 Основы интервального анализа

3 Интервальные алгоритмы в обучении нейронных сетей

4 Интервальные генетические алгоритмы

5 Заключение

6 Литература

Заключение

1. Вычислительные погрешности могут привести к катастрофе.
2. Интервальные вычисления – способ проведения достоверных расчетов.
3. Важные свойства арифметических операций с интервалами – монотонность по включению.
4. Функции включения предназначены для быстрого оценивания образа функции.
5. Интервальный алгоритм позволяет гарантировать глобальность найденного решения.
6. Для увеличения эффективности вычислительных методов применяются элементы стохастической оптимизации.
7. Возможно комбинирование методов интервальной оптимизации с генетическими алгоритмами.

1 Функционалы качества обучения, отличные от квадратичного

2 Основы интервального анализа

3 Интервальные алгоритмы в обучении нейронных сетей

4 Интервальные генетические алгоритмы

5 Заключение

6 Литература

ЛИТЕРАТУРА

1. Баженов А.Н., Жилин С.И., Кумков С.И., Шарый С.П. Обработка и анализ интервальных данных.– Издательство «ИКИ», 2024.– 356 с.
2. Androsov A. IntvalPy. Code released on Github, 2021.– https://github.com/AndrosovAS/intvalpy/
3. ByTofigh Allahviranloo, Witold Pedrycz, Armin Esfandiari. Advances in Numerical Analysis Emphasizing Interval Data, 2022.– 218 p. DOI: https://doi.org/10.1201/9781003218173

1 Функционалы качества обучения, отличные от квадратичного

2 Основы интервального анализа

3 Интервальные алгоритмы в обучении нейронных сетей

4 Интервальные генетические алгоритмы

5 Заключение

6 Литература

СПАСИБО ЗА ВНИМАНИЕ!

Построение и применение нейронных сетей и генетических алгоритмов
Лекция 5
Самоорганизующиеся карты Кохонена и их обучение

САРАЕВ ПАВЕЛ ВИКТОРОВИЧ

РТУ МИРЭА
Кафедра прикладной математики

Москва – 2026

ПЛАН ЛЕКЦИИ

1. Самоорганизующиеся карты Кохонена SOM
2. Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает все» и его модификации
3. Визуализация многомерных данных
4. Векторное квантование
5. Заключение
6. Литература

1 Самоорганизующиеся карты Кохонена SOM

2 Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает все» и его модификации

3 Визуализация многомерных данных

4 Векторное квантование

5 Заключение

6 Литература

Самоорганизующиеся карты Кохонена SOM I

Самоорганизующиеся карты (СОК, self-organizing maps, SOM, сети) Кохонена (1984?) –

класс НС, применяемых в задачах **кластеризации** (таксономии).

Первая работа – распознавание речи:

T. Kohonen, «Automatic formation of topological maps of patterns in a self-organizing system,» 2SCIA, Scand. Conf. on Image Analysis, Helsinki, Finland, pp. 214-220 (1981).

Teuvo Kohonen –

финский учёный в области ИТ и ИИ, 11.07.1934–13.12.2021

Идея:

исследованиями коры головного мозга – нейроны со схожими функциями обычно расположены рядом.

Самоорганизующиеся карты Кохонена SOM II

Цель кластеризации –

компактно описать информацию, заложенную в исходных данных (уменьшить количество данных).

Задача кластеризации –

разбиение множества объектов, представленных векторами $x^{(i)} \in \mathbb{R}^n$, $i = 1, \ldots, k$,

$$\begin{bmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(k)} \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \ldots & x_n^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \ldots & x_n^{(2)} \\ \vdots & \vdots & \vdots & \vdots \\ x_1^{(k)} & x_2^{(k)} & \ldots & x_n^{(k)} \end{bmatrix}$$

на группы, называемые **кластерами** или **таксонами**.

Самоорганизующиеся карты Кохонена SOM III

Принцип группировки объектов:

в пределах одного кластера они более похожи друг на друга, чем на объекты других кластеров (альтернативная постановка: чтобы объекты в пределах одного кластера были **эквивалентными**).

Ядра $w^{(i)}$ –

типичные представители каждого кластера , $i = 1, \ldots, m$, где m — количество кластеров (рис. 1).

Самоорганизующиеся карты Кохонена SOM IV

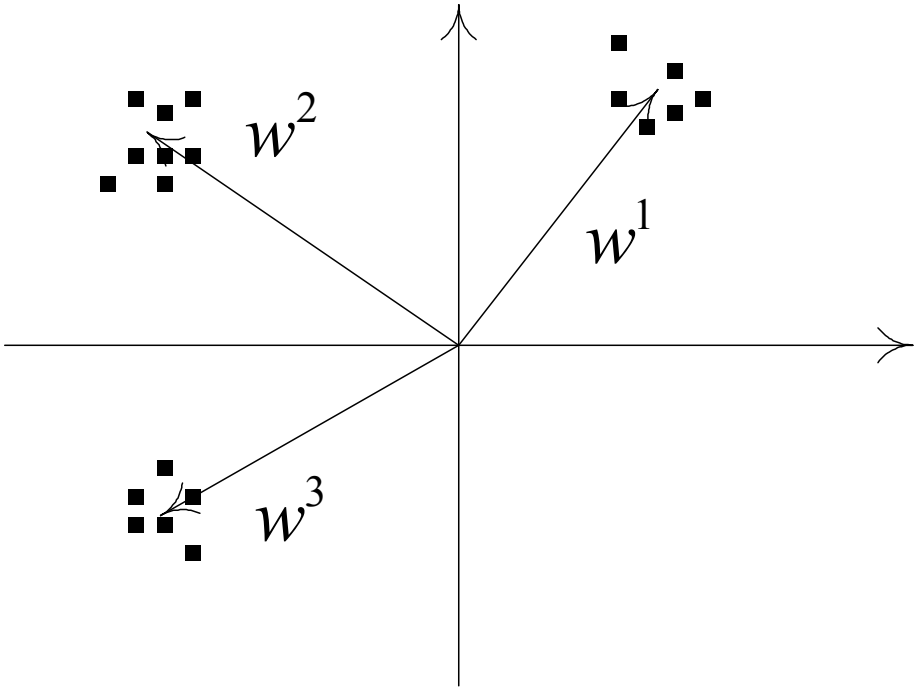


Рис. 1. Кластеры и их ядра

Самоорганизующиеся карты Кохонена SOM V

Оценка степени близости (сходства) объектов:

расстояние $d(\cdot, \cdot)$.
Чаще всего – евклидово расстояние или (для упрощения вычислений) его квадрат:

$$d^2(x^{(i)}, x^{(j)}) = \|x^{(i)} - x^{(j)}\|^2 = \sum_{l=1}^{n} (x_l^{(i)} - x_l^{(j)})^2, \quad (1)$$

где $x_l^{(i)}$ — l-й элемент вектора x^i.
Чем меньше расстояние, тем ближе друг к другу они располагаются, тем более похожи объекты, описываемые этими векторами.

Самоорганизующиеся карты Кохонена SOM VI

Пусть число кластеров m известно.

Обозначение:

ядро кластера, к которому относится вектор $x^{(i)}$, через

$$w(x^{(i)}).$$

Формализация задача кластеризации:

найти m ядер $w^{(i)}$ и разбить объекты на кластеры таким образом, чтобы

$$Q = \sum_{i=1}^{k} d^2\left(x^{(i)}, w(x^{(i)})\right) \to \min.$$

Самоорганизующиеся карты Кохонена SOM VII

В пространстве $\mathbb{R}^n$ задано **скалярное произведение** векторов

$$\langle x^{(i)}, x^{(j)} \rangle = \sum_{l=1}^{n} x_l^{(i)} x_l^{(j)}.$$

Рассмотрим квадрат нормы разности произвольного вектора $x^{(i)}$ и ядра некоторого кластера $w^{(j)}$, используя скалярное произведение:

$$\begin{aligned} \|x^{(i)} - w^{(j)}\|^2 = \langle x^{(i)} - w^{(j)}, x^{(i)} - w^{(j)} \rangle = \\ = \langle x^{(i)}, x^{(i)} \rangle - 2\langle x^{(i)}, w^{(j)} \rangle + \langle w^{(j)}, w^{(j)} \rangle = \\ = \|x^{(i)}\|^2 - 2\langle x^{(i)}, w^{(j)} \rangle + \|w^{(j)}\|^2. \end{aligned} \quad (2)$$

Если ядра кластеров являются **нормированными**, т.е. $\|w^{(j)}\| = 1$, $j = 1, \ldots, m$, то (2) можно записать в виде:

$$\|x^{(i)} - w^{(j)}\|^2 = \|x^{(i)}\|^2 - 2\langle x^{(i)}, w^{(j)} \rangle + 1. \quad (3)$$

Самоорганизующиеся карты Кохонена SOM VIII

Из (3): что ближайшим к вектору $x^{(i)}$ будет то ядро $w^{()}$, для которого **скалярное произведение** $\langle x^{(i)}, w^{(j)} \rangle$ является **максимальным**.

Замечание:

$\langle x^{(i)}, w^{(j)} \rangle$ – проекция вектора $x^{(i)}$ на направление $w^{(j)}$.

Рассмотрим НС, состоящую из входного слоя нейронов и еще одного (**слоя Кохонена**), у нейронов которого функция активации является единичной.
Входной слой состоит из n нейронов, а слой Кохонена – из m.
Вектор весов $w^{(j)}$ j-го нейрона слоя Кохонена – **нормированное ядро** j-го кластера.

Самоорганизующиеся карты Кохонена SOM IX

Алгоритм работы НС Кохонена:

1. Вектор $x^{(i)}$ подается на вход НС.
2. Для каждого j-го нейрона слоя Кохонена вычисляется скалярное произведение $\langle x^{(i)}, w^{(j)} \rangle$.
3. Среди скалярных произведений выбирается нейрон r, дающий максимальное значение:
$$m_0 = \arg \max_{j=1}^{m} \langle x^{(i)}, w^{(j)} \rangle. \quad (4)$$
4. Номер нейрона m_0 соответствует номеру кластера, к которому относится выбранный объект.

НС такой структуры, функционирующая по принципу выбора максимального скалярного произведения – **самоорганизующиеся карты (СОК) Кохонена, Self-Organising Maps, SOM** (рис. 2).

Самоорганизующиеся карты Кохонена SOM X

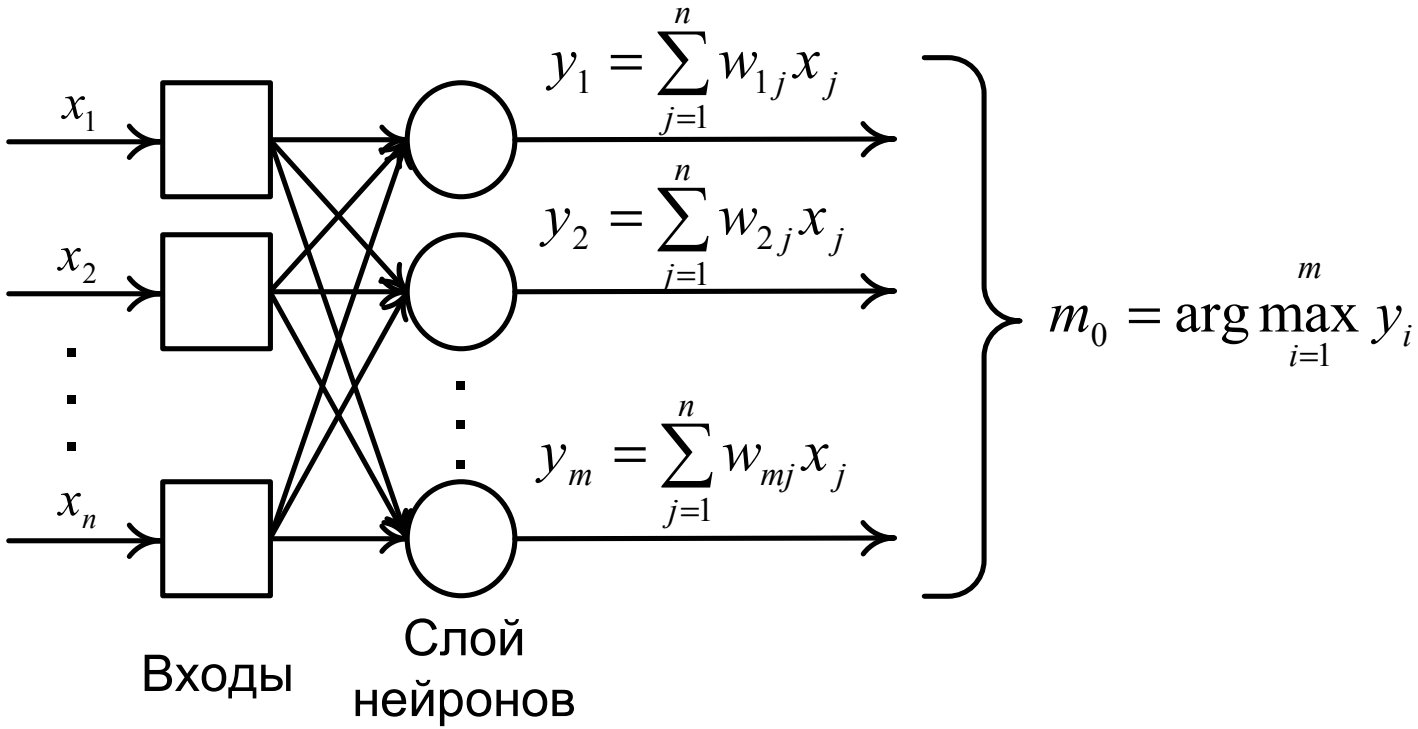


Рис. 2. Самоорганизующиеся карты Кохонена

Самоорганизующиеся карты Кохонена SOM XI

Особенности СОК Кохонена:

- Количество кластеров совпадает с числом нейронов Кохонена.
- Весовые вектора нормированы.
- Весовые векторы нейронов слоя Кохонена – ядра.

1 Самоорганизующиеся карты Кохонена SOM

2 Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает все» и его модификации

3 Визуализация многомерных данных

4 Векторное квантование

5 Заключение

6 Литература

Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает все» и его модификации I

Обучение СОК – задача **обучения без учителя** (реакция сети на входной вектор заранее неизвестна, т.к. неизвестны кластеры, к которым относятся объекты).
Классический алгоритм обучения СОК Кохонена – «Победитель забирает все» («Winner takes all»).
Название такое, т.к. на каждом шаге цикла происходит обновление только значений весов нейрона, имеющего максимальное скалярное произведение.

Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает все» и его модификации II

Обучение СОК Кохонена «Победитель забирает все»

1. **Задание** константы обучения $\eta \in [0.1, 0.7]$ и количества циклов обучения N_c.
2. **Инициализация**. Весовым коэффициентам нейронов слоя Кохонена присваиваются небольшие случайные числа:

$$w_j^{(i)} \in [-1; 1], \quad i = 1, \ldots, m, \quad j = 1, \ldots, n.$$

3. **Нормализация** векторов весов нейронов:

$$w^{(i)} := \frac{w^{(i)}}{\|w^{(i)}\|}, \quad i = 1, \ldots, m.$$

Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает все» и его модификации III

4. N_c циклов по всем векторам $x^{(i)}$, $i = 1, \ldots, k$.

4.1. **Определение нейрона-победителя (Best Matching Unit, BMU)** по формуле (4).

4.2. **Корректировка** весов нейрона:

$$w_{m_0} := w_{m_0} + \eta(x^{(i)} - w_{m_0}).$$

4.3. **Нормализация** вектора весов нейрона-победителя:

$$w^{m_0} := \frac{w^{m_0}}{\|w^{m_0}\|}.$$

Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает все» и его модификации IV

Замечание.

На шаге 4.2 алгоритма 15 по сути находится выпуклая комбинация векторов w^{m_0} и $x^{(i)}$, т.к.

$$w_{m_0} := w_{m_0} + \eta(x^{(i)} - w_{m_0}) = \eta x^{(i)} + (1-\eta)w_{m_0}.$$

В результате обновления весов ядро кластера w^{m_0} несколько приближается к вектору $x^{(i)}$ (рис. 3). Чем больше η, тем сильнее изменяется вектор w^{m_0}.

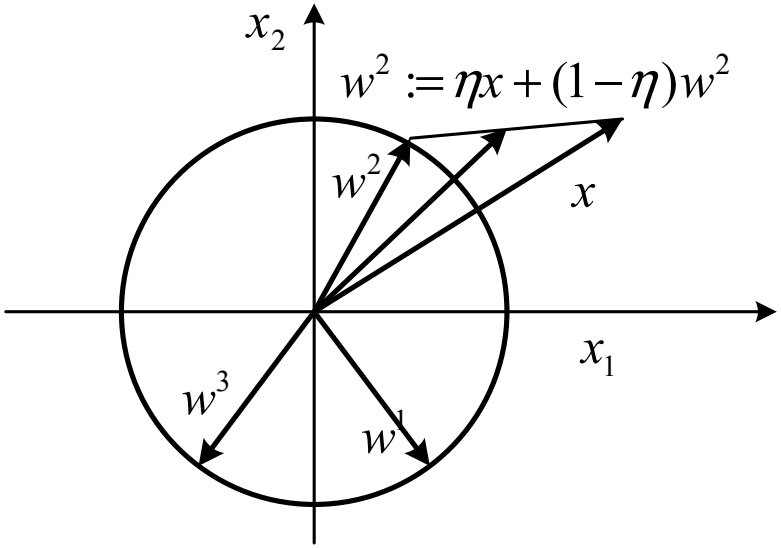


Рис. 3. Обновление весов нейрона-победителя

Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает все» и его модификации V

Алгоритм «Победитель забирает все» является не очень эффективным методом обучения:вследствие инициализации весов большинство векторов $x^{(i)}$ может попадать в один и тот же кластер, т.е. некоторые нейроны могут оказаться незадействованными в процессе обучения. Это так называемая проблема **мертвых нейронов**.

Модификации обучения СОК Кохонена:

- Уменьшение с течением времени (каждой цикл) константы обучения, т.е.

$$\eta = \eta(t) \to 0, \quad t \to \infty.$$

- Обновление весов нейронов пропорционально величине скалярного произведения. В этом случае на каждом шаге цикла участвует не только нейрон-победитель.

Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает все» и его модификации VI

- Использование «чувства справедливости»: для часто выигрывающего нейрона уменьшается вероятность обновления весов. Частный случай – «механизм утомления», когда каждому нейрону приписывается потенциал p_i. В определении победителя используются только те нейроны, у которых $p_i \geqslant p_{min}$, где p_{min} – некоторый пороговый потенциал. Если обновляются веса m_0-го нейрона, то для него полагают

$$p_{m_0} := p_{m_0} - p_{min}.$$

В случае неучастия нейрона в обновлении весов потенциал увеличивается:

$$p_i := \max\left\{1, p_i + \frac{1}{m}\right\}.$$

В начальный момент времени: $p_i = \frac{1}{m}, i = 1, \ldots, m$. Если $p_{min} = 0$, то получается классический алгоритм «Победитель забирает все». Хорошие результаты получаются при $p_{min} \approx 0,75$.

1 Самоорганизующиеся карты Кохонена SOM

2 Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает все» и его модификации

3 Визуализация многомерных данных

4 Векторное квантование

5 Заключение

6 Литература

Визуализация многомерных данных I

СОК Кохонена могут применяться для двумерной визуализации многомерных данных. Нейроны слоя Кохонена представляются (рис. 4) в одном из видов:

- гексагональной (6-угольной) сетки;
- прямоугольной (4-угольной) сетки.

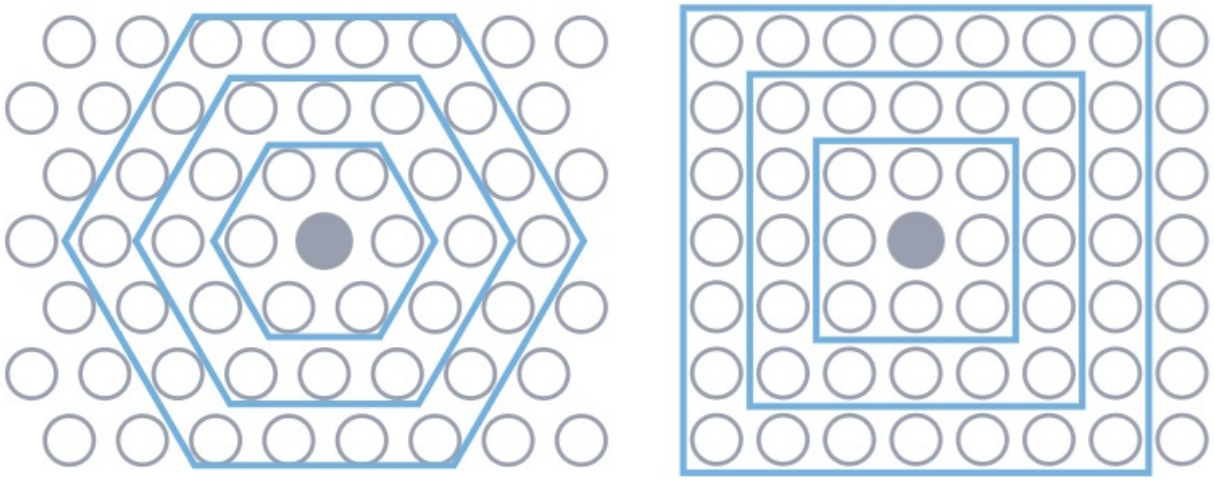


Рис. 4. Гексагональная и четырехугольная сетка

Количество нейронов определяет степень детализации результата.

Визуализация многомерных данных II

Формальная близость нейронов слоя Кохонена позволяет **модифицировать** значения весов нейронов **в некоторой окрестности** нейрона-победителя, пропорционально расстоянию.
В этом случае η – функция соседства нейронов:

$$\eta = \eta(d).$$

Здесь η – невозрастающая функцию от расстояния между нейроном-победителем и соседними нейронами (и времени, если учитывается).
Варианты функции соседства:

- константа:

$$\eta(d) = \begin{cases} const, & d \leqslant s; \\ 0, & d > s, \end{cases}$$

  где s – радиус обучения;

- гауссовского типа:

$$\eta(d) = e^{-\frac{d}{2s}},$$

  где s – радиус обучения.

Визуализация многомерных данных III

Алгоритм обучения с использованием функции соседства – «Победитель получает больше» (Winner Takes Most). Величина шага определяется на шаге t как

$$\eta = \eta(d) \cdot a(t),$$

где $a(t)$ – убывающая по t функция, например

$$a(t) = \frac{A}{t + B},$$

где A, B – некоторые константы.
Итог обучения: похожие объекты исходных данных оказываются рядом на карте.

Визуализация многомерных данных IV

Варианты визуализации (раскраски ячеек) СОК:

- в зависимости от значений некоторой компоненты векторов весов (меньше значения – темнее, больше – светлее);
- по матрице расстояний (U-Matrix) (расстояния до соседних нейронов; меньше расстояния – темнее, больше – светлее);
- на основе кластеризации нейронов слоя Кохонена (например, методом k-средних) (одинаковые кластеры – один цвет);
- по меткам (классам) данных, если они известны (по самой частой метке или пропорционально количеству меток).

Визуализация многомерных данных V

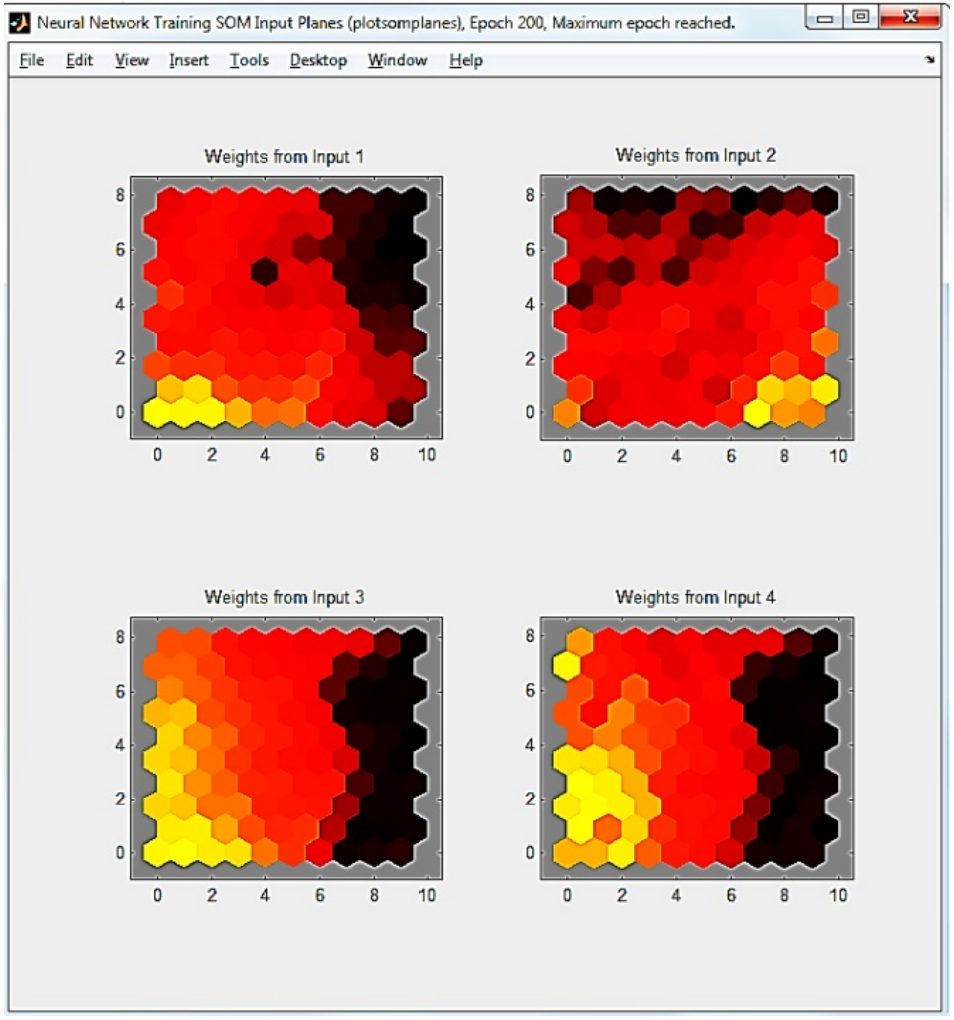


Рис. 5. Пример визуализации

Визуализация многомерных данных VI

Методы начальной инициализации весов:

- случайная – выбор случайного значения для каждой компоненты вектора весов из некоторого диапазона на основе р/р из $[0; 1]$ или гауссовского;
- инициализация примерами – значения случайно выбранных примеров из множества данных;
- линейная – упорядоченное начальное состояние весов СОК:
  - 2 наибольших собственных значения и вектора матрицы

$$K = \frac{1}{k-1}\tilde{X}(\tilde{X})^T,$$

    где $\tilde{X}$ – матрица из центрированных входных векторов (с 0-м математическим ожиданием);
  - размеры сетки равны 2 наибольшим собственным значениям;
  - начальные значения – линейные комбинации 2 собственных векторов, веса – координаты нейрона.

1 Самоорганизующиеся карты Кохонена SOM

2 Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает все» и его модификации

3 Визуализация многомерных данных

4 Векторное квантование

5 Заключение

6 Литература

Векторное квантование I

Векторное квантование (Learning Vector Quantization, LVQ) –

развитие SOM. Подход предложен Т. Кохоненом.

Особенности LVQ:

- количество кластеров m;
- количество классов p ($p \leqslant m$) (!);
- принадлежность каждого объекта классу (см. задачу классификации).

Цель LVQ –

минимизация ошибки классификации

Векторное квантование II

Отличия СОК и LVQ:

- СОК:
  - обучение без учителя;
  - сохранение топологических свойств входного пространства;
  - обновление весов нейрона-победителя и его соседей;
- LVQ:
  - обучение с учителем (используются метки классов);
  - минимизация ошибки классификации;
  - обновление только нейрона-победителя.

Обобщенный алгоритм векторного квантования:

1. Обучение СОК на данных без меток.
2. Использование весов СОК как начальные веса для LVQ.
3. Дообучение LVQ с использованием меток для тонкой настройки классификации.

Векторное квантование III

Обучение LVQ для вектора x, который относится к классу c:

1. Определение нейрона-победителя w_{m_0} (самый близкий к x).
2. Если c совпадает с меткой нейрона, притягиваем вектор весов к данным:

$$w_{m_0} = w_{m_0} + \eta(x - w_{m_0}).$$

3. Если c не совпадает с меткой нейрона, отталкиваем вектор весов от данных:

$$w_{m_0} = w_{m_0} - \eta(x - w_{m_0}).$$

1 Самоорганизующиеся карты Кохонена SOM

2 Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает все» и его модификации

3 Визуализация многомерных данных

4 Векторное квантование

5 Заключение

6 Литература

Заключение

1. Самоорганизующиеся карты Кохонена (СОК) – специальный класс НС.
2. Задача минимизации расстояния между векторами преобразуется в задачу максимизации скалярного произведения векторов.
3. Обучение СОК – обучение без учителя.
4. Классический алгоритм обучения СОК – алгоритм «Победитель забирает все».
5. Проблема классического алгоритма – отсутствие обновления весов ряда «мертвых» нейронов. Выход – использования «чувства справедливости».

1 Самоорганизующиеся карты Кохонена SOM

2 Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает все» и его модификации

3 Визуализация многомерных данных

4 Векторное квантование

5 Заключение

6 Литература

ЛИТЕРАТУРА

1. J. Zbontar, L. Jing, I. Misra, et al. Barlow Twins: Self-Supervised Learning via Redundancy Reduction. CoRR, abs/2103.03230, 2021.– https://arxiv.org/pdf/2103.03230
2. R. Balestriero, M. Ibrahim, V. Sobal, et al. A Cookbook of Self-Supervised Learning. CoRR, abs/2304.12210, 2023.– https://arxiv.org/pdf/2304.12210
3. Dorota Celinska-Kopczynska and Eryk Kopczynski.Non-euclidean self-organizing maps.In Luc De Raedt, editor, Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI 2022, Vienna, Austria, 2022.– P. 1938–1944.– ijcai.org,2022

1 Самоорганизующиеся карты Кохонена SOM

2 Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает все» и его модификации

3 Визуализация многомерных данных

4 Векторное квантование

5 Заключение

6 Литература

СПАСИБО ЗА ВНИМАНИЕ!

Построение и применение нейронных сетей и генетических алгоритмов
Лекция 6
Сверточные и глубокие нейронные сети

САРАЕВ ПАВЕЛ ВИКТОРОВИЧ

РТУ МИРЭА
Кафедра прикладной математики

Москва – 2026

ПЛАН ЛЕКЦИИ

1. Глубокие нейронные сети
2. Сверточные нейронные сети ЛеКуна
3. Особенности архитектур глубоких нейронных сетей
4. Применение СНС
5. Глубокое обучение и его особенности
6. Метод стохастического градиента и mini-batch метод
7. Заключение
8. Литература

1 Глубокие нейронные сети

2 Сверточные нейронные сети ЛеКуна

3 Особенности архитектур глубоких нейронных сетей

4 Применение СНС

5 Глубокое обучение и его особенности

6 Метод стохастического градиента и mini-batch метод

7 Заключение

8 Литература

Глубокие нейронные сети I

Глубокие НС (deep NN) –

НС с количеством **скрытых слоев** $\geqslant 2$.

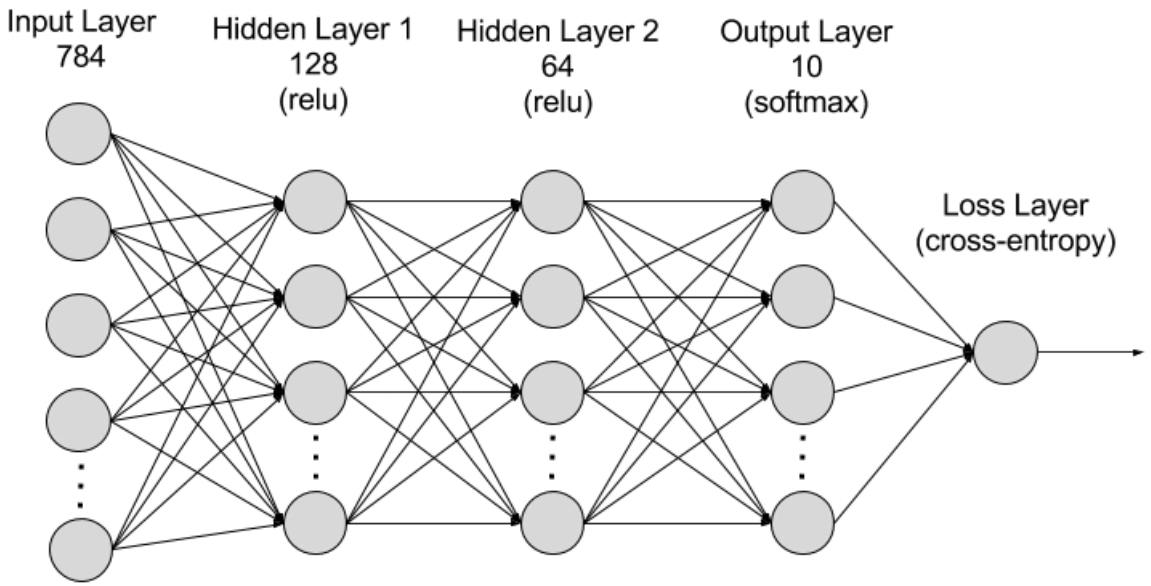


Рис. 1. Пример глубокой НС

Глубокие нейронные сети II

СНС и глубокие НС:

СНС – тип глубокой НС.
Deep – скорее маркетинговый термин.

Архитектуры глубоких НС:

- многослойные перцептроны;
- сверточные нейронные сети;
- рекуррентные нейронные сети (RNN) (неглубокие RNN можно рассматривать как часть глубокого обучения, т.к. разворачиваются во времени).

Глубокие нейронные сети III

Основные отличия многослойных перцептронов и глубоких НС:

- в глубоких НС допускается наличие обратных связей;
- количество параметров (весов) в НС прямого распространения **невелико** по сравнению с глубокими НС (как правило);
- при примерно одинаковом количестве параметров глубокие НС реализуют более высокую степень нелинейности модели.

Следствия:

- обычные НС требуют меньше вычислительных и временных затрат для обучения и применения;
- требуется существенной больший объем обучающего множества.

1 Глубокие нейронные сети

2 Сверточные нейронные сети ЛеКуна

3 Особенности архитектур глубоких нейронных сетей

4 Применение СНС

5 Глубокое обучение и его особенности

6 Метод стохастического градиента и mini-batch метод

7 Заключение

8 Литература

Сверточные нейронные сети (СНС) I

Проблемы

Вход – цветное изображение размером 224×224 пикселей, 3 цветовых канала (RGB). Число входных нейронов:

$$224 * 224 * 3 = 150528.$$

Пусть 1 скрытый слой содержит 100 нейронов, количество весовых коэффициентов между слоями: $\approx 15 \cdot 10^6$.
Проблемы:

- проклятие размерности – огромное количество параметров, требование гигантского множества данных для обучения;
- потеря пространственной структуры – отсутствие связи между соседними пикселями.

Сверточные нейронные сети (СНС) II

Идея – из биологии

Hubel, Wiesel, 1962: Зрительная кора имеет небольшие участки клеток, которые чувствительны к конкретным областям поля зрения. Некоторые нейроны активируются, когда воспринимают вертикальные границы, другие – когда горизонтальные или диагональные. Человек смотрит на собаку: классифицирует, если видит 4 лапы, характерный силуэт, уши и т.д.

Распознавание изображений

Компьютер выполняет классификацию изображений путем поиска низкоуровневых объектов: краев, кривых, собирая их в более высокоуровневые объекты. Это идея реализована в **сверточных нейронных сетях** (СНС) – **Convolutional Neural Networks** (CNN).

История

Архитектура СНС предложена Яном Лекуном (LeCun) в 1988 году.

Сверточные нейронные сети (СНС) III

Входы

Цветное изображение в формате JPG размера 480×480 в палитре RGB: массив $480 \times 480 \times 3$ чисел из диапазона 0..255.

Выходы

Вероятности того, что изображение относится к определенному классу.

Идея СНС

Переход (**не отказываются полностью**) от полносвязных слоев к **сверточным (convolution)** слоям и слоям **объединения, подвыборки (pooling, subsampling)**.
Комбинирование (чередование) слоев.
Структура – однонаправленная, принципиально многослойная (относится к группе **глубоких НС, deep NN**). Функции активации нейронов – по выбору разработчика.

Сверточные нейронные сети (СНС) IV

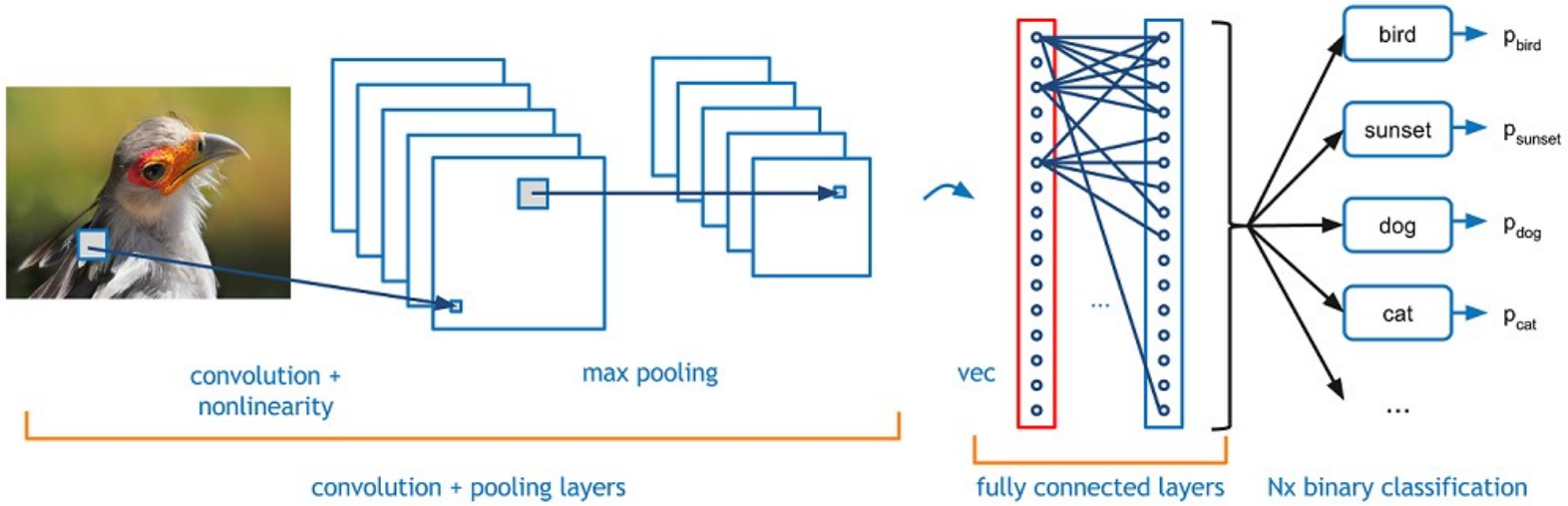


Рис. 2. Вид структуры СНС для задачи распознавания образов

Сверточный слой I

Пример

Цветное изображение размера в виде массива $32 \times 32 \times 3$ чисел из диапазона 0..255.

Сверточный слой

Использование фильтров, например, 5×5, заданной глубины (в нашем случае 3). Каждый фильтр – **матрица весов** (параметров), **ядро свертки**. Вычисляется поэлементные умножения ($5 \times 5 \times 3 = 75$), которые потом суммируются.
Пусть W – матрица фильтра размера $n \times n$, $X^{(l)}$ – матрица входных значений $n \times n$ для компоненты $l = R, G, B$, начиная с верхнего угла. Результат применения фильтра – скаляр s:

$$s = \sum_{l=\{R,G,B\}} \sum_{i=1}^{n} \sum_{j=1}^{n} w_{ij} X_{ij}^{(l)}.$$

Может быть: $W = W^{(l)}$.

Сверточный слой II

Карта активации (карта объектов, activation map, feature map) –

результат перемещения фильтра 5×5 по всему входному изображению.

$$32 \times 32 \times 3 \to 28 \times 28 \times 1.$$

Матрицу весов передвигают обычно не на полный шаг (размер матрицы), а на небольшое расстояние: на один или два нейрона (пикселя) вместо пяти, чтобы не «перешагнуть» искомый признак.
Замечание: сравните с уровнем активации нейрона *net*.

Сверточный слой III

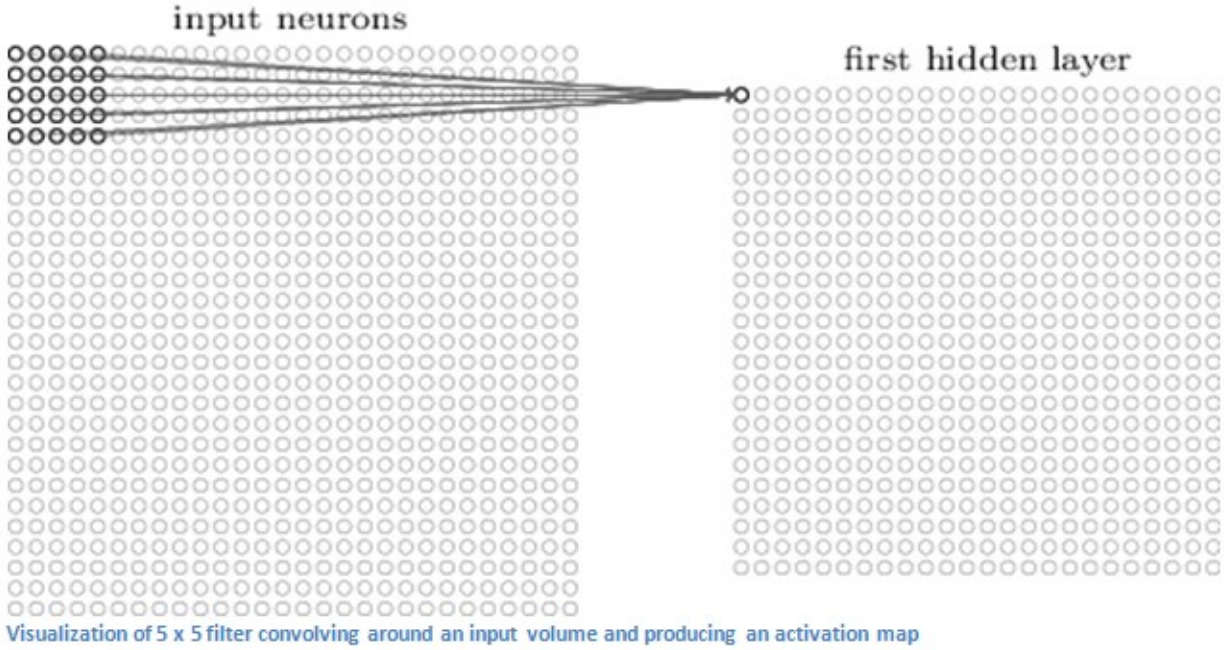
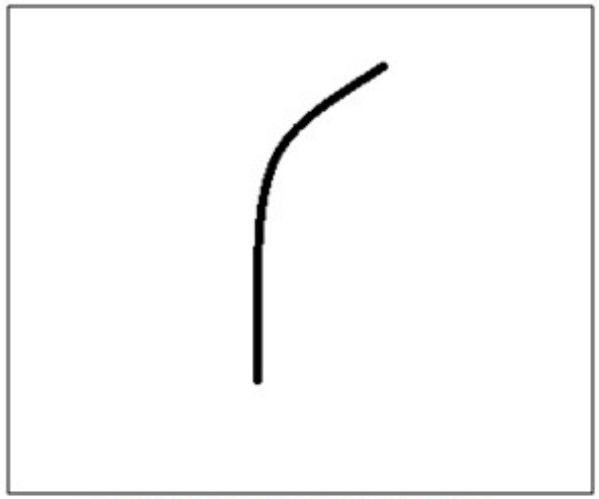


Рис. 3. Применение фильтра для получения карты активации

Сверточный слой IV

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter



Visualization of a curve detector filter

Рис. 4. Пример фильтра (детектора) кривой

Сверточный слой V

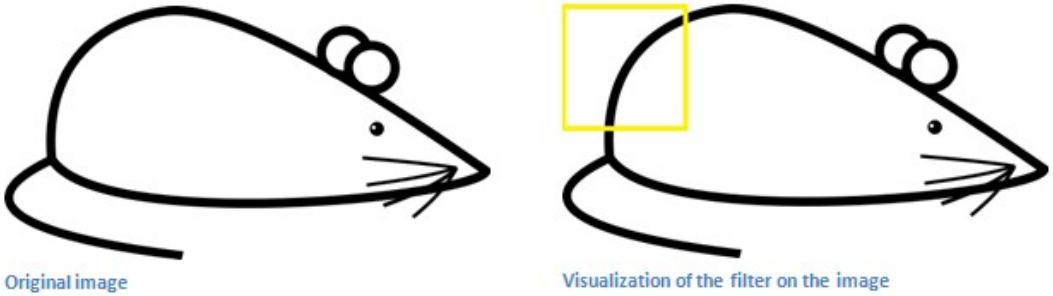


Рис. 5. Фильтр кривой для изображения мышки

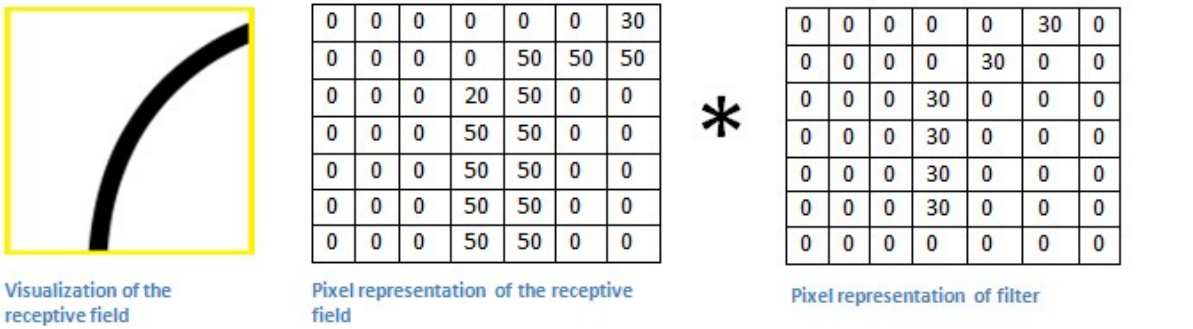


Рис. 6. Фильтр кривой для участка 1

Сверточный слой VI

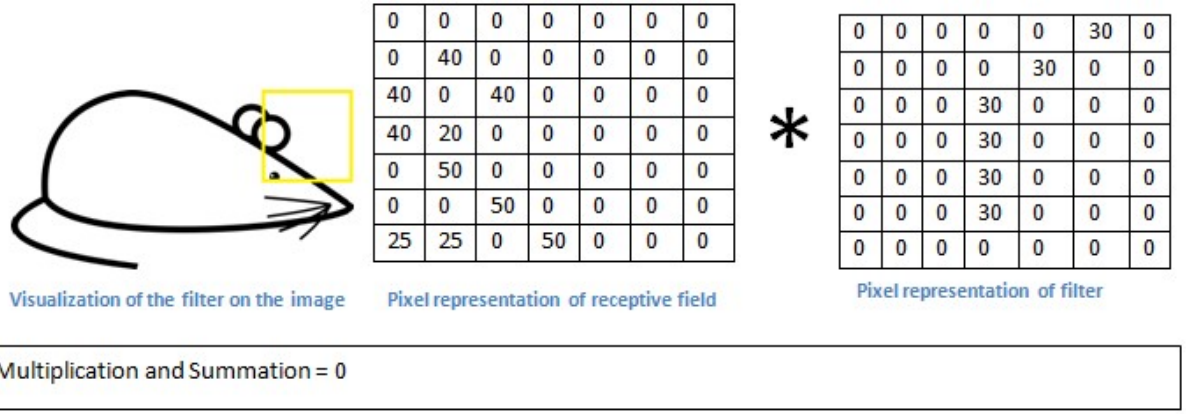


Рис. 7. Фильтр кривой для участка 2

Результаты применения фильтров:

- участок 1: $s = 6600$;
- участок 2: $s = 0$.

Сверточный слой VII

Разные фильтры

могут применяться к одному входу. Если **два** фильтра, то:

$$32 \times 32 \times 3 \to 28 \times 28 \times 2.$$

Чем больше фильтров, тем больше глубина карты свойств, тем больше информации об исходной картинке.
Возможно применение фильтров для определенного цвета (красного и т.д.).

Распространение сигналов:

выход первого – вход для 2-го слоя.
Применение 2-го сверточного слоя – свойства более высокого уровня: полукольца (комбинация прямой границы с изгибом) или квадраты (сочетание нескольких прямых ребер).

Сверточный слой VIII

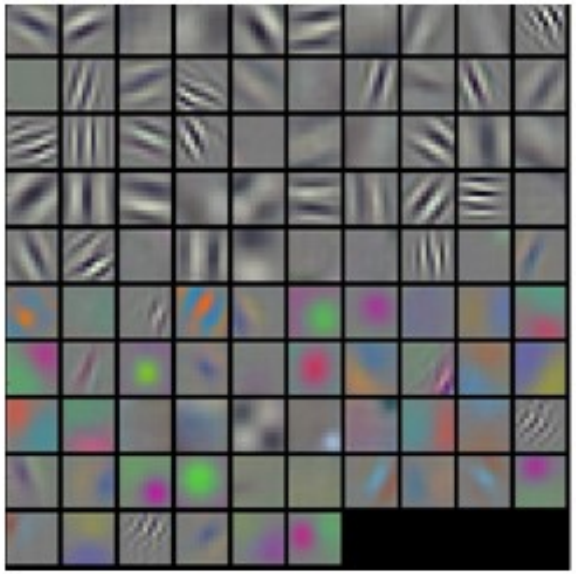


Рис. 8. Визуализация применения фильтров

Сверточный слой IX

Иерархия признаков –

СНС «самонастраивается», «организует иерархию» абстрактных признаков (последовательности карт признаков), фильтруя маловажные детали и выделяя существенное.
Это **метафорический** характер – признаки трудны для интерпретации, суть признаков не пытаются понять.

Параметры (гиперпараметры) свертки:

1. размер ядра (Kernel Size): обычно 3×3 или 5×5 – чем больше, тем меньше размер карты признаков;
2. шаг (Stride): на сколько пикселей сдвигается окно на каждом шаге – Stride=1 – плотная свертка, Stride=2 – размер карты признаков в 2 раза меньше;
3. дополнение нулями (Padding): для сохранения размера изображения – окружение изображения рамкой из нулей;
4. количество фильтров: каждый фильтр добавляет карту признаков.

Другие слои CHC I

Слой активации (nonlinearity на рис. 2)–

применение нелинейных функций активации (ФА) (сигмоидных и т.д.).
Замечание: обычно выделяются отдельными слоями.
Популярная ФА (стандарт «де-факто» для CHC) – **ReLU, Rectified Linear Unit** (с 2000-х гг.):

$$f(x) = \max(0, x).$$

Причины популярности:

- вычислительная простота – очень быстрая операция;
- борьба с затуханием градиента (см. далее): ReLU позволяет градиентам эффективно распространяться при обучении.

Другие слои СНС II

Слой объединения (подвыборки, пулинга) –

уменьшение размерности сформированных карт признаков.
Обычно: max или среднее по группе соседних нейронов.
Это нелинейное уплотнение карты признаков – группа пикселей (обычно 2×2) уплотняется до одного пикселя, проходя нелинейное преобразование.
Используются непересекающиеся прямоугольники или квадраты.

Задачи слоя:

- уменьшение размерности (основная*);
- повышение инвариантности – НС менее нечувствительна к малым смещениям, поворотам и искажениям объекта;
- контроль переобучения – уменьшение рисков запоминания данных из обучающего множества.

Другие слои CHC III

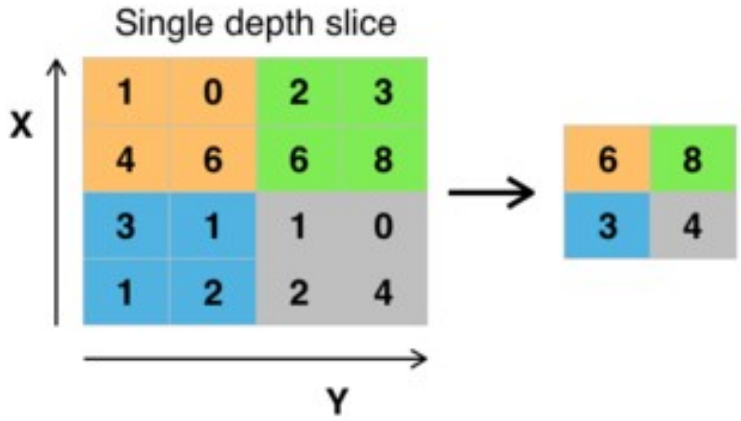


Рис. 9. Пример объединения на основе max

Чередование слоев –

слой объединения обычно вставляется после слоя свертки перед слоем следующей свертки.

Другие слои CHC IV

Фундаментальный блок построения CNN:

- сверточный слой;
- функция активации ReLU (иногда относится к сверточному слою);
- слой объединения.

Другие слои CHC V

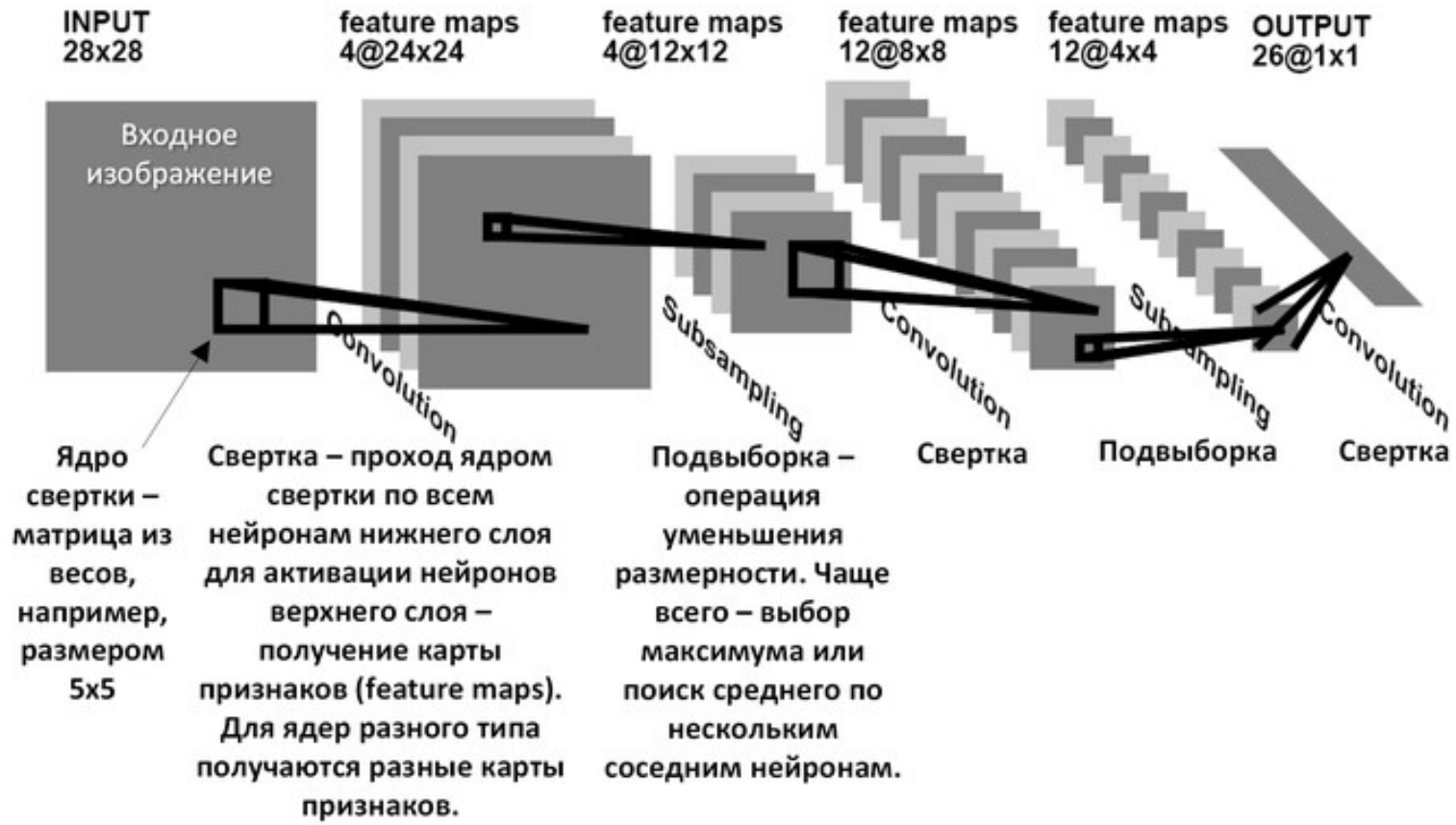


Рис. 10. Последовательность сверточных слоев

Другие слои СНС VI

Порядок применения слоев после сверточных:

1. Представление всех карт признаков в один длинный вектор (Flatten).
2. Один или несколько полносвязных слоев (вычисление выходов аналогично многослойным перцептронам).
3. На последнем слое – вычисление вероятностей определения класса $p(C_i), i = 1, \ldots, k$. Обычно – функция активации Softmax:

$$p(C_i) = \mathrm{Softmax}(y_i) = \frac{e^{y_i}}{\sum_{j=1}^{k} e^{y_j}}.$$

Другие слои СНС VII

Пример – распознавание цифр

$k = 10$ для 10 цифр $(0, 1, \ldots, 9)$.
Если выход $y = [0 \quad 0,1 \quad 0,1 \quad 0,75 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0,05]$, то:

- цифра 1: $p = 0,1$;
- цифра 2: $p = 0,1$;
- цифра 3: $p = 0,75$;
- цифра 9: $p = 0,05$.

1 Глубокие нейронные сети

2 Сверточные нейронные сети ЛеКуна

3 Особенности архитектур глубоких нейронных сетей

4 Применение СНС

5 Глубокое обучение и его особенности

6 Метод стохастического градиента и mini-batch метод

7 Заключение

8 Литература

Особенности архитектур глубоких нейронных сетей I

Архитектура	**Год**	**Описание**
LeNet	1998	Первая СНС. Медленное обучение. Низкая эффективность в сложных задачах.
AlexNet	2012	Увеличение глубины. Обучение на GPU. Использование ReLU. Применение Dropout.
VGGNet	2014	Увеличение глубины – несколько сверток 3×3. Огромное число параметров.
ResNet	2015-2016	Решение проблемы исчезающего градиента: остаточные блоки, сквозные связи. Незначительный рост точности. Сложная настройка гиперпараметров.
U-Net	2015	Энкодер-декодер для точной локализации. Работа с малым количеством данных.
MobileNet	2017-2019	Сокращение числа параметров при сохранении точности – разделения сверточного слоя по глубине.
EfficientNet	2019	Метод compound scaling – улучшение производительности. Долгий автоматического поиска архитектуры.
YOLO	2015-2025	Одновременное определение рамок, классов объектов.

Особенности архитектур глубоких нейронных сетей II

CNN LeNet-5:

предложена Лекуном в 1988 г. для распознавания цифр (датасет MNIST):

- вход (32×32);
- свертка -> ReLU -> пулинг;
- свертка -> ReLU -> пулинг;
- Flatten;
- полносвязный слой -> ReLU;
- выходной полносвязный слой (Softmax).

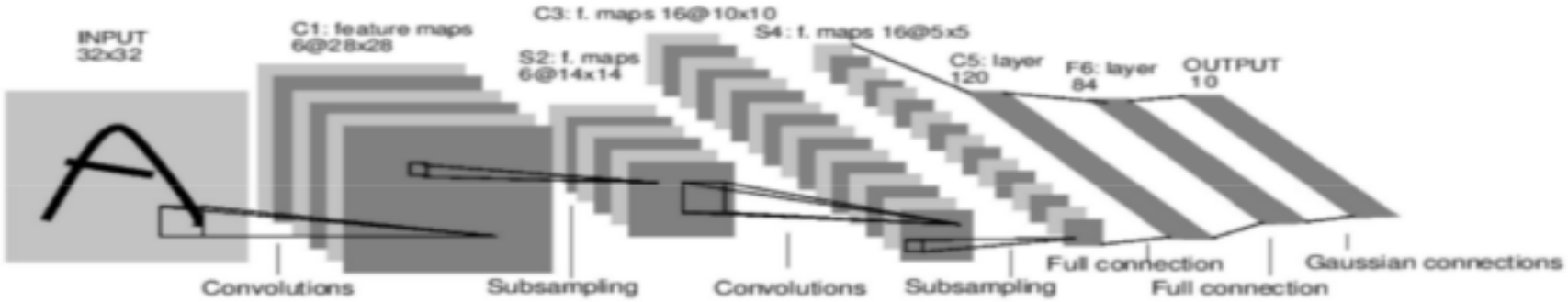


Рис. 11. LeNet

Особенности архитектур глубоких нейронных сетей III

CNN AlexNet:

предложена Крижевским (Krizhevsky) в 2014 г. для решения задач компьютерного зрения.

НС: 5 сверточных слоев и 2 полносвязных слоя.

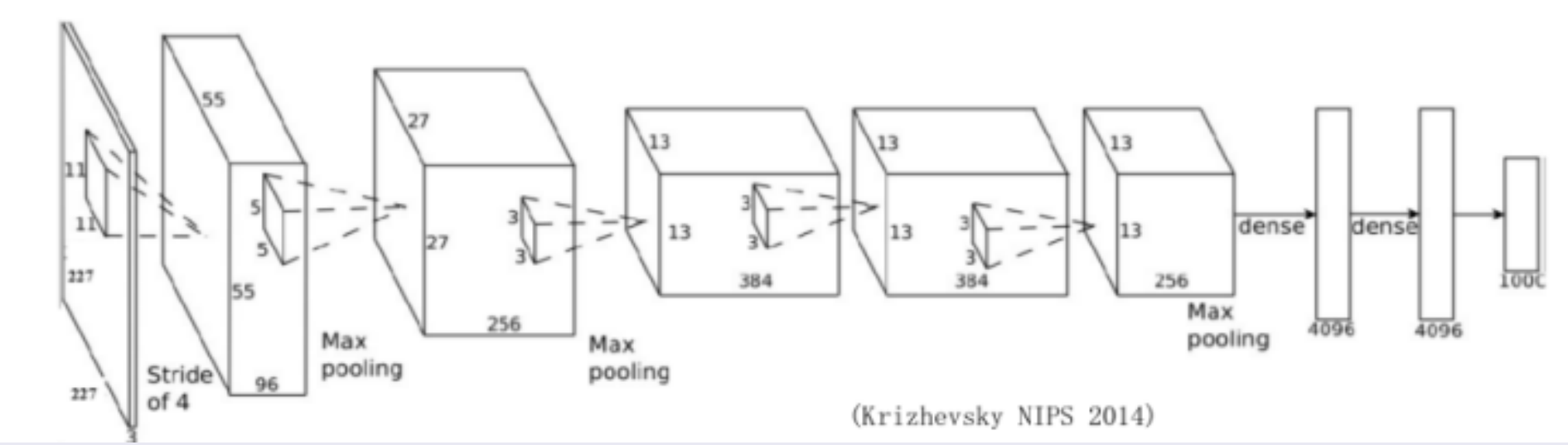


Рис. 12. AlexNet

Особенности архитектур глубоких нейронных сетей IV

Автокодировщики (autoencoders):

предложена Д. Баллардом (D. Ballard) в 1987 г. для снижения размерности данных.

НС: многослойный перцептрон, количество входов = количество выходов.

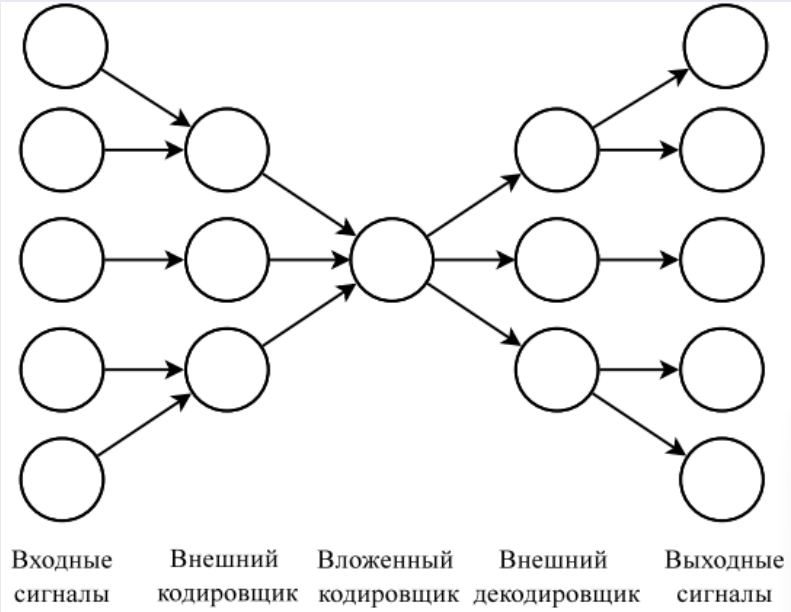


Рис. 13. Autoencoder

Особенности архитектур глубоких нейронных сетей V

НС Inception:

предложена Google в 2014 г. для решения задач компьютерного зрения.
НС: набор блоков, реализующих операции свертки и подвыборки разной размерности.

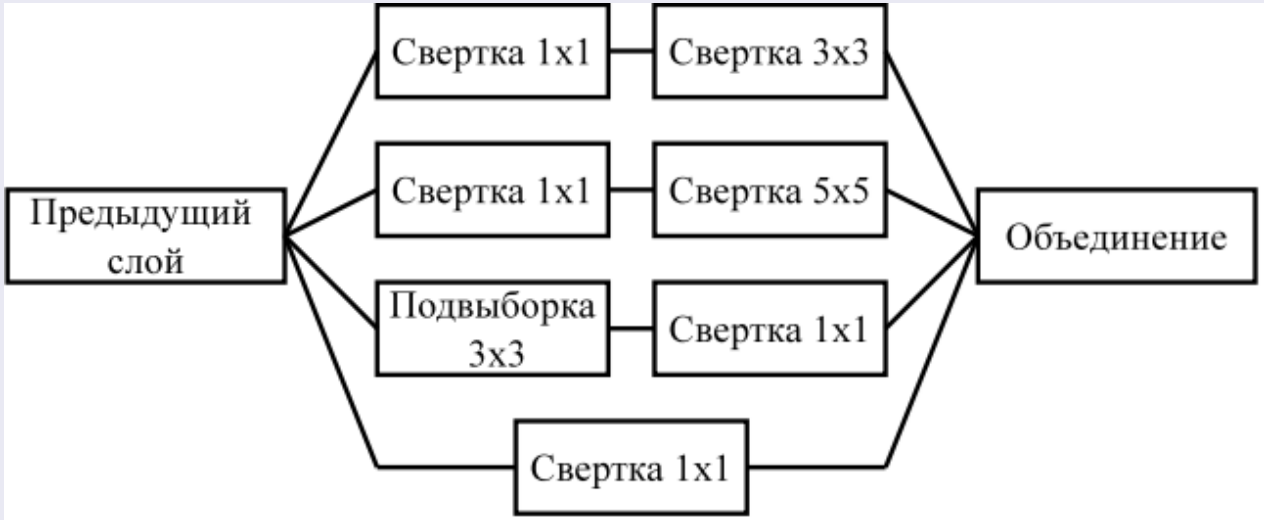


Рис. 14. Inception

Особенности архитектур глубоких нейронных сетей VI

НС с остаточным обучением (Residual Learning):

предложена Microsoft в 2015 г. для решения задач компьютерного зрения.
НС: набор блоков, реализующих 2-слойные НС простой структуры, тождественного слоя и обучения невязок (остатков).



Рис. 15. Residual Learning

$$y = x + F(x)$$

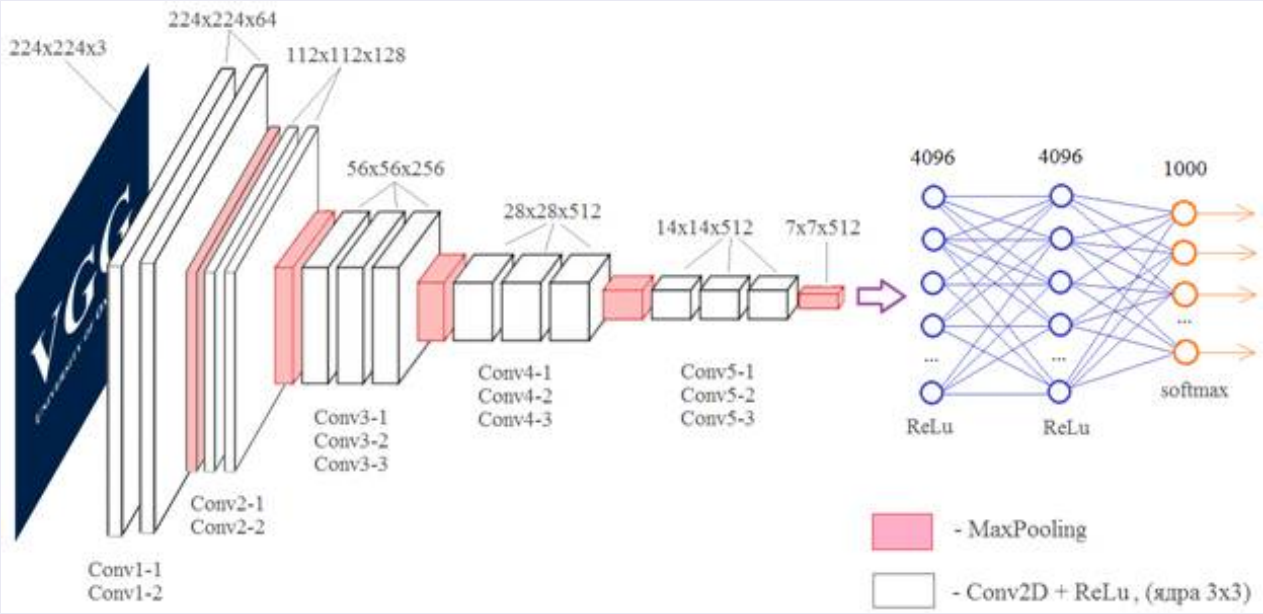
Градиент выхода слоя по входу (отсутствие проблемы затухания):

$$\frac{\partial y}{\partial x} = 1 + \frac{\partial F(x)}{\partial x}$$

Особенности архитектур глубоких нейронных сетей VII

Семейство НС VGG:

включает, в частности, VGG-11, VGG-13, VGG-16 и VGG-19 (номер – количество слоев в НС). Вход: $224 \times 244 \times 3$.
Победитель соревнования ImageNet 2013-ого года – VGG-16. Ядра свертки – 3×3. У ядра 5×5 число параметров $5 \cdot 5 + 1 = 26$. У 2-х ядер 3×3 число параметров $2 \cdot (3 \cdot 3 + 1) = 20$.



1 Глубокие нейронные сети

2 Сверточные нейронные сети ЛеКуна

3 Особенности архитектур глубоких нейронных сетей

4 Применение СНС

5 Глубокое обучение и его особенности

6 Метод стохастического градиента и mini-batch метод

7 Заключение

8 Литература

Применение СНС

Сферы применения СНС:

- классификация изображений, представленных на изображении;
- обнаружение объектов – где находится и какой это объект (YOLO, Faster R-CNN);
- семантическая сегментация – присвоение каждому пикселю класса («дорога», «человек»);
- обработка медицинских изображений: поиск опухолей на МРТ;
- генерация изображений: StyleGAN, DeepDream.

1 Глубокие нейронные сети

2 Сверточные нейронные сети ЛеКуна

3 Особенности архитектур глубоких нейронных сетей

4 Применение СНС

5 Глубокое обучение и его особенности

6 Метод стохастического градиента и mini-batch метод

7 Заключение

8 Литература

Глубокое обучение и его особенности I

Глубокое обучение (deep learning) –

совокупность методов машинного обучения для обучения глубоких (многослойных) НС.

Обучение –

метод обратного распространения ошибки, стохастические методы.

Глубокое обучение и его особенности II

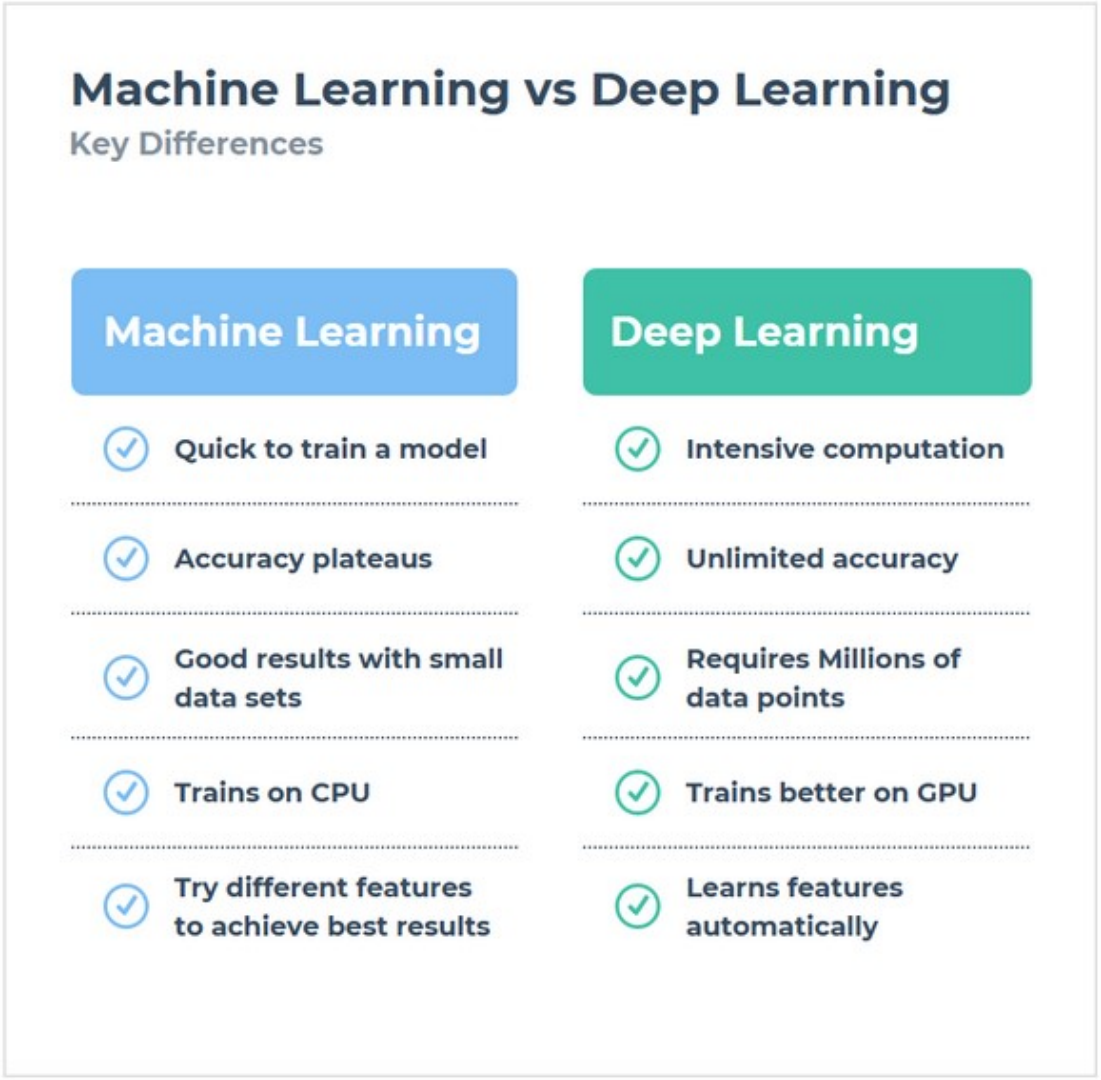


Рис. 16. Машинное VS глубокое обучение

Глубокое обучение и его особенности III

Проблема затухания градиента

Затухающий градиент (**Vanishing Gradient**): в процессе ОРО элементы градиента становится все меньше. Проблема: эти веса изменяются на слишком малые значения, обучение фактически не проводится (чем меньше номер слоя, тем меньше обучение). Причина: ограничение производной ФА (униполярная сигмоидная логистическая – [0; 0,25], биполярная сигмоидная логистическая – [0; 1]) и использование большого количества слоев. При насыщении этих ФА производная $\rightarrow 0$.

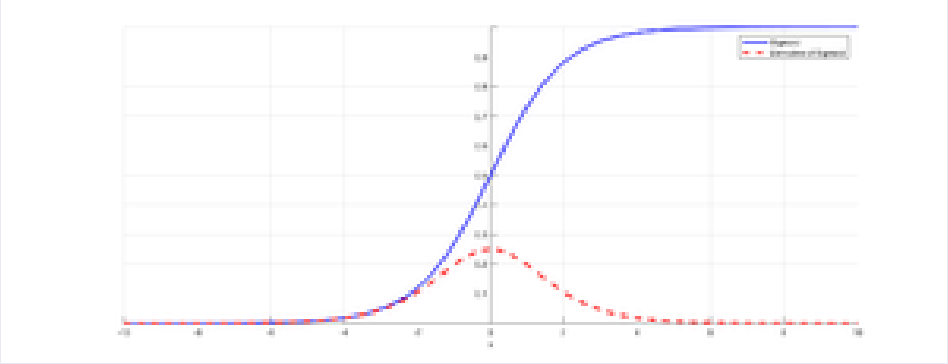


Рис. 17. Униполярная сигмоидная логистическая фнукция и ее производная

Глубокое обучение и его особенности IV

Пусть $w^{(k)}$ – вес некоторого нейрона на слое k, $x^{(k)}$ – некоторый вход этого нейрона.
Пусть $Q = \frac{1}{2}MSE$.

$$\frac{\partial Q}{\partial w^{(k)}} = \frac{\partial Q}{\partial y} \cdot \frac{\partial y}{\partial w^{(k)}} = (y - \tilde{y})\sigma' x^{(k)} \leqslant \frac{1}{4}(y - \tilde{y})x^{(k)}$$

$$\frac{\partial Q}{\partial w^{(k-1)}} = \frac{\partial Q}{\partial w^{(k)}} \cdot \frac{\partial w^{(k)}}{\partial w^{(k-1)}} \leqslant \left(\frac{1}{4}\right)^2 (y - \tilde{y})x^{(k)}x^{(k-1)}$$

Глубокое обучение и его особенности V

Решение 1 – использование других ФА:

Производная ReLU (недифференцируемость при $x = 0$ игнорируется):

$$\frac{d}{dx} ReLU(x) = \begin{cases} 1, & x > 0; \\ 0, & x \leqslant 0. \end{cases}$$

Устранение недифференцируемости – Softplus (используется редко из-за усложнения):

$$\sigma(x) = \ln(1 + e^x).$$

Решение 2 – использование остаточного обучения

Глубокое обучение и его особенности VI

Решение 3 – регуляризация весов:

слишком большие значения весов увеличивают значение функционала качества.
LASSO:

$$Q + \lambda \|w\|_1$$

Гребневая (Ridge) регрессия:

$$Q + \lambda \|w\|_2$$

Здесь $\lambda > 0$ – параметр регуляризации.

Глубокое обучение и его особенности VII

Решение 4 – инициализация весов Ксавье/Глорота (Xavier/Glorot):

инициализация малыми числами (около нуля).
Пусть N_{in} – число входов нейрона, N_{out} – число выходов нейрона.

$$w_i \cong N(0, \sigma),$$

$$\sigma = \sqrt{\frac{1}{N_{in} + N_{out}}}.$$

Глубокое обучение и его особенности VIII

Проблема взрывного роста градиента

Взрывающийся градиент (**Exploding Gradient**): в процессе ОРО при прохождении через слои в элементах градиента могут накапливаться большие значения. Проблема: возможно переполнение, другие веса не обновляются. Причина: в НС с обратными связями при BPTT, используются одни и те же веса.

Глубокое обучение и его особенности IX

Решение – градиентная обрезка (Gradient Clipping) (масштабирование)

- Обрезка по норме (Clipping-by-Norm):

$$\nabla Q_w(clipped) = \begin{cases} \nabla Q_w & \|\nabla Q_w\| \leqslant T; \\ \nabla Q_w \cdot \frac{T}{\|\nabla Q_w\|}, & \|\nabla Q_w\| > T. \end{cases}$$

- Обрезка по значению (Clipping-by-Value):

$$\nabla Q_{wi}(clipped) = \begin{cases} \nabla Q_{wi} & |\nabla Q_{wi}| \leqslant T; \\ T, & |\nabla Q_{wi}| > T. \end{cases}$$

Здесь T –пороговое значение.

1 Глубокие нейронные сети

2 Сверточные нейронные сети ЛеКуна

3 Особенности архитектур глубоких нейронных сетей

4 Применение СНС

5 Глубокое обучение и его особенности

6 Метод стохастического градиента и mini-batch метод

7 Заключение

8 Литература

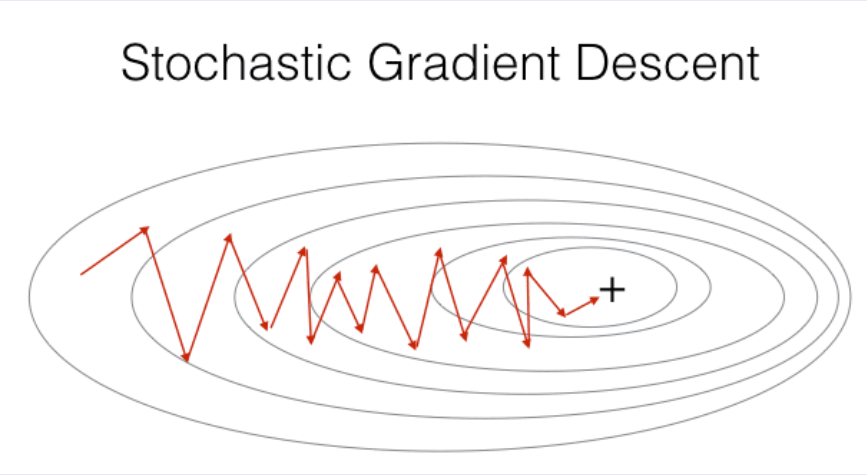
Метод стохастического градиента и mini-batch метод I

Цель – повышение эффективности обучения на обучающем множестве **огромного объема**.

Стохастический градиентный спуск (Stochastic Gradient Descent, SGD):

корректировка весов после использования одного **случайно выбранного** примера за один так обучения. Это позволяет:

- повысить скорость прохождения этапов обучения;
- избежать локальных экстремумов.



Метод стохастического градиента и mini-batch метод II

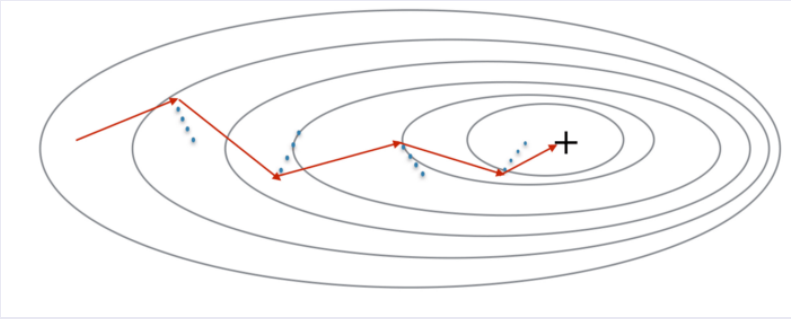
SGD с импульсом (SGD with momentum):

на каждой итерации используется скользящее среднее градиента. Цель – использование инерции в обучении. Используется приближенное скользящее среднее градиентов на предыдущих этапах:

$$g_0 = \nabla Q_{w0}; \quad g_t = \alpha g_{t-1} + \eta \nabla Q_{wt};$$

$$w_{t+1} = w_t - g(t).$$

Здесь g_t – накопленное среднее градиентов на шаге t, $\alpha \in [0, 1]$ – степень сохранения истории значений (обычно ≈ 0.9).



Метод стохастического градиента и mini-batch метод III

Пакетный градиентный спуск (Batch Gradient Descent, BatchGD):

расчет градиента функционала качества как суммы градиентов по всем примерам (сумма всех мгновенных функционалов качества). Это затратно по памяти.

Mini-batch градиентный спуск (Mini-Batch Gradient Descent):

комбинация SGD и BatchGD – случайное подмножество заданного размера из примеров обучающего множества.

1 Глубокие нейронные сети

2 Сверточные нейронные сети ЛеКуна

3 Особенности архитектур глубоких нейронных сетей

4 Применение СНС

5 Глубокое обучение и его особенности

6 Метод стохастического градиента и mini-batch метод

7 Заключение

8 Литература

Заключение (основные моменты лекции)

1. СНС – архитектуры НС для распознавания образов.
2. Идея СНС – в биологии, в характере работы нейронов.
3. Архитектуры СНС – большое количество чередующихся слоев.
4. Основной слой СНС – сверточный.
5. Идея применения фильтров и слоев объединения – снижение размерности.
6. СНС – тип глубоких НС.
7. Глубокие НС – НС с количеством скрытых слоев $\geqslant 2$.
8. Глубокое обучение – разновидность методов МО для глубоких НС.

1 Глубокие нейронные сети

2 Сверточные нейронные сети ЛеКуна

3 Особенности архитектур глубоких нейронных сетей

4 Применение СНС

5 Глубокое обучение и его особенности

6 Метод стохастического градиента и mini-batch метод

7 Заключение

8 Литература

ЛИТЕРАТУРА

1. Джейд Картер. Сверточные нейросети.– М.: Изд-во Литрес, 2024.– 157 с.
2. Desai S., Ramaswamy H.G. Ablation-cam: Visual explanations for deep convolutional network via gradient-free localization. In 2020 IEEE Winter Conference on Applications of Computer Vision (WACV), IEEE, 2020.– P. 972-980.
3. Wang H., et al. Score-cam: Score-weighted visual explanations for convolutional neural networks. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, 2020.– P. 24-25.

1 Глубокие нейронные сети

2 Сверточные нейронные сети ЛеКуна

3 Особенности архитектур глубоких нейронных сетей

4 Применение СНС

5 Глубокое обучение и его особенности

6 Метод стохастического градиента и mini-batch метод

7 Заключение

8 Литература

СПАСИБО ЗА ВНИМАНИЕ!

Построение и применение нейронных сетей и генетических алгоритмов
Лекция 7
Нейронные сети с обратными связями

САРАЕВ ПАВЕЛ ВИКТОРОВИЧ

РТУ МИРЭА
Кафедра прикладной математики

Москва – 2026

ПЛАН ЛЕКЦИИ

1. Понятие нейронных сетей с обратными связями
2. Архитектура сетей Хопфилда
3. Архитектура сетей Хэмминга
4. Временная обработка с использованием многослойных перцептронов
5. Алгоритм обратного распространения во времени
6. НС Элмана и Джордана
7. LSTM- и GRU-сети
8. Заключение
9. Литература

1 Понятие нейронных сетей с обратными связями

2 Архитектура сетей Хопфилда

3 Архитектура сетей Хэмминга

4 Временная обработка с использованием многослойных перцептронов

5 Алгоритм обратного распространения во времени

6 НС Элмана и Джордана

7 LSTM- и GRU-сети

8 Заключение

9 Литература

Понятие нейронных сетей с обратными связями I

Принципиально отличные архитектуры НС (1982 г.) от многослойных перцептронов (*). Эти НС передают информацию выходов нейронов обратно на нейроны НС.

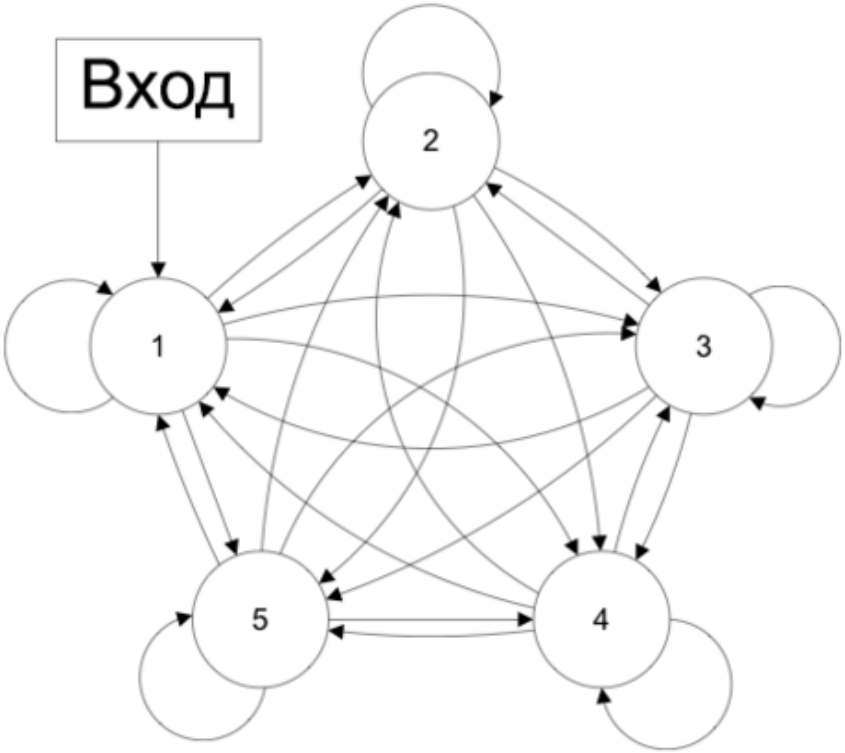


Рис. 1. Пример НС с обратными связями

Понятие нейронных сетей с обратными связями II

Синонимы:

- НС с обратными связями;
- динамические НС;
- рекуррентные НС;
- Recurrent Neural Networks (RNN).

Назначение –

обработка временных зависимостей и последовательных данных (прогнозирование временных рядов и языковое моделирование).

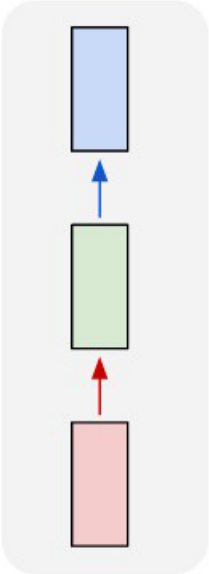
Нобелевская премия (2024 г.) –

Джон Хопфилд и Джеффри Хинтон по физике за фундаментальный вклад в машинное обучение, в частности, за НС Хопфилда.

Понятие нейронных сетей с обратными связями III

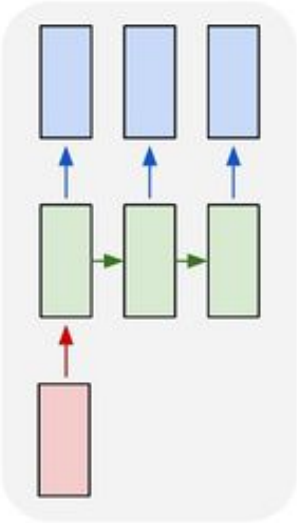
Виды НС с обратными связями:

- Один-к-одному: обычная НС прямого распространения



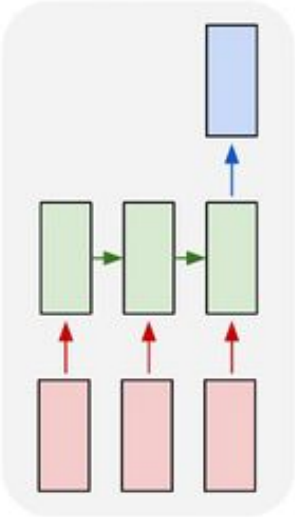
Понятие нейронных сетей с обратными связями IV

- Один-ко-многим: генерация аудио (вход – жанр, выход – музыка)



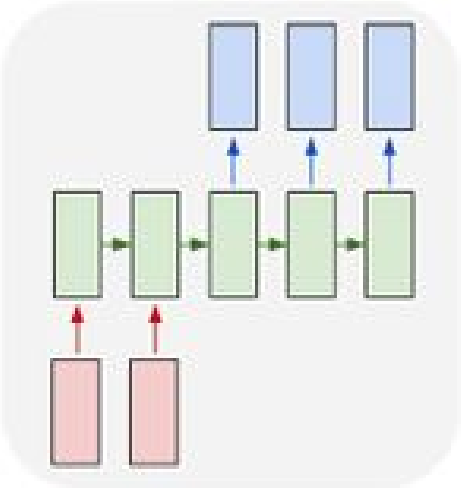
Понятие нейронных сетей с обратными связями V

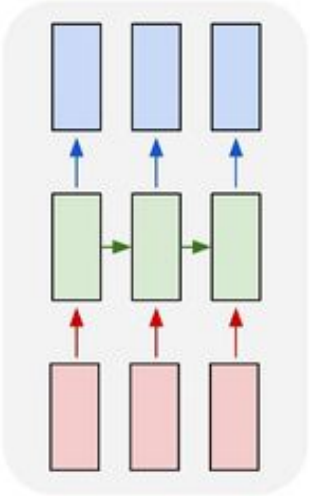
- Многие-к-одному: оценка тональности рецензии (вход – слова рецензии, выход – позитивная или негативная оценка тональности)



Понятие нейронных сетей с обратными связями VI

- Многие-ко-многим: перевод текста на другой язык (вариант 1); классификация каждого слова в зависимости от контекста (вариант 2)





1 Понятие нейронных сетей с обратными связями

2 Архитектура сетей Хопфилда

3 Архитектура сетей Хэмминга

4 Временная обработка с использованием многослойных перцептронов

5 Алгоритм обратного распространения во времени

6 НС Элмана и Джордана

7 LSTM- и GRU-сети

8 Заключение

9 Литература

Архитектура сетей Хопфилда I

НС Хопфилда

Суть – сигналы нейронов могут подаваться на входы нейронов, располагающихся в предыдущих слоях.

Сети Хопфилда – **модель автоассоциативной памяти**. НС содержит **1 скрытый слой**.

Область применения – задачи распознавания образов (по части признаков вспоминается объект).

Архитектура сетей Хопфилда II

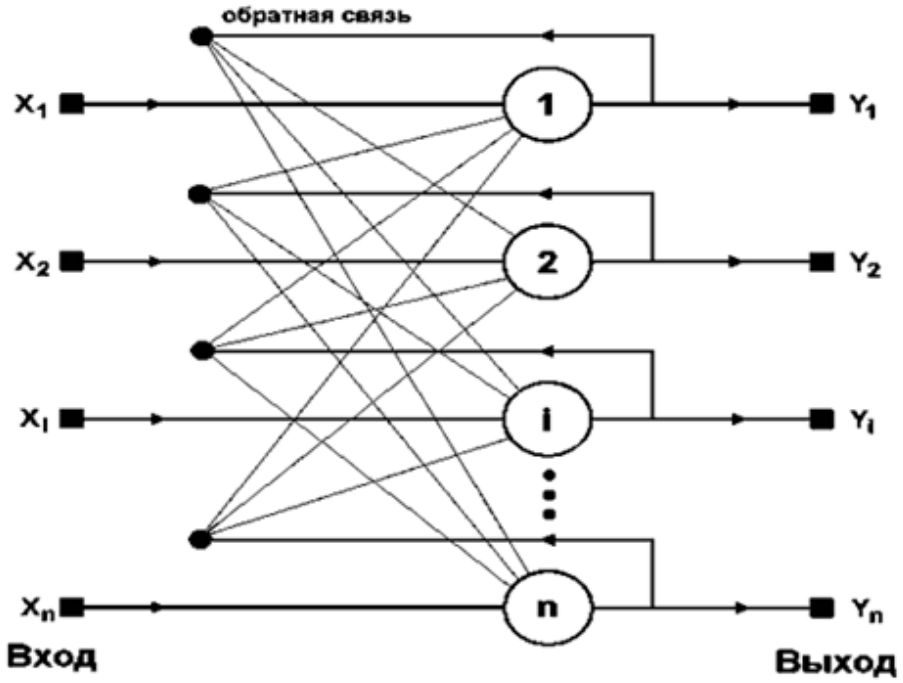


Рис. 2. НС Хопфилда (вид 1)

Архитектура сетей Хопфилда III

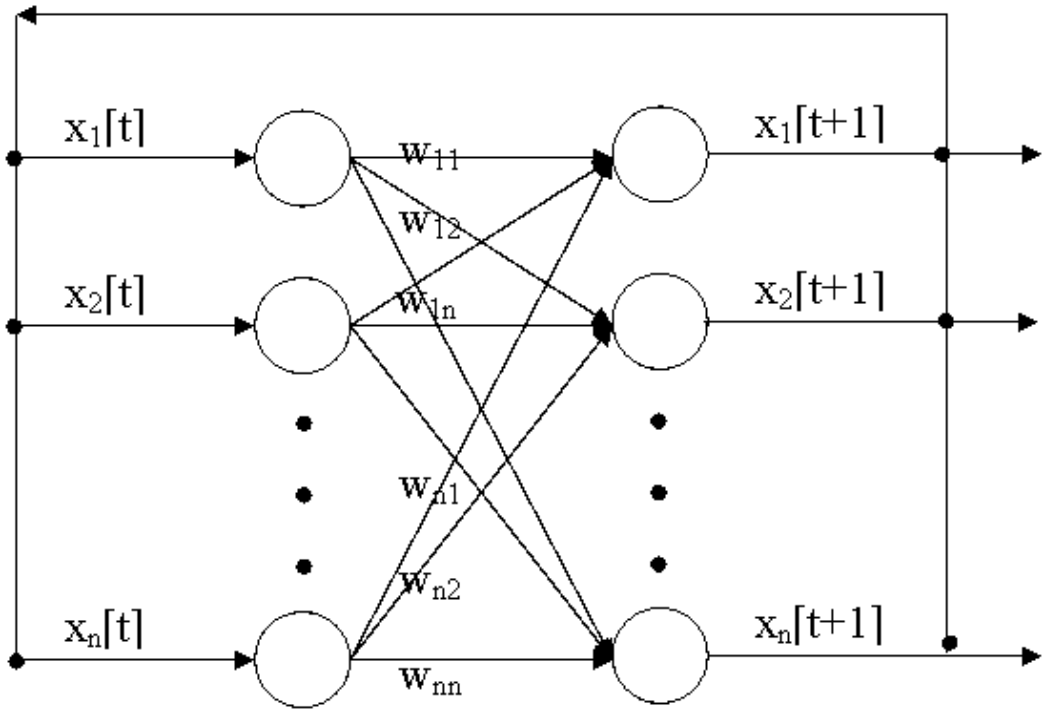


Рис. 3. НС Хопфилда (вид 2)

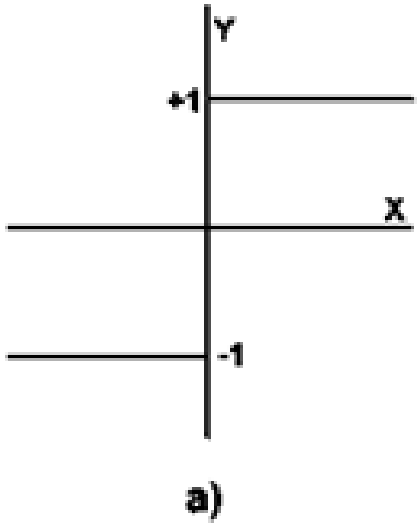
Архитектура сетей Хопфилда IV

Функционирование: $x_i \in \{-1, +1\}$, $+1$ – есть цвет; -1 – нет цвета

1. Задается начальное состояние x_0.
2. Потактовое выполнение:

$$x[t+1] = \sigma(Wx[t]),$$

σ – пороговая функция активации;



Архитектура сетей Хопфилда V

матрица весов:

$$W = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & w_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ w_{n1} & w_{n2} & \dots & w_{nn} \end{bmatrix}$$

3. Проверка критерия остановки (устойчивости состояния):

$$x[t+1] = x[t].$$

Архитектура сетей Хопфилда VI

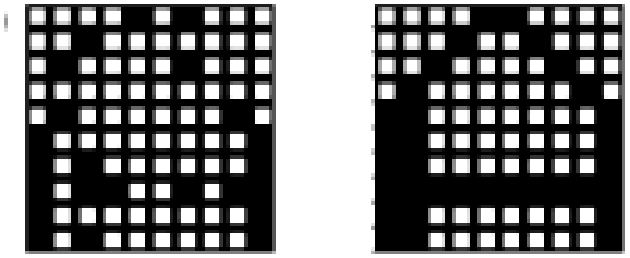


Рис. 4. Динамика НС Хопфилда-1

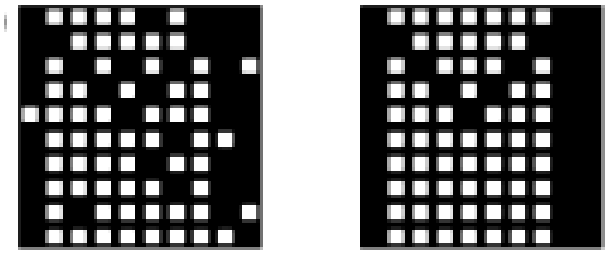


Рис. 5. Динамика НС Хопфилда-2

Архитектура сетей Хопфилда VII

Настройка матрицы весов (обучение*).

Задано множество из m векторов, подлежащих запоминанию в сети:

$$x^k, k = 1, \ldots, m.$$

Мера близости векторов – евклидово расстояние.

Задача поиска ассоциации – поиск минимума (энергетической) функции

$$Q = \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} w_{ij} x_i x_j.$$

Архитектура сетей Хопфилда VIII

Утверждение (Хопфилд)

Если в качестве весов сети выбрать

$$w_{ij} = \begin{cases} \frac{1}{n}\sum_{k=1}^{m} x_i^k x_j^k, & i \neq j \\ 0 & i = j \end{cases}$$

в векторной форме:

$$W = \frac{1}{n}\sum_{k=1}^{m} x^k (x^k)^T,$$

то исходные векторы – точки минимума (аттракторы).

Результат – матрица связей – **симметричная** матрица ($w_{ij} = w_{ji}$) с **нулевой диагональю** ($w_{ii} = 0$).
Такая сеть устойчива в исходных точках. Это необходимое, но не достаточное условие.
Емкость НС: $0,14n$, где n – количество образов.

1 Понятие нейронных сетей с обратными связями

2 Архитектура сетей Хопфилда

3 Архитектура сетей Хэмминга

4 Временная обработка с использованием многослойных перцептронов

5 Алгоритм обратного распространения во времени

6 НС Элмана и Джордана

7 LSTM- и GRU-сети

8 Заключение

9 Литература

Архитектура сетей Хэмминга I

НС Хэмминга (1987 г.) – развитие НС Хопфилда, предложена Р. Липпманом.

Отличие от сети Хопфилда:

выдает не сам вектор $x[t]$, а номер класса.

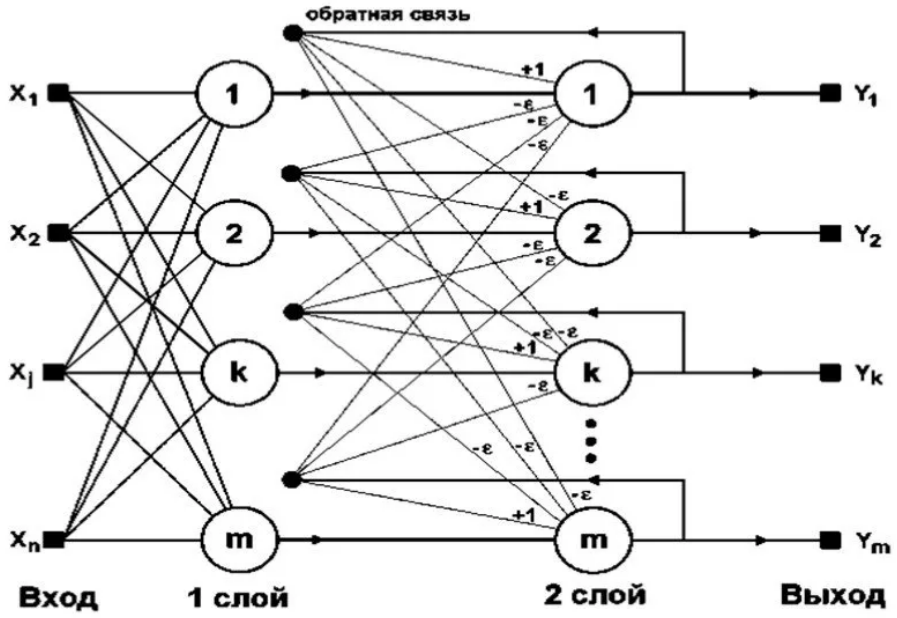


Рис. 6. НС Хэмминга

Архитектура сетей Хэмминга II

Сеть Хэмминга – 2-слойная НС с обратной связью. Количество нейронов – количество классов для классификации. Обратная связь – нейроны 2-го слоя: связь нейрона **с самим собой положительная**, связь **с другими –отрицательная**.

Обучение НС Хэмминга.

1. Формируется матрица эталонов (строки – бинарные векторы эталонов):

$$X \in R^{m \times n}$$

2. Матрица весов 1-го слоя:

$$W^{(1)} = \frac{X}{2},$$

функция активации:

$$\sigma(net) = \begin{cases} 0, & net \leqslant 0; \\ net, & 0 < net \leqslant T; \\ T, & net > T, \end{cases}$$

где $T = \frac{n}{2}$.

Архитектура сетей Хэмминга III

3 Матрица весов 2-го слоя:

$$W^{(2)} = \begin{bmatrix} 1 & -\varepsilon & \dots & -\varepsilon \\ -\varepsilon & 1 & \dots & -\varepsilon \\ \vdots & \vdots & \vdots & \vdots \\ -\varepsilon & -\varepsilon & \dots & 1 \end{bmatrix} \in R^{m \times m},$$

где $\varepsilon \in (0, \frac{1}{m}]$.

Архитектура сетей Хэмминга IV

Функционирование

1. Рассчитываются выходы нейронов 1-го слоя $x^{(1)}$.
2. Рассчитываются выходы нейронов 2-го слоя $x^{(2)}[0]$, рекурсивно пересчитываются как
$$x^{(2)}[t+1] = W^{(2)}x^{(2)}[t].$$
3. Критерий останова: $\|x^{(2)}[t+1] - x^{(2)}[t]\| < E_{max}$, где $E_{max} \approx 0,1$ – малая величина.
4. Если один элемент в итоговом векторе положительный, а остальные отрицательные, то это определяет класс. Если несколько элементов вектора положительны и все они $< E_{max}$, то НС не может отнести входной вектор ни к одному из классов (положительные выходы – кандидаты классов).

1 Понятие нейронных сетей с обратными связями

2 Архитектура сетей Хопфилда

3 Архитектура сетей Хэмминга

4 Временная обработка с использованием многослойных перцептронов

5 Алгоритм обратного распространения во времени

6 НС Элмана и Джордана

7 LSTM- и GRU-сети

8 Заключение

9 Литература

Временная обработка с использованием многослойных перцептронов I

Задание начального значения $x[0]$ позволяет генерировать последовательность сигналов на основе динамики модели

$$x[1], x[2], \ldots$$

Фрагмент НС A принимает вход x_t и возвращает h_t.

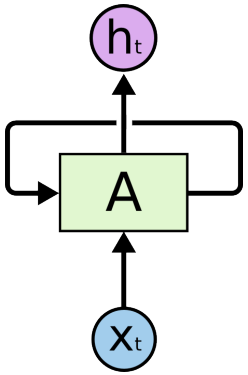


Рис. 7. Элемент НС с обратными связями

Временная обработка с использованием многослойных перцептронов II

Представление фрагмента НС с обратными связями в виде многослойного перцептрона.

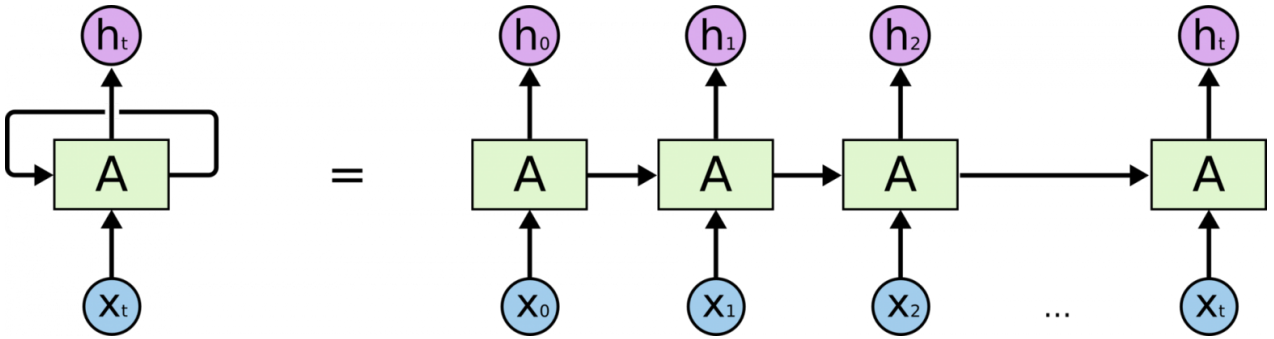


Рис. 8. Представление НС с обратными связями в виде многослойного перцептрона

1 Понятие нейронных сетей с обратными связями

2 Архитектура сетей Хопфилда

3 Архитектура сетей Хэмминга

4 Временная обработка с использованием многослойных перцептронов

5 Алгоритм обратного распространения во времени

6 НС Элмана и Джордана

7 LSTM- и GRU-сети

8 Заключение

9 Литература

Алгоритм обратного распространения во времени I

Одни и те же веса на всех временных этапах в НС, т.о. градиент на выходе зависит не только от расчетов текущего шага, но и от предыдущих временных шагов.

Алгоритм обратного распространения ошибки во времени – Backpropagation Through Time, BPTT.

Методика применения BPTT:

1. Фиксируется глубина динамики – количество тактов работы НС T.
2. Вычисляются градиенты функционала качества обучения по вектору весов для предыдущего такта, двух предыдущих и т.д. до достижения глубины динамики.
3. Производится суммирования полученных градиентов:

$$\frac{\partial Q}{\partial w} = \sum_{t=1}^{T} \frac{\partial Q}{\partial h_t} \cdot \left(\prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}} \right) \cdot \frac{\partial h_t}{\partial w}$$

Алгоритм обратного распространения во времени II

Проблемы обучения НС с ОС:

- затухающий градиент: если

$$\left|\frac{\partial h_k}{\partial h_{k-1}}\right| < 1,$$

  то $\prod$ стремится к 0 при больших T;
- взрывающийся градиент: если

$$\left|\frac{\partial h_k}{\partial h_{k-1}}\right| > 1,$$

  то градиенты экспоненциально растут.

1 Понятие нейронных сетей с обратными связями

2 Архитектура сетей Хопфилда

3 Архитектура сетей Хэмминга

4 Временная обработка с использованием многослойных перцептронов

5 Алгоритм обратного распространения во времени

6 НС Элмана и Джордана

7 LSTM- и GRU-сети

8 Заключение

9 Литература

НС Элмана и Джордана I

Простые сети с обратными связями (Simple Recurrent Networks, SRN):

- НС Элмана;
- НС Джордана.

НС Элмана (1990 г.) состоит из 2 слоев: y, z (и входного слоя x).
Дополнение: слой «контекстных блоков» u для слоя y: сохраняются значения скрытого слоя в контекстных блоках (до того, как слой y меняет значение).

$$u = h_{t-1}$$

НС **сохраняет состояние**, что может использоваться в прогнозировании последовательностей.

НС Элмана и Джордана II

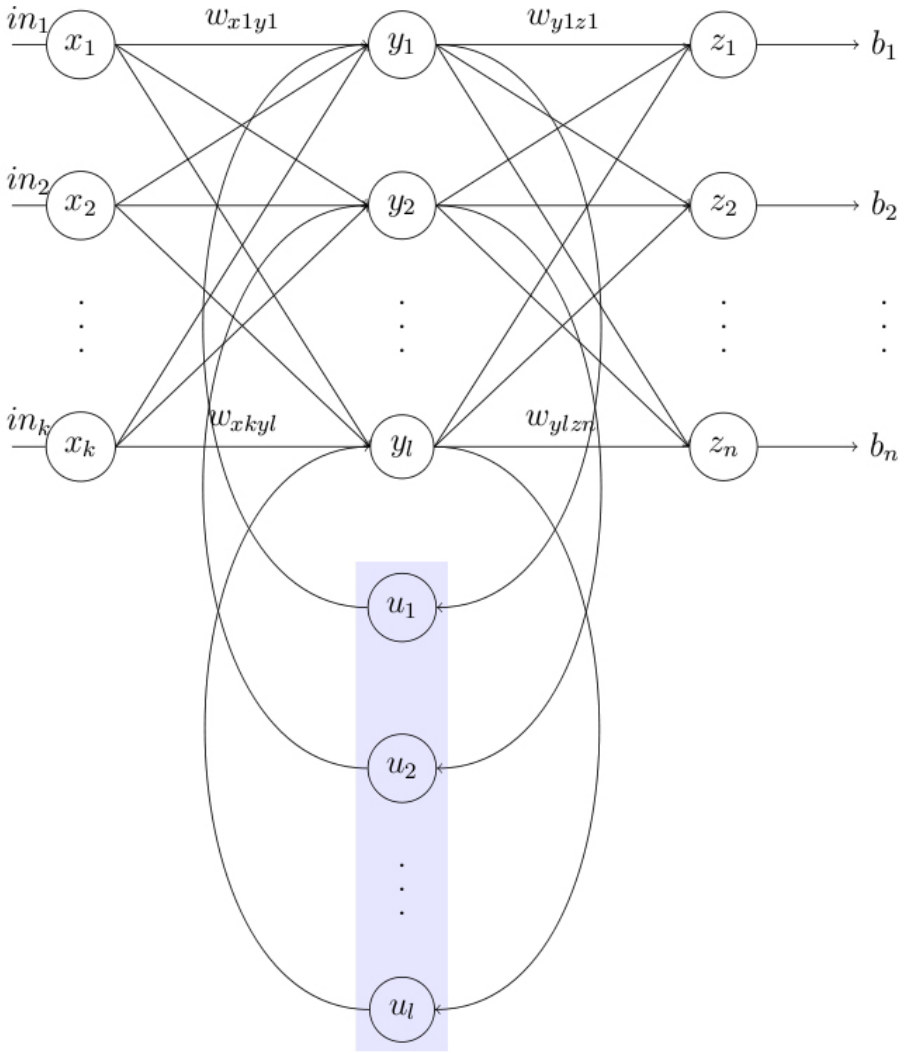


Рис. 9. НС Элмана

НС Элмана и Джордана III

Обозначение

- x_t – вектор входов.
- h_t – вектор выходов нейронов 1-го скрытого слоя.
- y_t – вектор выходов нейронов 2-го слоя (вектор выходов НС).
- W_h, W_y – матрицы весов скрытого и выходного слоев соответственно.
- b_h, b_y – векторы смещений скрытого и выходного слоев соответственно.
- U – матрица весов соединений с контекстным блоком.
- σ_h, σ_y – функции активации нейронов скрытого и выходного слоев соответственно.

Функционирование НС Элмана:

$$h_t = \sigma_h \left(W_h x_t + U_h h_{t-1} + b_h \right)$$

$$y_t = \sigma_y \left(W_y h_t + b_y \right)$$

НС Элмана и Джордана IV

НС Джордана (1986 г.) аналогична НС Элмана.

$$u = y_{t-1}$$

Функционирование НС Джордана:

$$h_t = \sigma_h \left(W_h x_t + U_h y_{t-1} + b_h \right)$$

$$y_t = \sigma_y \left(W_y h_t + b_y \right)$$

НС Элмана и Джордана V

Подумать: связь с конечными автоматами (КА) (см. лит.)

КА – устройство, перерабатывающее дискретную информацию, работа которого задается шестеркой объектов:

$$< X, Y, V, \varphi, \psi, x[0] >,$$

- X, V, Y – конечные внутренний, входной и выходной алфавиты (множества) соответственно;
- $x[0]$ – начальное состояние;
- φ – функция переходов:

$$x[t] = \varphi(x[t-1], v[t]);$$

- ψ – функция выходов:
  - КА Мили: $y[t] = \psi(x[t-1], v[t])$;
  - КА Мура: $y[t] = \psi(x[t])$ (ср. с НС Элмана).

КА Мили: КА Мура:

1 Понятие нейронных сетей с обратными связями

2 Архитектура сетей Хопфилда

3 Архитектура сетей Хэмминга

4 Временная обработка с использованием многослойных перцептронов

5 Алгоритм обратного распространения во времени

6 НС Элмана и Джордана

7 LSTM- и GRU-сети

8 Заключение

9 Литература

LSTM-сети I

LSTM-сети – Long Short Term Memory - сети.

История LSTM:

- 1997 г.: предложены З. Хохрайтером и Ю. Шмидхубером;
- 2009 г.: победа на соревнованиях по распознаванию рукописного текста;
- 2013 г.: основной компонент сети, которая достигла рекордного порога ошибки в $17,7\%$ в задаче распознавания фонем на классическом корпусе естественной речи;
- 2016 г.: применение в решении задач распознавания речи в ведущих компаниях: Google, Apple, Microsoft.

LSTM-сети II

Особенности LSTM:

- вид НС с ОС, способные обучаться долгосрочным зависимостям;
- приспособлена для решения задач классификации, обработки и прогнозирования временных рядов: события разделены временными лагами с неопределенной продолжительностью.

LSTM состоит из набора связанных **блоков**.

LSTM-сети III

Основное – разделение памяти на 2 компоненты:

- состояние ячейки c_t – «долговременная память» (L), слабо меняется по всей последовательности;
- скрытое состояние h_t – «рабочая (кратковременная) память» (S).

Процесс обработки данных на одном шаге t:

- входы:
  - x_t – новый элемент последовательности;
  - h_{t-1} – предыдущее скрытое состояние;
  - c_{t-1} – предыдущее состояние ячейки;
- выходы:
  - h_t – новое скрытое состояние;
  - c_t – новое состояние ячейки.

LSTM-сети IV

LSTM-блок:

Содержит 3-4 **вентиля** (gate): используются для контроля потоков информации на входах и на выходах.
Вентиль – логическая функции, результат– значения в диапазоне $[0; 1]$ (мера сохранения/пропуска информации). Сигналы умножаются на это значение.
Интерпретация – частичный допуск/запрет передачи потока информации внутрь/наружу памяти.
Цель – контроль потоков информации, позволяя избежать проблемы затухания градиента.

- Входной вентиль контролирует меру вхождения нового значения в память.
- Вентиль забывания контролирует меру сохранения значения в памяти.
- Выходной вентиль контролирует меру того, в какой степени значение, находящееся в памяти, используется при расчёте выходной функции активации для блока.

LSTM-сети V

Подумайте – 2 основные функции человеческой памяти:

- запоминать;
- забывать.

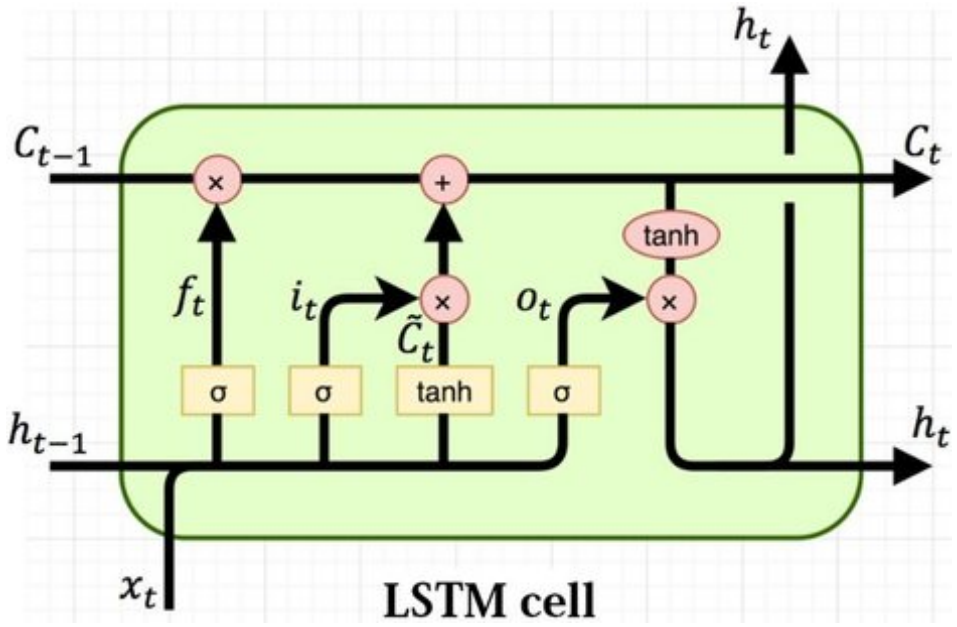


Рис. 10. LSTM-блок (ячейка)

LSTM-сети VI

Обозначение

- x_t – вектор входов;
- h_t – вектор выходов (скрытых состояний);
- c_t – вектор состояний (ячейки);
- W, b – матрицы весов и вектор смещений соответственно;
- i_t – вектор входного вентиля (вес получения новой информации);
- f_t – вектор вентиля забывания (вес запоминания старой информации);
- o_t – вектор выходного вентиля (кандидат на выход);
- σ_g, σ_c, σ_h – функции активации (сигмоидная логистическая, гиперболический тангенс, гиперболический тангенс или тождественная соответственно).

LSTM-сети VII

Функционирование LSTM-блока:

- вентили:

$$f_t = \sigma_g \left(W_{fx} x_t + W_{fh} h_{t-1} + b_f \right)$$

$$i_t = \sigma_g \left(W_{ix} x_t + W_{ih} h_{t-1} + b_i \right)$$

$$o_t = \sigma_g \left(W_{ox} x_t + W_{oh} h_{t-1} + b_o \right)$$

- кандидат на изменение состояния ячейки (Candidate Cell):

$$\tilde{c}_t = \sigma_c \left(W_{cx} x_t + W_{ch} h_{t-1} + b_c \right)$$

- состояния ячейки и скрытое (выхода):

$$c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t$$

$$h_t = o_t \circ \sigma_h \left(c_t \right)$$

Здесь $\circ$ $(*)$ – произведение Адамара.

LSTM-сети VIII

Полезно считать, что

есть составной вход нейронов:

$$[x_t \quad h_{t-1}]^T,$$

есть составные матрицы весов

$$[W_x \quad W_h].$$

Тогда вычисление f_t, i_t, o_t, $\tilde{c}_t$ – результат работы обычных нейронов.

Обучение блока LSTM: итеративный градиентный спуск (обычно BPTT).
Позволяет настроить блок для долговременности/краткосрочности забывания значений.
LSTM иногда обучают с помощью эволюционных алгоритмов для весов в скрытых слоях.

Отличие LSTM от традиционных НС с ОС:

ячейка состояния в LSTM – носитель **долгосрочной** памяти – может запоминать информацию на долгие периоды времени.

LSTM-сети IX

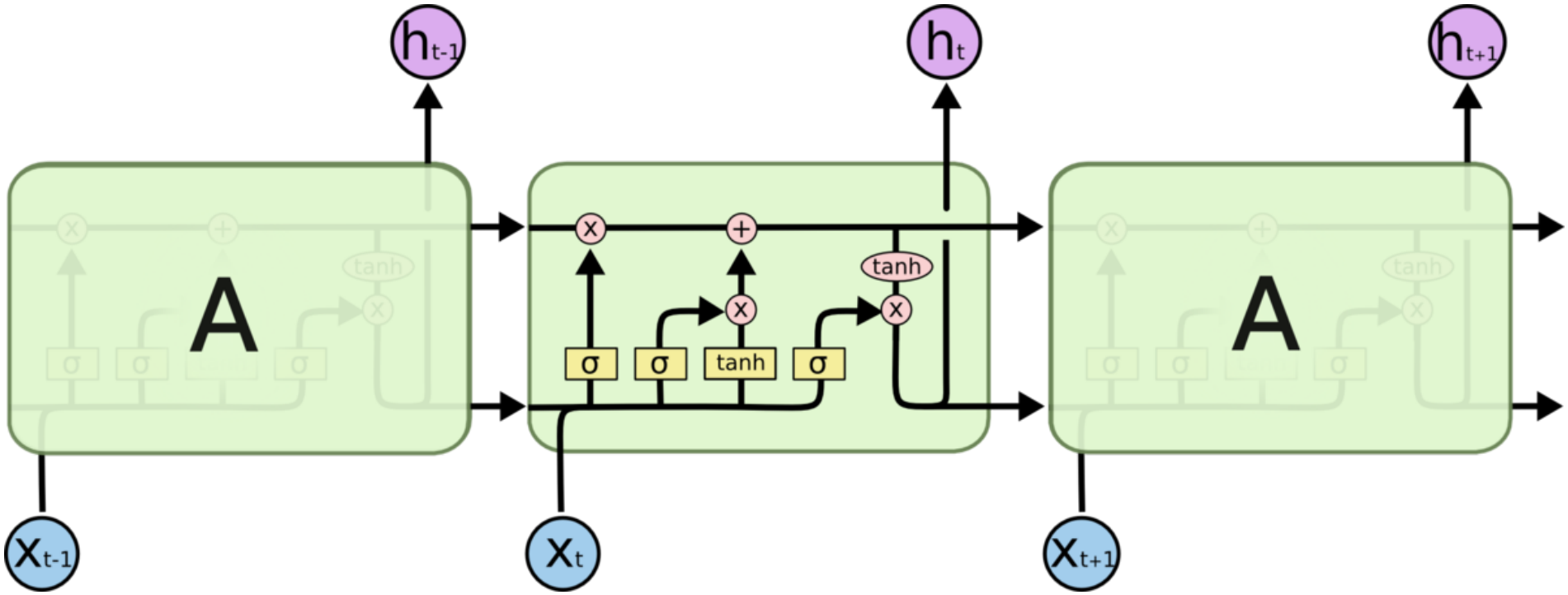


Рис. 11. Структура LSTM-сети

LSTM-сети X

Иногда:

входной вентиль и вентиль забывания – единый вентиль: старое значение забывается, когда появляется новое значение, достойное запоминания.

GRU (2014 г.) – Gated Recurrent Unit:

вид НС с ОС, развитие LSTM.
Цель – упрощение и ускорение LSTM. Вентили:

- вентиль сброса (reset gate) – какая часть прошлого состояния важна для текущего ввода;
- вентиль обновления (update gate) – какая часть предыдущего скрытого состояния сохраняется, используется для формирования нового скрытого состояния.

Нет состояний ячейки c_t, только скрытых состояний h_t – выполняет роль **и рабочей, и долговременной памяти** (LS).

LSTM-сети XI

Процесс обработки данных на одном шаге t:

- входы:
  - x_t – новый элемент последовательности;
  - h_{t-1} – предыдущее состояние;
- выход:
  - h_t – новое состояние.

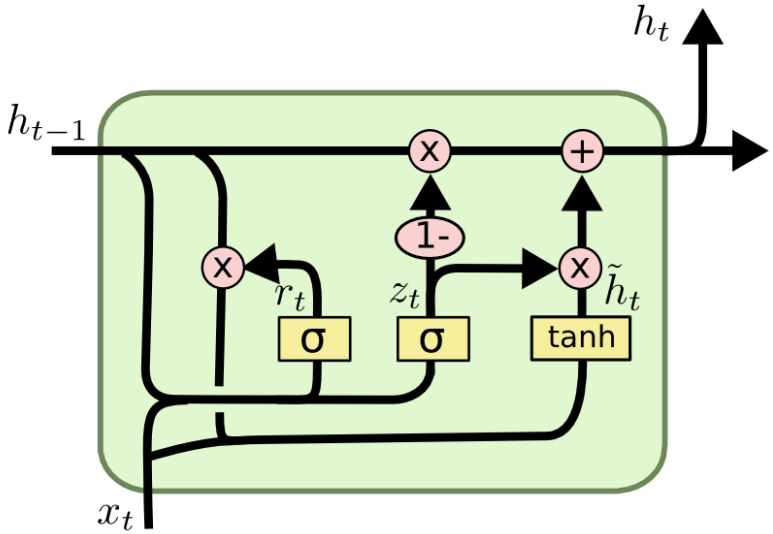


Рис. 12. GRU-блок (ячейка)

LSTM-сети XII

Функционирование GRU-блока:

- вентили:

$$r_t = \sigma_g \left(W_{rx} x_t + W_{rh} h_{t-1} + b_r \right)$$

$$z_t = \sigma_g \left(W_{zx} x_t + W_{zh} h_{t-1} + b_z \right)$$

- кандидат на изменение скрытого состояния:

$$\tilde{h}_t = \sigma_h \left(W_{hx} x_t + W_{hr} (r_t \circ h_{t-1}) + b_h \right)$$

- скрытое состояние (выход):

$$h_t = (1 - z_t) \circ h_{t-1} + z_t \circ \tilde{h}_t$$

LSTM-сети XIII

Подумать – связь с экспоненциальным сглаживанием:

простое сглаживание:

$$\tilde{x}_t = (1 - \alpha)\tilde{x}_{t-1} + \alpha x_t,$$

где $\tilde{x}_t$ – сглаженный уровень, $0 < \alpha < 1$ – параметр сглаживания.
Отличие от расчета скрытого состояния в GRU:

- α – константа;
- z_t – динамический сигнал.

GRU можно рассматривать как **более общий** случай экспоненциального сглаживания.

LSTM-сети XIV

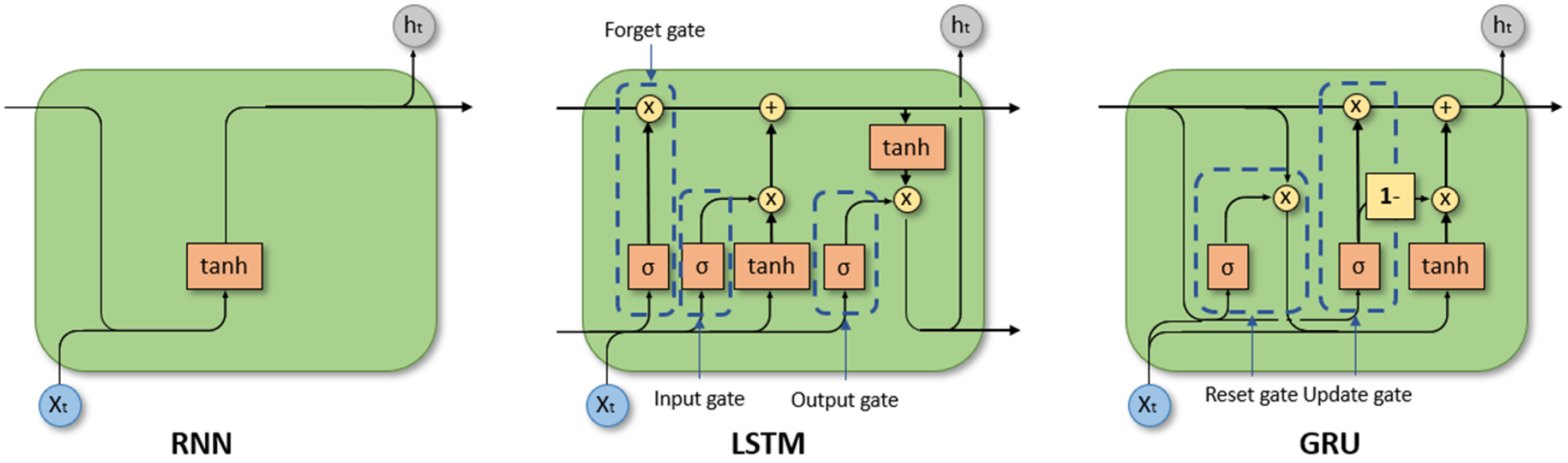


Рис. 13. Сравнение традиционных НС с ОС, LSTM, GRU

LSTM-сети XV

Показатель	НС с ОС (RNN)	LSTM	GRU
Архитектура	Простая структура с петлями	Ячейки памяти с входами, выходами и элементами забывания	Объединяет входы и выходы забывания в выход обновления; меньше параметров
Обработка длинных последовательностей	Проблемы с сохранением долгосрочных зависимостей из-за затухающих градиентов	Отлично подходит для выявления долгосрочных зависимостей	Лучше, чем RNN, но немного менее эффективно, чем LSTM, при работе с долгосрочными зависимостями
Время обучения	Быстро, но менее точно при работе со сложными данными	Работает медленнее из-за множества вентилей и операций с памятью	Быстрее, чем LSTM, но медленнее, чем RNN
Использование памяти	Низкие требования к памяти	Более высокий расход памяти из-за сложной архитектуры	Меньший расход памяти по сравнению с LSTM, но больший, чем у RNN
Количество параметров	Мало параметров	Больше параметров, чем в НС с ОС, из-за вентилей и ячеек памяти	Меньшее количество параметров, чем у LSTM, благодаря упрощенной структуре управления
Простота обучения	Склонность к исчезающим градиентам затрудняет обучение на длинных последовательностях	Благодаря эффективному управлению градиентом обучение на длинных последовательностях становится проще	Проще, чем LSTM, и легче обучается, чем RNN
Варианты использования	Подходит для моделирования простых последовательностей, например цен на акции	Идеально подходит для прогнозирования временных рядов, генерации текста и задач, требующих учёта долгосрочных зависимостей	Аналогичные приложения, как и LSTM, но предпочтительные в тех случаях, когда важна вычислительная эффективность
Производительность на длинных последовательностях	Низкая производительность при работе с длинными последовательностями	Хорошая производительность при длинных последовательностях	От умеренной до хорошей производительности при работе с длинными последовательностями

1 Понятие нейронных сетей с обратными связями

2 Архитектура сетей Хопфилда

3 Архитектура сетей Хэмминга

4 Временная обработка с использованием многослойных перцептронов

5 Алгоритм обратного распространения во времени

6 НС Элмана и Джордана

7 LSTM- и GRU-сети

8 Заключение

9 Литература

Заключение (основные моменты лекции)

1. НС с обратными связями предназначены для моделирования динамики.
2. Исходной архитектурой НС с ОС является НС Хопфилда.
3. Основное направление развития НС и ОС – долговременный учет информации.
4. Промежуточный этап развития НС с ОС – НС Элмана и Джордана.
5. Основной способ работы с НС с ОС – развертывание в пространстве в виде многослойного перцептрона.
6. Основной подход в обучении НС с ОС – алгоритм обратного распространения ошибки во времени (BPTT).
7. Современные архитектуры НС с ОС – LSTM- и GRU-сети.
8. GRU-сети – упрощенная эффективная версия LSTM.
9. LSTM/GRU – не новая парадигма, а архитектурная модификация внутри семейства НС с ОС, которая решает проблему обучения (затухающего градиента).

1 Понятие нейронных сетей с обратными связями

2 Архитектура сетей Хопфилда

3 Архитектура сетей Хэмминга

4 Временная обработка с использованием многослойных перцептронов

5 Алгоритм обратного распространения во времени

6 НС Элмана и Джордана

7 LSTM- и GRU-сети

8 Заключение

9 Литература

ЛИТЕРАТУРА

1. Чару Аггарвал. Нейронные сети и глубокое обучение. Учебный курс.– М.: Вильямс, 2020.– 752 с.
2. Schmidhuber Jurgen. Annotated History of Modern AI and Deep Learning, 2020.– https://arxiv.org/pdf/2212.11279
3. RNN vs LSTM vs GRU vs Transformers.– https://www.geeksforgeeks.org/deep-learning/rnn-vs-lstm-vs-gru-vs-transformers/ (last updated 23.07.2025
4. Understanding LSTM Networks.– https://colah.github.io/posts/2015-08-Under (last updated 27.08.2015)
5. Feedback Neural Networks: Structure, Training, and Applications.– https:/www.geeksforgeeks.org/feedback-neural-networks-structure-training-and-app (last updated 23.07.2025)
6. Joshua J. Michalenko, Ameesh Shah, Abhinav Verma, Richard G. Baraniuk, Swarat Chaudhuri, Ankit B. Patel. Representing Formal Languages: A Comparison Between Finite Automata and Recurrent Neural Networks.– https://openreview.net/forum?id=H1zeHnA9KX (last updated 05.05.2023)

1 Понятие нейронных сетей с обратными связями

2 Архитектура сетей Хопфилда

3 Архитектура сетей Хэмминга

4 Временная обработка с использованием многослойных перцептронов

5 Алгоритм обратного распространения во времени

6 НС Элмана и Джордана

7 LSTM- и GRU-сети

8 Заключение

9 Литература

СПАСИБО ЗА ВНИМАНИЕ!

Построение и применение нейронных сетей и генетических алгоритмов
Лекция 8
Современные архитектуры НС. НС для решения задач компьютерного зрения и обработки текста

САРАЕВ ПАВЕЛ ВИКТОРОВИЧ

РТУ МИРЭА
Кафедра прикладной математики

Москва – 2026

ПЛАН ЛЕКЦИИ

1. Архитектура Seq2Seq
2. Механизмы внимания
3. Механизм внимания «Scaled Dot-Product Attention»:
4. Архитектура трансформера
5. Архитектура GPT
6. Архитектура BERT
7. Генеративно-состязательные сети (GAN)
8. Диффузионные модели
9. Заключение
10. Итоги курса «ППНСГА»
11. Литература

1 Архитектура Seq2Seq
2 Механизмы внимания
3 Механизм внимания «Scaled Dot-Product Attention»:
4 Архитектура трансформера
5 Архитектура GPT
6 Архитектура BERT
7 Генеративно-состязательные сети (GAN)
8 Диффузионные модели
9 Заключение
10 Итоги курса «ППНСГА»
11 Литература

Архитектура Seq2Seq I

Цель создания Seq2Seq:

преобразование (трансформация) входной последовательности из языка A в новую последовательность из языка B.

Архитектура Seq2Seq – обобщенный механизм внимания

Две части – две НС с обратными связями:

- энкодер (кодер) – принимает предложение на языке A и сжимает его в вектор скрытого состояния;
- декодер – выдает слово на языке B, принимает последнее скрытое состояние энкодера и предыдущее предсказанное слово.

Энкодер обрабатывает входные данные и создает одно компактное представление из всех временных шагов ввода.
Выход энкодера – **контекст (контекстный вектор)**.
Декодер получает вектор контекста и генерирует выходную последовательность.

Архитектура Seq2Seq II

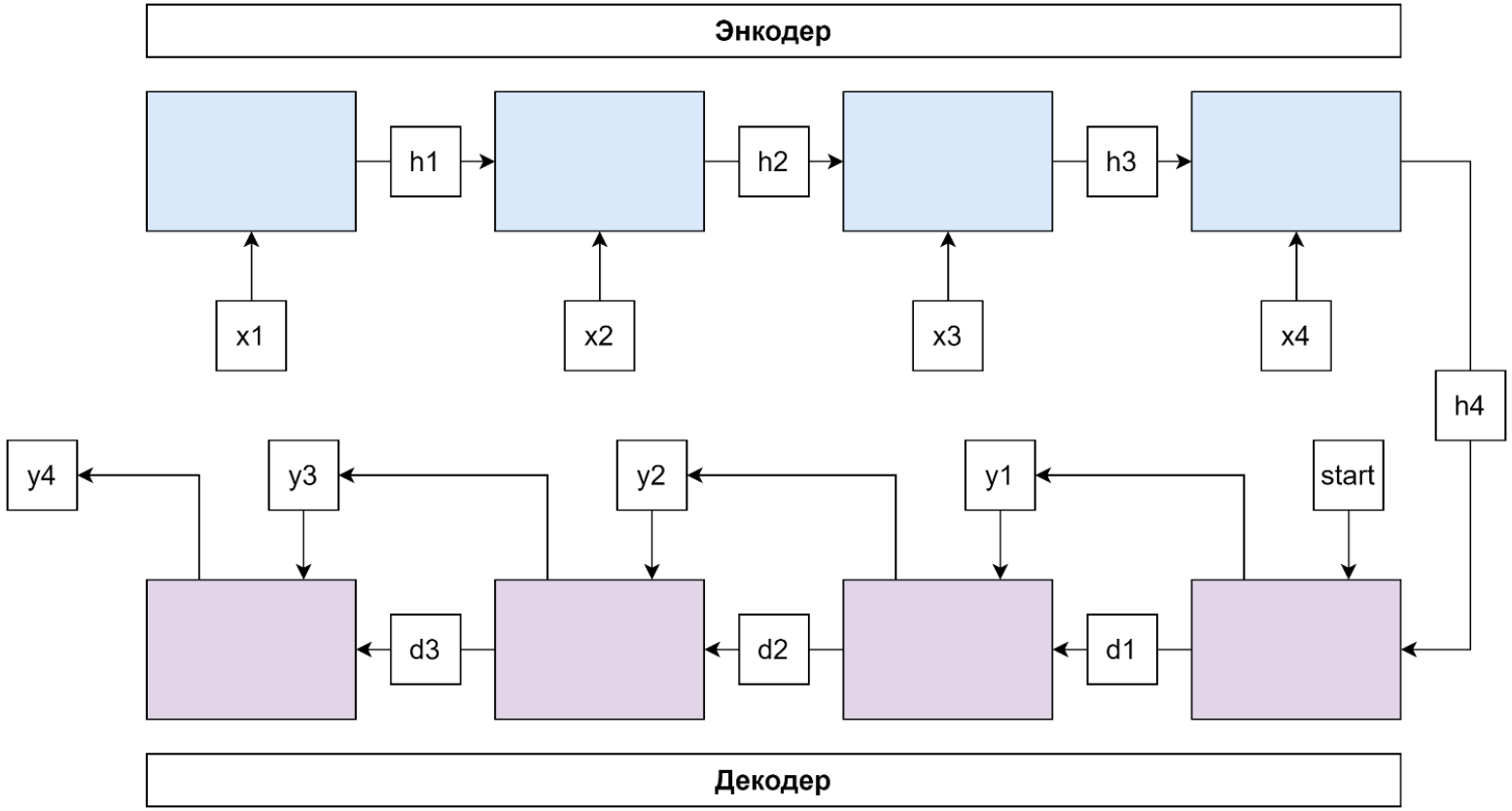


Рис. 1. Архитектура Seq2Seq

Архитектура Seq2Seq III

Обозначения:

- x_i – слова в предложении на языке A;
- h_i – скрытые состояния энкодера;
- d_i – скрытые состояния декодера;
- y_i – слова в предложении на языке B.

Алгоритм работы НС Seq2seq

1. Блоки декодера получают на вход специальный токен **start** (для первой итерации) или y_{i-1}.
2. Возвращают слова в предложении на языке B.
3. Передают скрытое состояние декодера на следующую итерацию.
4. Перевод считается завершенным при y_i, равном специальному токену **end**.

Архитектура Seq2Seq IV

Токены:

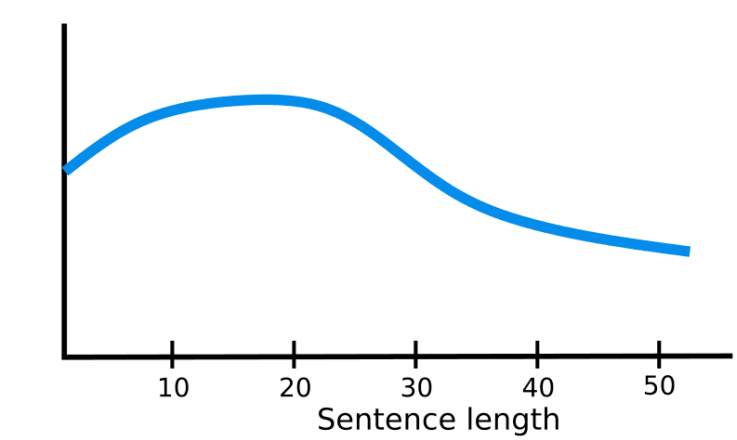
слова, текст, пиксель, батчи в изображении, изображение в видео.

Связь механизма внимания и НС с обратными связями:

Энкодер и декодер – глубокие НС на основе НС с обратными связями.

Проблема НС с обратными связями (LSTM и GRU):

только для коротких последовательностей (< 20 шагов).



Архитектура Seq2Seq V

Минусы подхода с НС с обратными связями:

- содержат всю информацию о последовательности в скрытом состоянии, которое обновляется. Если необходимо «вспомнить» что-то давнее, эту информацию необходимо хранить внутри скрытого состояния и не заменять чем-то новым. Следовательно, нужно очень большое скрытое состояние, либо мириться с потерей информации (**проблема забывания информации**);
- проблема затухающего градиента при обучении;
- обучение сложно распараллелить: чтобы получить скрытое состояние RNN-слоя для шага $i+1$, необходимо вычислить состояние для шага i. Таким образом, обработка батча длиной 1000 требует 1000 последовательных операций, что занимает много времени и неэффективно работает на GPU.

Основная проблема Seq2Seq:

все исходное предложение кодируется в один фиксированный (контекстный) вектор.

1 Архитектура Seq2Seq

2 Механизмы внимания

3 Механизм внимания «Scaled Dot-Product Attention»:

4 Архитектура трансформера

5 Архитектура GPT

6 Архитектура BERT

7 Генеративно-состязательные сети (GAN)

8 Диффузионные модели

9 Заключение

10 Итоги курса «ППНСГА»

11 Литература

Механизмы внимания I

Проблемы до внимания:

1. НС с обратными связями:
   - обрабатывают последовательность по очереди;
   - информация из начала последовательности может забываться/размываться (проблема долгосрочных зависимостей);
   - вычисление непараллелизуемо.
2. Сверточные НС для обработки последовательностей:
   - могут работать параллельно, но имеют ограниченный размер рецептивного поля (receptive field) – область, на которую реагирует нейрон в сверточном слое;
   - далекие элементы напрямую не взаимодействуют.

Механизм внимания (attention mechanism, attention model) –

техника для поиска взаимосвязей между различными частями входных и выходных данных, развивающая идеи из НС с обратными связями и СНС.

Механизмы внимания II

Основные виды механизмов внимания:

- обобщенный механизм внимания (general attention) – разновидность механизма внимания, задачей которой является выявление закономерности между входными и выходными данными;
- самовнимание (self-attention) – разновидность механизма внимания, задача которого – выявление закономерности только между входными данными.

Ключевая идея внимания –

фокус на разных элементах данных (не все элементы одинаково важны друг для друга). Для текста это:

- ключевые термины (например, «нейронные сети»);
- местоимения, которые ссылаются на понятия («они», «эти модели»).
- логические связки («следовательно», «поэтому»).

Это формализация процесса оценки важности слов в контексте всего предложения/ абзаца/ текста.

Механизмы внимания III

Особенность внимания –

позволяет напрямую связать любые 2 элемента последовательности, независимо от расстояния между ними, и оценить (взвесить) влияние одного на другой. Возможна параллельная реализация.

Формализация внимания –

для каждого элемента (например, слова) вычисляется **контекстное представление** – взвешенная сумма представлений всех (не только последнего) элементов последовательности, где веса определяются их релевантностью друг другу.

Замечание –

в классическом случае в НС с ОС используется только последнее скрытое состояние h_m, где m – длина последовательности входных данных. Использование механизма внимания позволяет использовать информацию **любого** скрытого состояния h_t.

Механизмы внимания IV

Слой механизма внимания –

НС, обычно 1-слойная. Вход:

- $h_t, t = 1, \ldots, m$;
- вектор d, зависящий от конкретной задачи (Seq2Seq: d_{i-1} – предыдущее скрытое состояние декодера).

Выход: вектор s-оценок (scores) размерности m – степень внимания на скрытое состояние h_i.
Далее – нормализация значений $s \to e = softmax(s)$:

$$e_i = \frac{e^{s_i}}{\sum_{j=1}^{m} e^{s_j}};$$

Результат работы слоя внимания – вектор контекста c:

$$c = \sum_{i=1}^{m} e_i h_i.$$

Механизмы внимания V

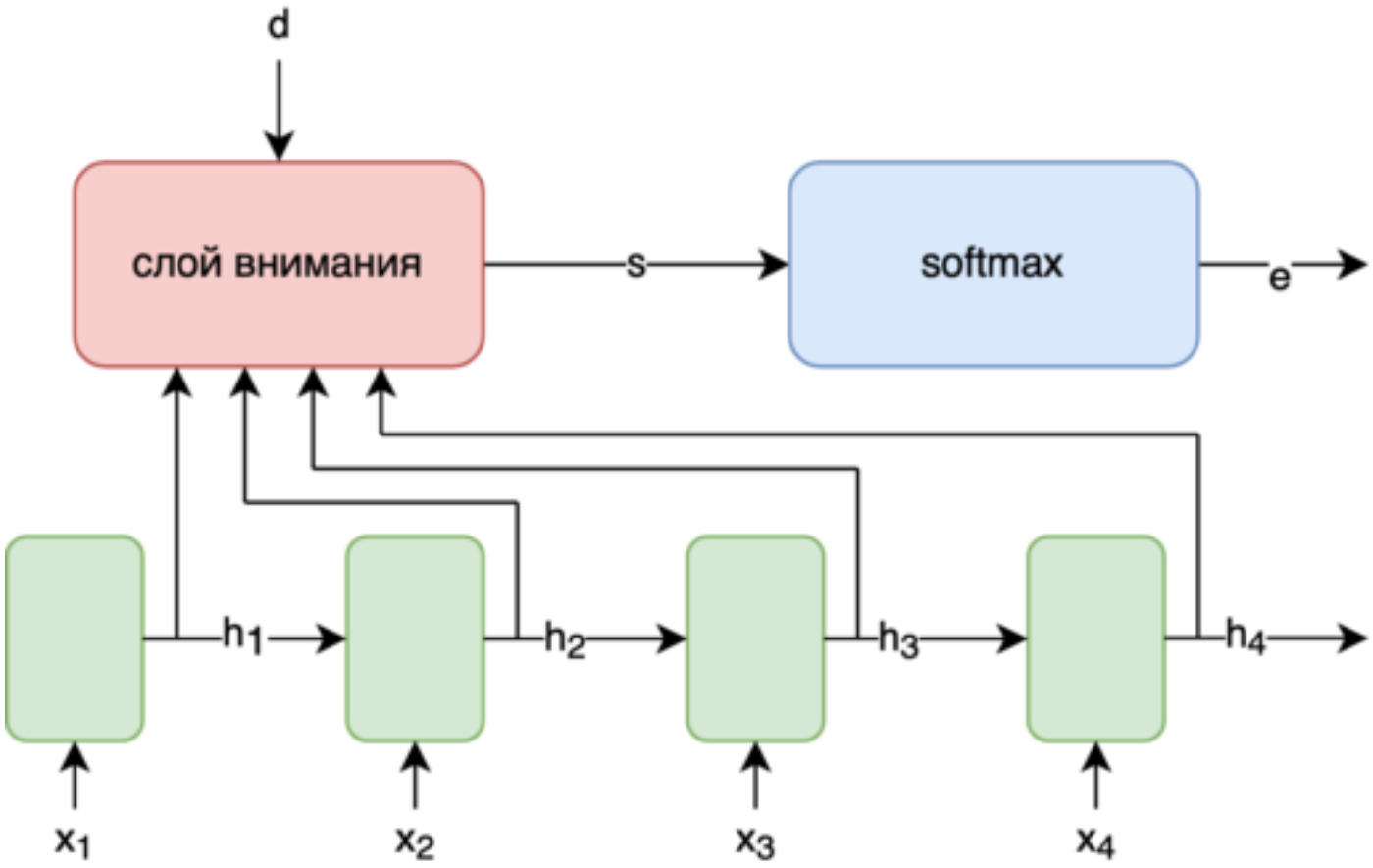


Рис. 2. Слой внимания

Механизмы внимания VI

Добавление между энкодером и декдодером **механизма внимания** (2014-2015 – Bahdanau, Luong).

Алгоритм работы НС с механизмом внимания

- x_i, h_i, d_i, y_i – те же назначения, что и в варианте без механизма внимания;
- агрегатор скрытых состояний энкодера агрегирует в себе все векторы h_i и возвращает всю последовательность векторов $h = \begin{bmatrix} h_1 & h_2 & \dots & h_m \end{bmatrix}$;
- c_i – вектор контекста на итерации i;
- блоки механизма внимания принимают h и d_{i-1}, возвращают c_i;
- по сравнению с обычной НС Seq2Seq меняются входные данные в блоках декодера: на итерации i на вход подается не y_{i-1}, а конкатенация y_{i-1} и c.

Механизмы внимания VII

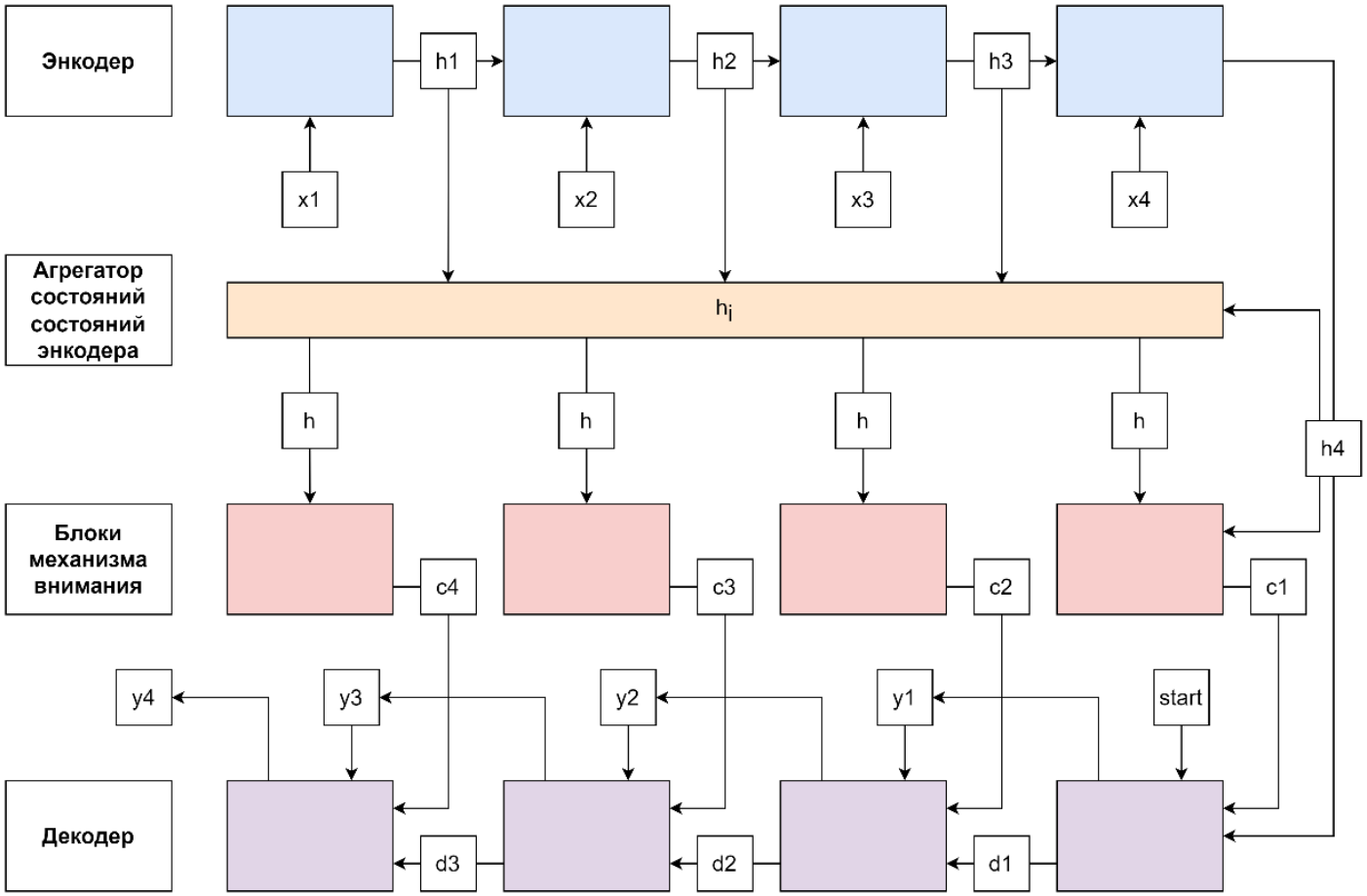


Рис. 3. Seq2Seq с механизмом внимания

Механизмы внимания VIII

Замечание

При помощи механизма внимания достигается «фокусирование» декодера на определенных скрытых состояниях. Машинный перевод: эта возможность помогает декодеру предсказывать, на какие слова из исходного текста A обратить внимание при переводе конкретного слова на язык назначения B.

Мнение: **внимание** – понятие памяти, приобретаемой в результате многократных наблюдений за происходящим во времени.

Механизмы внимания IX

Глобальное и локальное внимание:

Вычисление внимания по всей входной последовательности (глобальное внимание) может быть дорогостоящим или ненужным.
При локальном внимании мы рассматриваем только подмножество входных единиц/токенов.

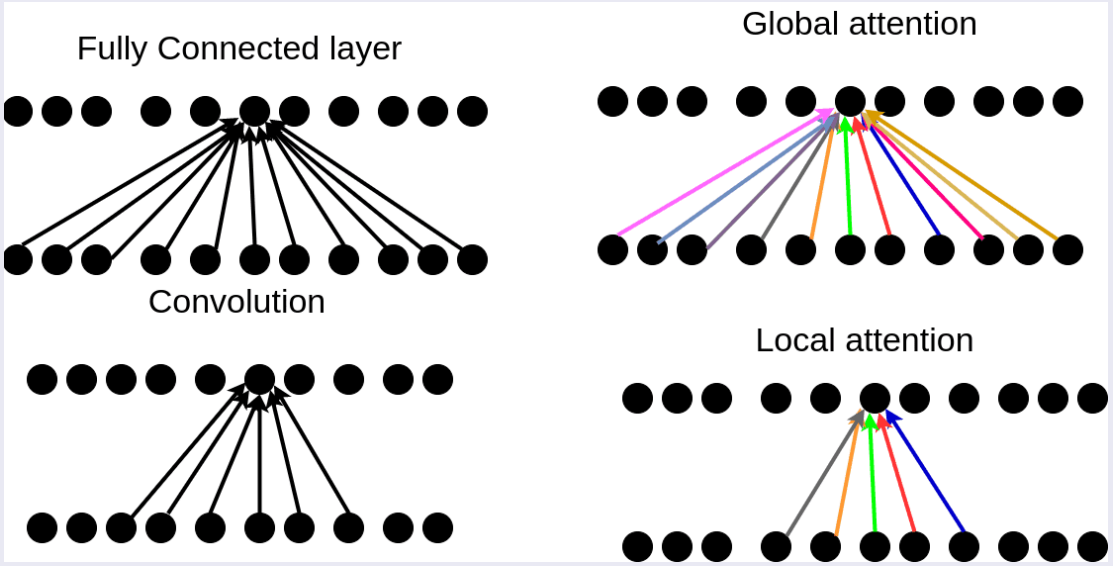


Рис. 4. Аналогия глобального и локального внимания

Механизмы внимания X

Достоинство внимания в НС с ОС:

решена проблема с доступом ко всему контексту.

Недостаток внимания в НС с ОС:

проблема последовательных вычислений – медленные вычисления, плохая масштабируемость.

1 Архитектура Seq2Seq

2 Механизмы внимания

3 Механизм внимания «Scaled Dot-Product Attention»:

4 Архитектура трансформера

5 Архитектура GPT

6 Архитектура BERT

7 Генеративно-состязательные сети (GAN)

8 Диффузионные модели

9 Заключение

10 Итоги курса «ППНСГА»

11 Литература

Механизм внимания «Scaled Dot-Product Attention»: I

Понятия:

- запрос (Query, Q) – что ищется – вектор, соответствующий текущему элементу поиска;
- ключ (Key, K) – индексы/названия того, среди чего ищется (несколько) – вектор «адреса» другого элемента, с которым Q сравнивается (той же размерности, что и Q);
- значение (Value, V) – содержимое (столько же, сколько K) – вектор.

Обобщенный алгоритм:

- вычисляется сходство Q со всеми K: чем больше сходство, тем лучше Q соответствует K (больше важность K для Q) – получаются веса на основе скалярного произведения $< Q, K >$ (ср. – **косинусное расстояние**);
- ответ – взвешенная сумма значений V.

Механизм внимания «Scaled Dot-Product Attention»: II

Матрицы весов получены в ходе обучения: $W_i^Q \in R^{d_{model} \times d_k}$, $W_i^K \in R^{d_{model} \times d_k}$, $W_i^V \in R^{d_{model} \times d_v}$, где

- d_{model} – размерность выходов всех эмбеддингов и всех слоев НС (для упрощения одинаковая), в оригинальной работе* $d_{model} = 512$;
- d_k – размерность запросов и ключей;
- d_v – размерность значений.

Вход – множество входных векторов (например, эмбеддинги слов)

$$X = \begin{bmatrix} x_1 & x_2 & \dots x_n \end{bmatrix} \in R^{n \times d_{model}}.$$

Алгоритм.

1. Расчет линейных проекций – матриц запросов, ключей и значений соответственно:

$$Q = X \cdot W_i^Q \in R^{n \times d_k}; \quad K = X \cdot W_i^K \in R^{n \times d_k}; \quad V = X \cdot W_i^V \in R^{n \times d_v}.$$

   Строки матриц q_i, k_i, v_i соответствуют запросу, ключу, значению для i-го входного слова x_i из X.

Механизм внимания «Scaled Dot-Product Attention»: III

2. Вычисление весов внимания (Attention Scores): насколько каждый запрос q_i похож на каждый ключ k_j:

$$Scores = Q \cdot K^T \in R^{n \times n}.$$

Смысл: $score_{ij}$ показывает, насколько i-му элементу запросу нужно обратить внимание на j-й ключ.

3. Масштабирование (название Scaled) и нормализация (в смысле softmax): чтобы большие скалярные произведения не затрудняли обучение (см. снова косинусное расстояние), они масштабируются на $\sqrt{d_k}$, затем получение вероятностного распределения (сумма весов $= 1$):

$$AttentionWeights = softmax\left(\frac{Scores}{\sqrt{d_k}}\right).$$

4. Результат – взвешенная сумма значений:

$$Output = AttentionWeights \cdot V.$$

Механизм внимания «Scaled Dot-Product Attention»: IV

Замечание по масштабированию:

при большой размерности d_k скалярное произведение векторов с независимыми компонентами (с нулевым средним и единичной дисперсией) будет иметь дисперсию, равную d_k. Это значит, что значения *Scores* могут становиться очень большими, особенно в начале обучения.

Итоговая формула:

$$Attention(Q, K, V) = softmax\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V.$$

Механизм внимания «Scaled Dot-Product Attention»: V

Маскировка (в трансформерах)

Проблема – каждая позиция в декодере влияет на все позиции в декодере, включая саму позицию. Чтобы значения декодера влияли только на следующие позициию

Решение – маскировка тех значений во входе softmax, которые соответствуют недопустимым связям (устанавливаются значения $-\infty$).

1 Архитектура Seq2Seq
2 Механизмы внимания
3 Механизм внимания «Scaled Dot-Product Attention»:
4 Архитектура трансформера
5 Архитектура GPT
6 Архитектура BERT
7 Генеративно-состязательные сети (GAN)
8 Диффузионные модели
9 Заключение
10 Итоги курса «ППНСГА»
11 Литература

Архитектура трансформера I

Самовнимание –

разновидность механизма внимания, задачей которой является выявление закономерности только между входными данными (Q, K, V происходят из одного и того же источника).

Происходит обновление информации («обогащение») на основе всех слов во фразе. Это позволяет улавливать синтаксические и семантические связи внутри последовательности:

- согласование слов (яблоко $\rightarrow$ красное);
- кореференцию (соотношение слов в одним смыслом – он, она, оно);
- логические зависимости.

Архитектура трансформера II

Эффективность самовнимания:

позволила отказаться от использования НС с обратными связями; заменить их на обычные НС в комбинации с механизмом самовнимания.
Это позволило ускорить работу алгоритма: при использовании трансформера каждое слово в предложении может обрабатываться параллельно (за счет использования матричных вычислений с Q, K, V).

Идея самовнимания:

поиск не в памяти энкодера, а **внутренний анализ самой последовательности** для построения обогащенных контекстных представлений.

Пример определения контекстной взаимосвязи

Предложение **The animal didn't cross the street because it was too tired**.
Результат работы алгоритма самовнимания для слова **it**. Полученный вектор соответствует **взаимосвязи** слова **it** со всеми остальными словами в предложении.

Архитектура трансформера III

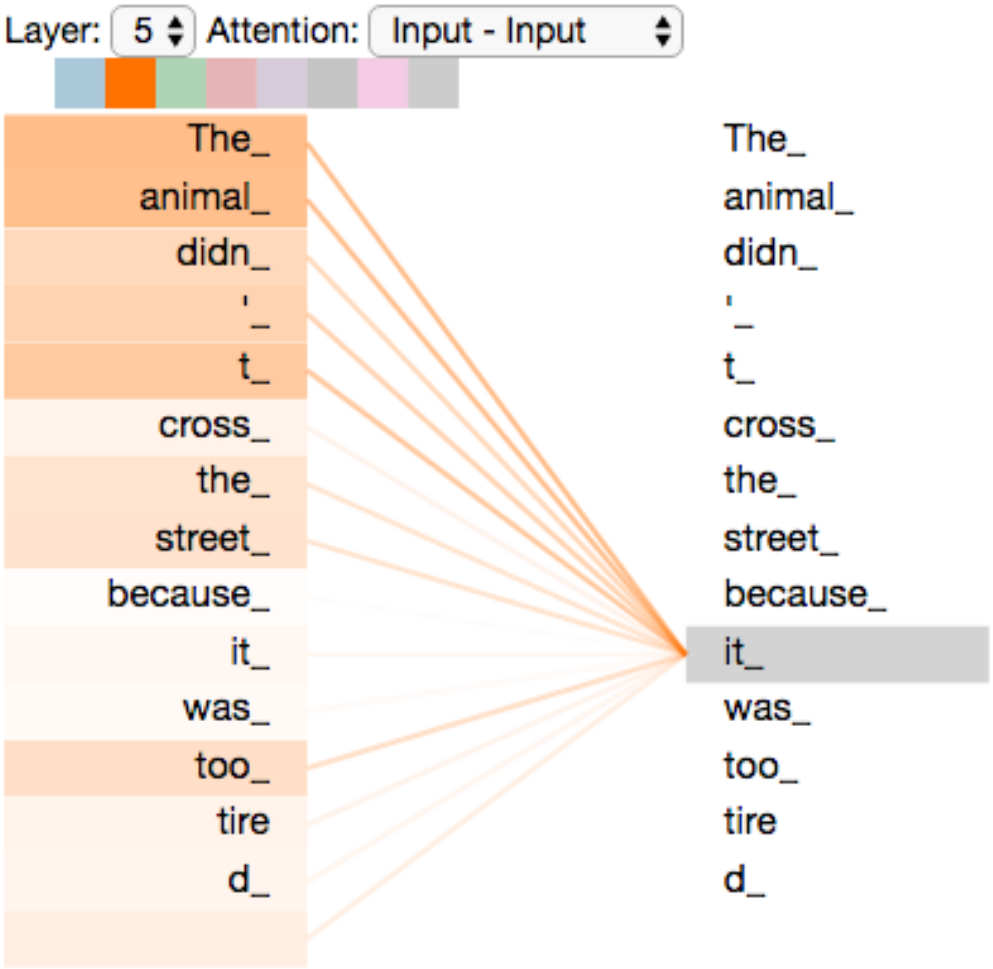


Рис. 5. Пример взаимосвязи элементов в предложении (поиск кореференции)

Архитектура трансформера IV

Как это происходит в процессе обучения:

1. Эмбеддинг слова it создает свой вектор.
2. В одном из внутренних слоев его Query-вектор начинает сильнее всего соответствовать Key-вектору слова animal.
3. Это происходит потому что в обучающих данных (миллиарды предложений) связки «animal... it» встречаются часто, и матрицы W_i^Q и W_i^K обучаются так, чтобы их проекции для связанных слов давали большое скалярное произведение.
4. В результате, когда вычисляются веса внимания для it, самый большой вес приходится на animal, и в итоговое представление слова it добавляется значение (информация) слова animal.

Архитектура трансформера V

Самовнимание как граф

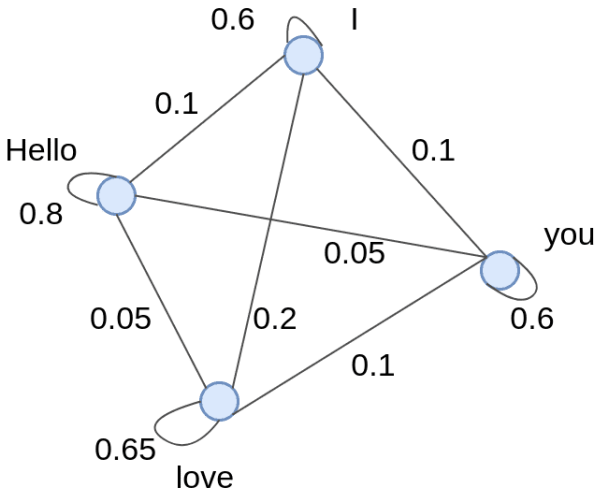
Self-attention
Probability score matrix

	Hello	I	love	you
Hello	0.8	0.1	0.05	0.05
I	0.1	0.6	0.2	0.1
love	0.05	0.2	0.65	0.1
you	0.2	0.1	0.1	0.6

Softmax(Attention)
equation



Self-attention as a
undirected weighted graph



Архитектура трансформера VI

Многоголовое внимание (multi-head attention) –

самовнимание реализуется в h разных подпространствах: в каждом свои матрицы W_i^Q, W_i^K, W_i^V. Это позволяет одновременно отслеживать разные типы зависимостей:

- семантические;
- синтаксические;
- грамматические;
- стиль.

Архитектура трансформера VII

Алгоритм многоголового внимания

1. Q, K, V проецируются с помощью набора из hматриц.

$$head_i = Attention(Q \cdot W_i^Q, K \cdot W_i^K, V \cdot W_i^V).$$

2. Расчет выходного контекстного вектора для каждой головы h.
3. Конкатенация (Concat) – объединяются выходы всех голов $[head_1; head_2; ...; head_h]$.
4. Финальная проекция – объединенный вектор проходит через последнюю обучаемую линейную проекцию W_O для получения окончательного выходного вектора многоголового внимания:

$$MultiHead(Q, K, V) = Concat(head_1, ..., head_h) \cdot W_O. :$$

Архитектура трансформера VIII

Размерности

- Размерность каждого проекционного пространства обычно $\frac{d_{model}}{h}$. Если $d_{model} = 512$, $h = 8$. Тогда $d_k = d_v = \frac{d_{model}}{h} = 64$.
- Для каждой головы $h = 1, \ldots, 8$, матрицы проекций размеры $d_{model} \times d_k = d_{model} \times d_v = 512 \times 64$.
- Каждая голова работает в своем 64-мерном подпространстве.
- Выходы всех голов (8 векторов по 64) конкатенируются в вектор размером $d_{model} = 512$ (поэтому $\frac{d_{model}}{h}$).
- Этот вектор проходит через финальную линейную проекцию W_O размером 512×512.

Механизм внимания – в основе архитектуры систем – **трансформеров (transformers)**.

Архитектура трансформера IX

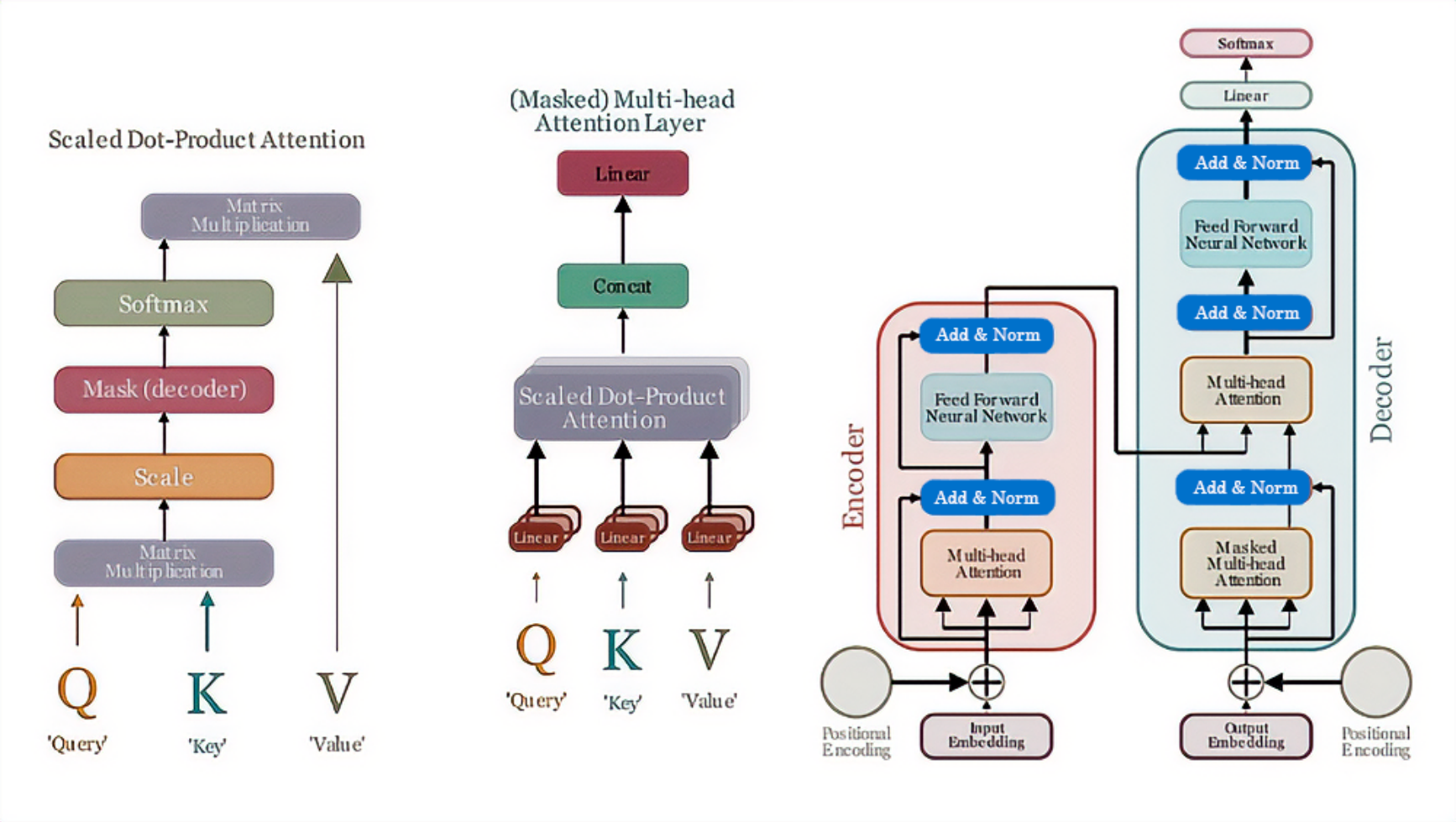


Рис. 6. Архитектура трансформера

Архитектура трансформера X

Главное достоинство трансформера:

отказ от рекуррентности.

Описание энкодера

1. Входные эмбеддинги + позиционное кодирование (чтобы модель знала порядок слов).
2. Количество слоев обычно $N = 6$ или $N = 12$.
3. Блок самовнимания (Self-Attention Layer): вычисляет Attention(Q, K, V).
4. Слой Feed-Forward: многослойный перцептрон (пост-обработка), обычно ФА – ReLU.
5. Остаточные связи (Add / Residual Connections) и нормализация слоя (Norm): стабилизируют обучение глубокой сети.

Архитектура трансформера XI

Инвариантность

Все операции внутри трансформер-блока **инвариантны к порядку элементов** в последовательности. Результат внимания зависит от скалярных произведений между эмбеддингами токенов, но расположение этих токенов внутри текста значения не имеет. Таким образом, итоговые представления каждого токена на выходе из модели будут одинаковыми вне зависимости от порядка слов.

Позиционные эмбеддинги –

вспомогательные представления, которые прибавляются к обычным эмбеддингам токенов входной последовательности и позволяют слоям внимания различать одинаковые токены на разных местах.

Архитектура трансформера XII

Фиксированные эмбеддинги:

однозначно кодируют позицию тригонометрическими функциями. Обозначения:

- pos – номер позиции;
- i – индекс элемента в векторе, кодирующем эту позицию;
- d – размерность эмбеддинга.

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right); \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right);$$

Распределенное внимание (Sparse Attention) –

вариация механизмов внимания: внимания между не всеми парами, а только между ближайшими соседями и некоторыми глобальными токенами. Это позволяет работать с длинными документами.

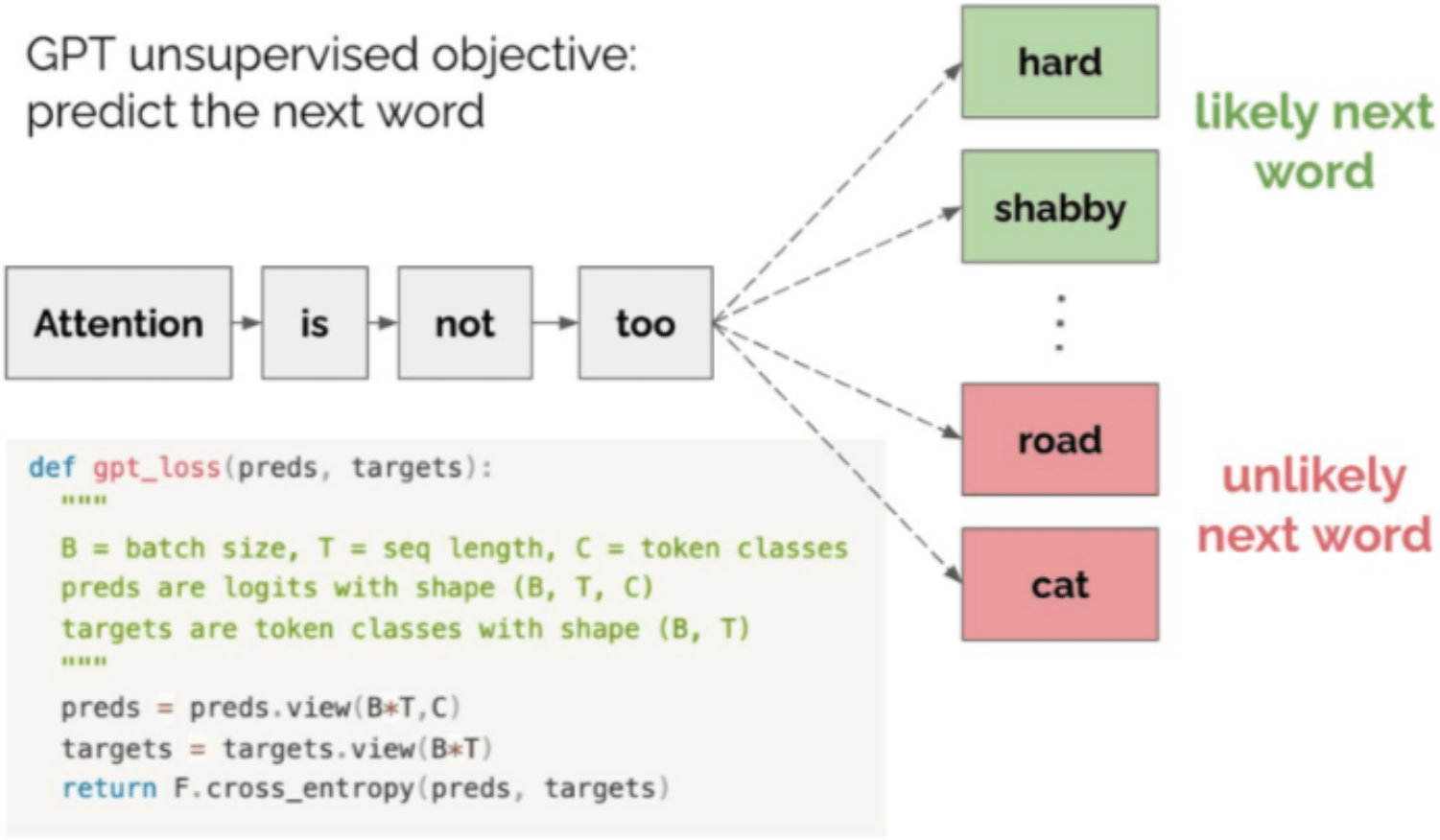
Главные факторы, повлиявшие на бурный рост популярности самовнимания, – две архитектуры – GPT и BERT, которые в некотором роде являются энкодером и декодером трансформера.

1 Архитектура Seq2Seq
2 Механизмы внимания
3 Механизм внимания «Scaled Dot-Product Attention»:
4 Архитектура трансформера
5 Архитектура GPT
6 Архитектура BERT
7 Генеративно-состязательные сети (GAN)
8 Диффузионные модели
9 Заключение
10 Итоги курса «ППНСГА»
11 Литература

Архитектура GPT I

GPT (**Generative Pretrained Transformer**) хронологически появилась раньше.
Это обычная языковую модель, реализованная в виде последовательности слоев декодера трансформера.
В качестве задачи при обучении выступает обычное предсказание следующего токена (многоклассовая классификация по словарю). Полученная модель используется для генерации текстов и решения других задач, которые на это опираются.
ChatGPT по своей сути незначительно отличается от базовой модели.

Архитектура GPT II




```
def gpt_loss(preds, targets):
  """
  B = batch size, T = seq length, C = token classes
  preds are logits with shape (B, T, C)
  targets are token classes with shape (B, T)
  """
  preds = preds.view(B*T,C)
  targets = targets.view(B*T)
  return F.cross_entropy(preds, targets)
```

Рис. 7. Иллюстрация применения GPT

1 Архитектура Seq2Seq
2 Механизмы внимания
3 Механизм внимания «Scaled Dot-Product Attention»:
4 Архитектура трансформера
5 Архитектура GPT
6 Архитектура BERT
7 Генеративно-состязательные сети (GAN)
8 Диффузионные модели
9 Заключение
10 Итоги курса «ППНСГА»
11 Литература

Архитектура BERT I

BERT (**Bidirectional Encoder Representations from Transformers**) отличается от GPT двунаправленностью внимания: при обработке входной последовательности все токены могут использовать информацию друг о друге. Это более удобно для задач, где нужно предсказать относительно всего входа целиком без генерации: при классификации предложений; при поиске пар похожих документов.
BERT не учится генерировать тексты с нуля: одна из его задач при обучении –предсказание случайно замаскированных слов по оставшимся (masked language modeling), вторая – предсказание по паре текстовых фрагментов, следуют они друг за другом или нет (next sentence prediction).

Архитектура BERT II

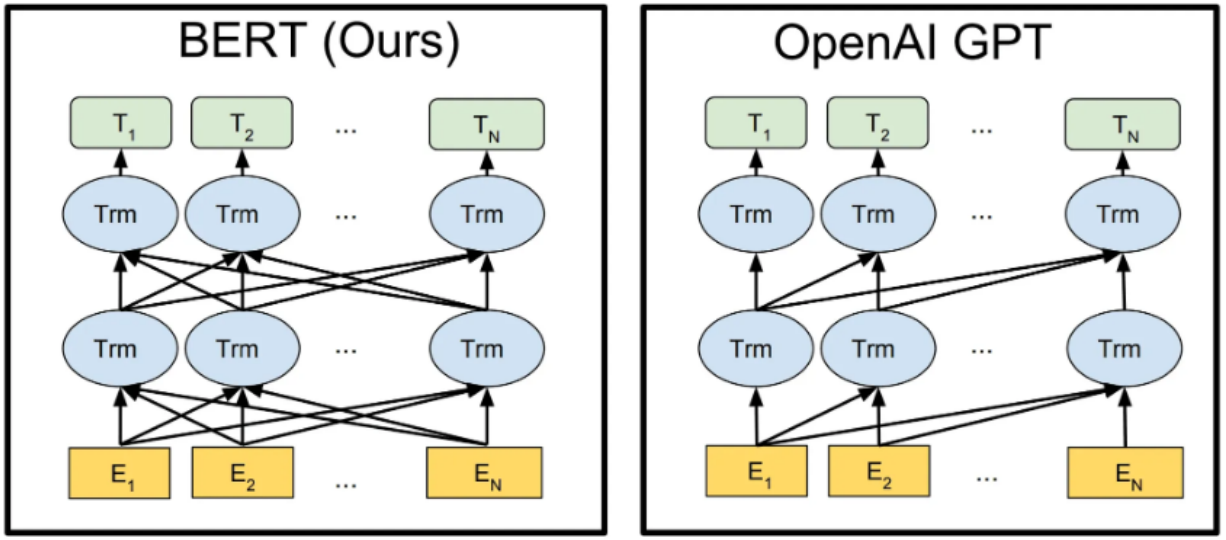


Рис. 8. Отличие механизмов внимания в моделях GPT и BERT

1 Архитектура Seq2Seq
2 Механизмы внимания
3 Механизм внимания «Scaled Dot-Product Attention»:
4 Архитектура трансформера
5 Архитектура GPT
6 Архитектура BERT
7 Генеративно-состязательные сети (GAN)
8 Диффузионные модели
9 Заключение
10 Итоги курса «ППНСГА»
11 Литература

Генеративно-состязательные сети (GAN) I

Генеративно-состязательные сети (Generative Adversarial Networks, GAN) (Гудфеллоу, 2014) –

класс генеративных моделей, общая черта которых: обучаются одновременно с другой сетью, которая старается отличить сгенерированные объекты от настоящих.

Основная концепция:

состязательное обучение с двумя НС:

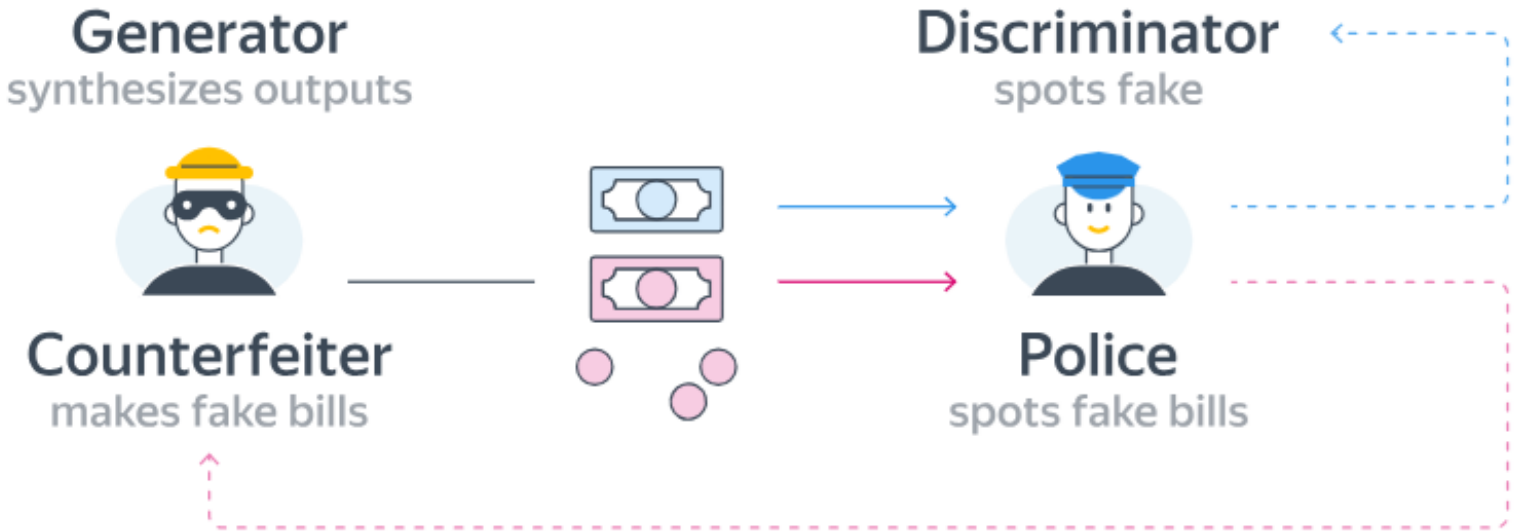
1. Генератор (G). Цель – создать на основе случайного шума данные (например, изображение), неотличимые от настоящих.
2. Дискриминатор (D). Цель – анализ поступающих данных и определение, настоящие они (1) или сгенерированные (0).

Генеративно-состязательные сети (GAN) II

Классическая аналогия, как учатся GAN:

фальшивомонетчик и **полицейский**. Фальшивомонетчик учится создавать купюры, которые полицейский не сможет отличить от реальных. Полицейский учится отличать купюры фальшивомонетчика от настоящих.

Предположим, фальшивомонетчик взаимодействует с полицейским много раз. С каждым разом купюры фальшивомонетчика всё труднее отличить от настоящих. И с каждым разом умение выявлять поддельные купюры у полицейского выше.



Генеративно-состязательные сети (GAN) III

Рис. 9. Иллюстрация процесса обучения GAN

Генератор (G):

обычно – транспонированная СНС (ConvTranspose) (обратные свертки, выходы больше, чем входы) или декодер.
Вход: вектор шума (z) из многомерного нормального распределения $p(z)$ (например, размерности 100).
Выход: вектор $\hat{x} = G(z)$, который должен быть близок распределению $p(x)$ множеству известных данных p_{data} (например, изображений).

Дискриминатор (D):

обычно СНС для изображений или классификатор.
Выход $D(\hat{x})$ – число из $[0; 1]$ (вероятность «подлинности»).

Генеративно-состязательные сети (GAN) IV

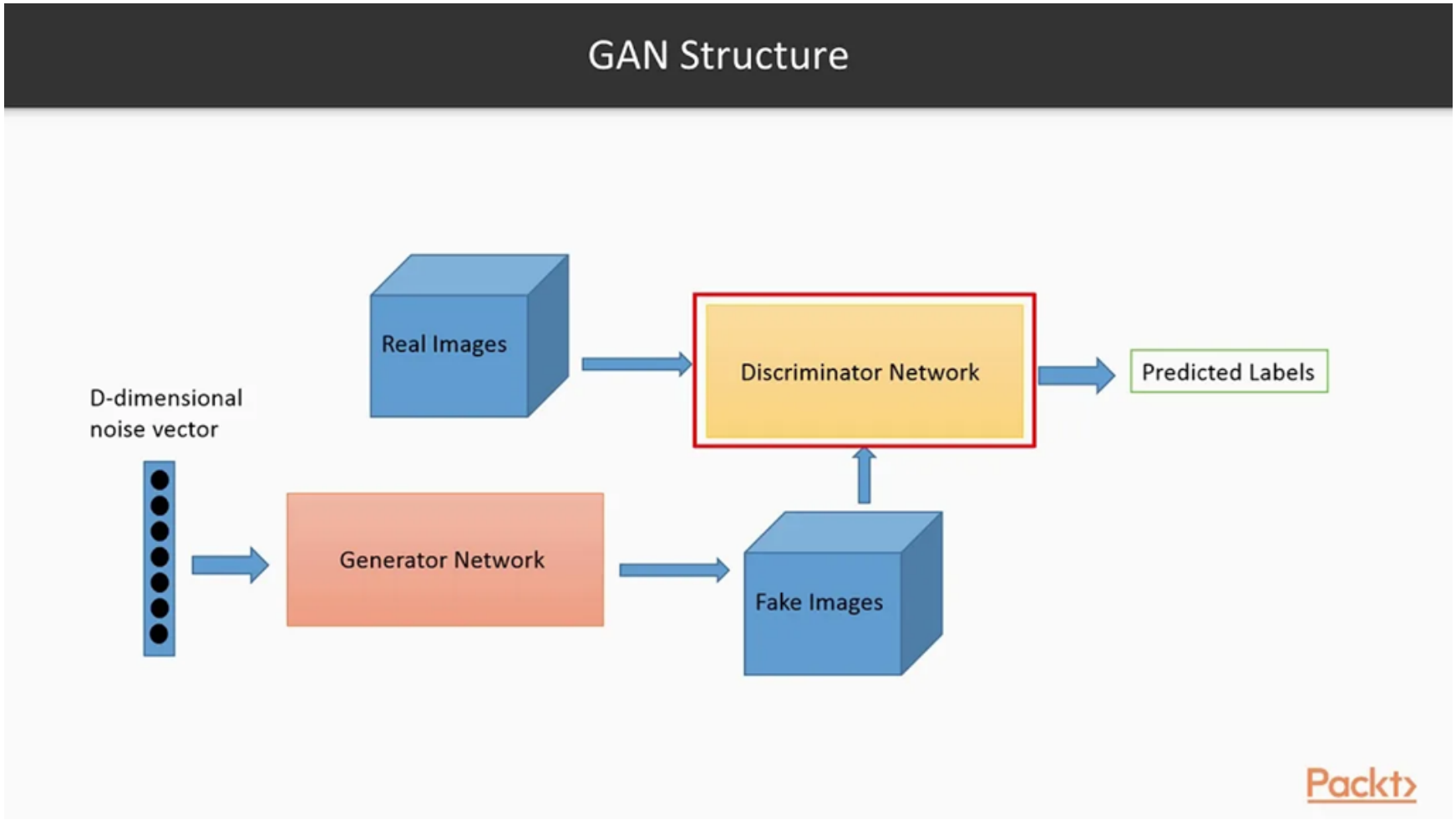


Рис. 10. Схема GAN

Генеративно-состязательные сети (GAN) V

Минимаксная целевая функция:

$$\min_G \max_D V(D, G),$$

$$V(D, G) = E(x \sim p_{data})[\log D(x)] + E(z \sim p_z)[\log(1 - D(G(z)))],$$

где

- $E(x \sim p_{data})$ – матожидание распределения реальных данных;
- $E(z \sim p_z)$ – матожидание распределения входного шума;
- $D(x)$ – вероятность, что x – реальный образец;
- $D(G(z))$ – вероятность, что сгенерированный образец принадлежит к реальным данным.

Генеративно-состязательные сети (GAN) VI

Интерпретация:

- $E(x \sim p_{data})[\log D(x)]$: максимизация логарифма вероятности присвоения метки «настоящее» ($D(x) \approx 1$) реальным данным (x).
- $E(z \sim p_z)[\log(1 - D(G(z)))]$: максимизация логарифма вероятности присвоения метки «поддельное» ($D(G(z)) \approx 0$) сгенерированным данным ($G(z)$).
- Генератор стремится минимизировать второе слагаемое, то есть сделать так, чтобы $D(G(z)) \to 1$.

Генеративно-состязательные сети (GAN) VII

Обычно на практике используется бинарная кросс-энтропия (BCE):

- Функция потерь генератора:

$$J_G = -\frac{1}{m}\sum_{i=1}^{m}\log(D(G(z_i))) \to \min\,.$$

- Функция потерь дискриминатора:

$$J_D = -\frac{1}{m}\sum_{i=1}^{m}\log(D(x_i)) - \frac{1}{m}\sum_{i=1}^{m}\log(1 - D(G(z_i))) \to \min\,.$$

Генеративно-состязательные сети (GAN) VIII

Ошибки, которые может совершать GAN:

- плохое качество или наличие артефактов – вызвана ограничениями, связанными с емкостью (capacity) генератора и несовершенством самих методов обучения. GAN плохо работает, это видно на сгенерированных сэмплах;
- сбой режима (mode collapse): обученный генератор выдаёт реалистично выглядящие картинки, но они не покрывают всё разнообразие распределения $p(x)$. Если модель учится генерировать изображения с животными, то она может проигнорировать более редкие виды, а учится генерировать только наиболее часто встречающиеся;
- модель выдаёт вариацию одной картинки (экстремальный вариант предыдущей картинки). Иногда в литературе общее качество результатов работы НС, по аналогии с задачей классификации, измеряется точностью метода (precision), а отсутствие сбоя режима измеряется полнотой (recall).

1 Архитектура Seq2Seq
2 Механизмы внимания
3 Механизм внимания «Scaled Dot-Product Attention»:
4 Архитектура трансформера
5 Архитектура GPT
6 Архитектура BERT
7 Генеративно-состязательные сети (GAN)
8 Диффузионные модели
9 Заключение
10 Итоги курса «ППНСГА»
11 Литература

Диффузионные модели I

Диффузионные модели –

класс генеративных моделей, которые учатся генерировать данные, похожие на те, на которых они обучаются.

Цель создания ДМ –

избавиться от проблем GAN.

Прямой этап (зашумление) –

постепенно (**детерминированно**) добавляют шум ε к данным, пока они не станут чистым шумом. В результате простое распределение данных за серию небольших шагов (например, за $T = 1000$) превращается в сложное x_T. Обучение не требуется.

Диффузионные модели II

Обратный этап (расшумление) –

обучается обратным процессам. Она (обычно НС U-Net для сегментации изображений) начинает с чистого шума и постепенно удаляет его, чтобы получить данные, которые с определенной близостью напоминают обучающие примеры. На основе зашумленного изображения x_t и номер шага t, она предсказывает шум ε_θ, который был добавлен. Вычитая предсказанный шум, делается шаг назад, к менее зашумленному состоянию x_{t-1}.

Функция потерь –

$$J = \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2.$$

Диффузионные модели III

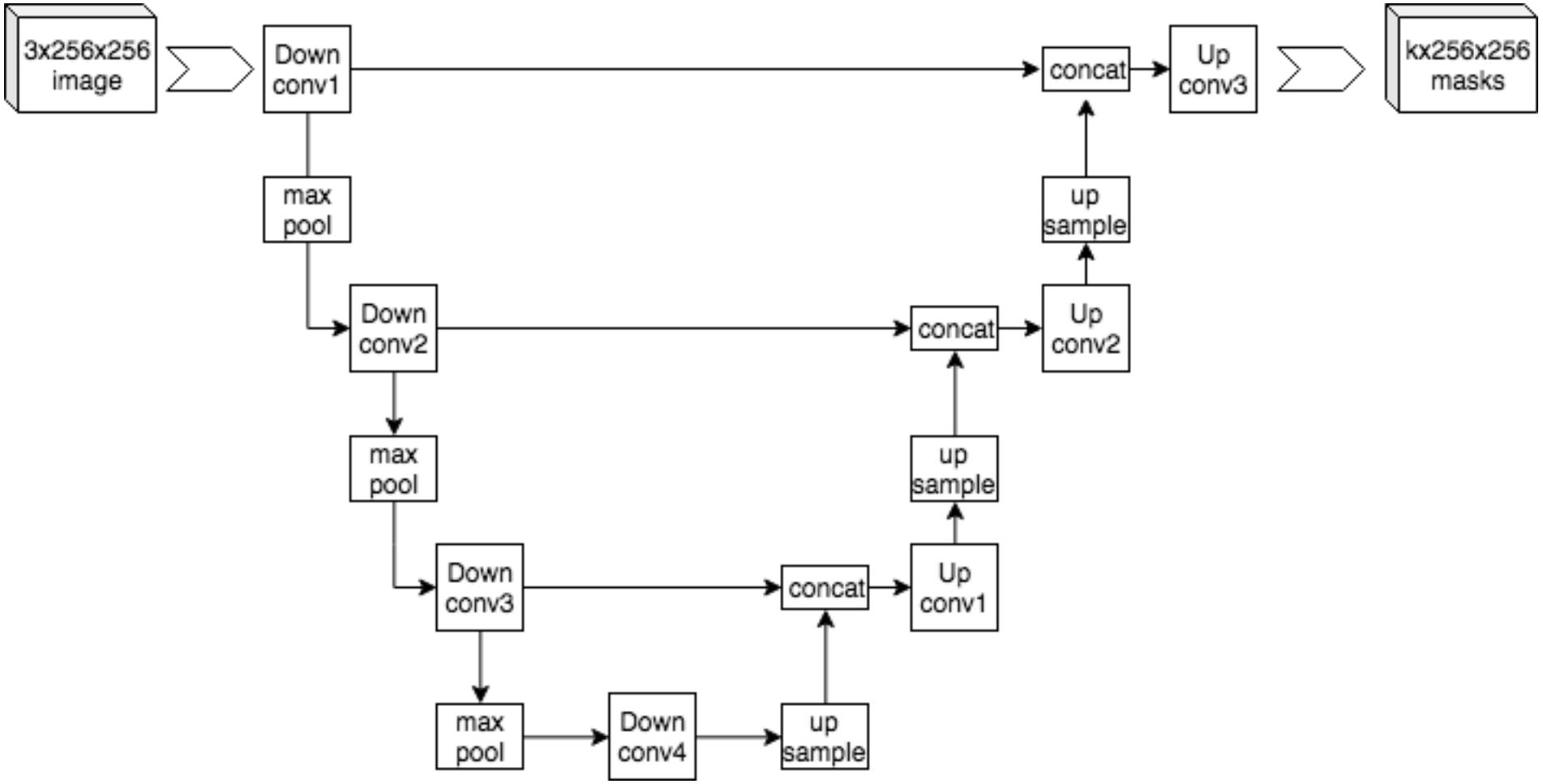


Рис. 11. Схема U-Net

Диффузионные модели IV

Диффузионные модели используются для задач компьютерного зрения, включая шумоподавление изображений, закрашивание, сверхразрешение, генерацию изображений и видео.

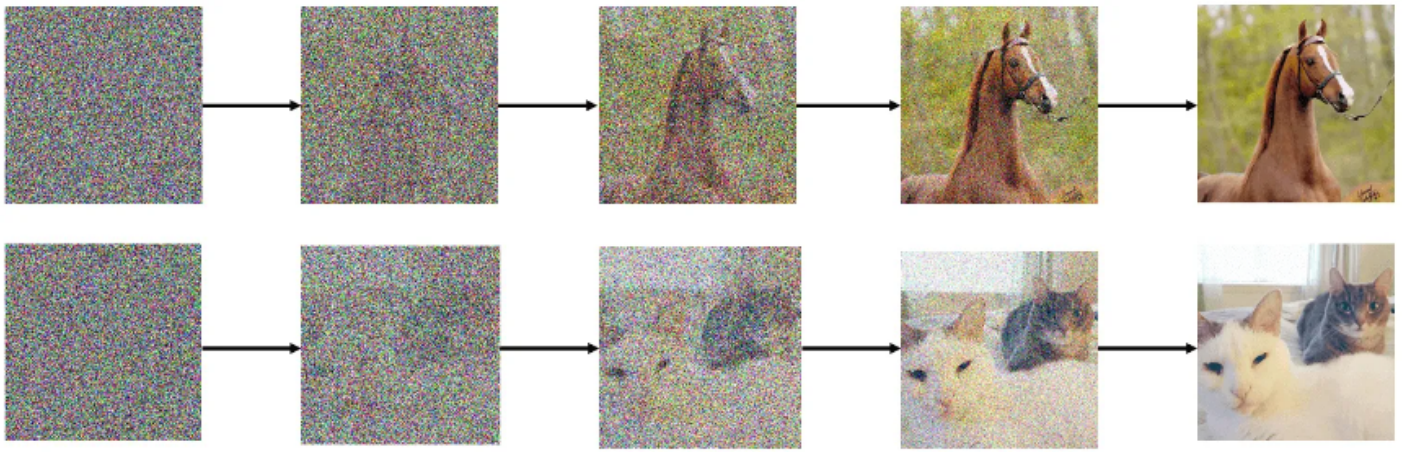


Рис. 12. Пример применения диффузионных моделей

Диффузионные модели V

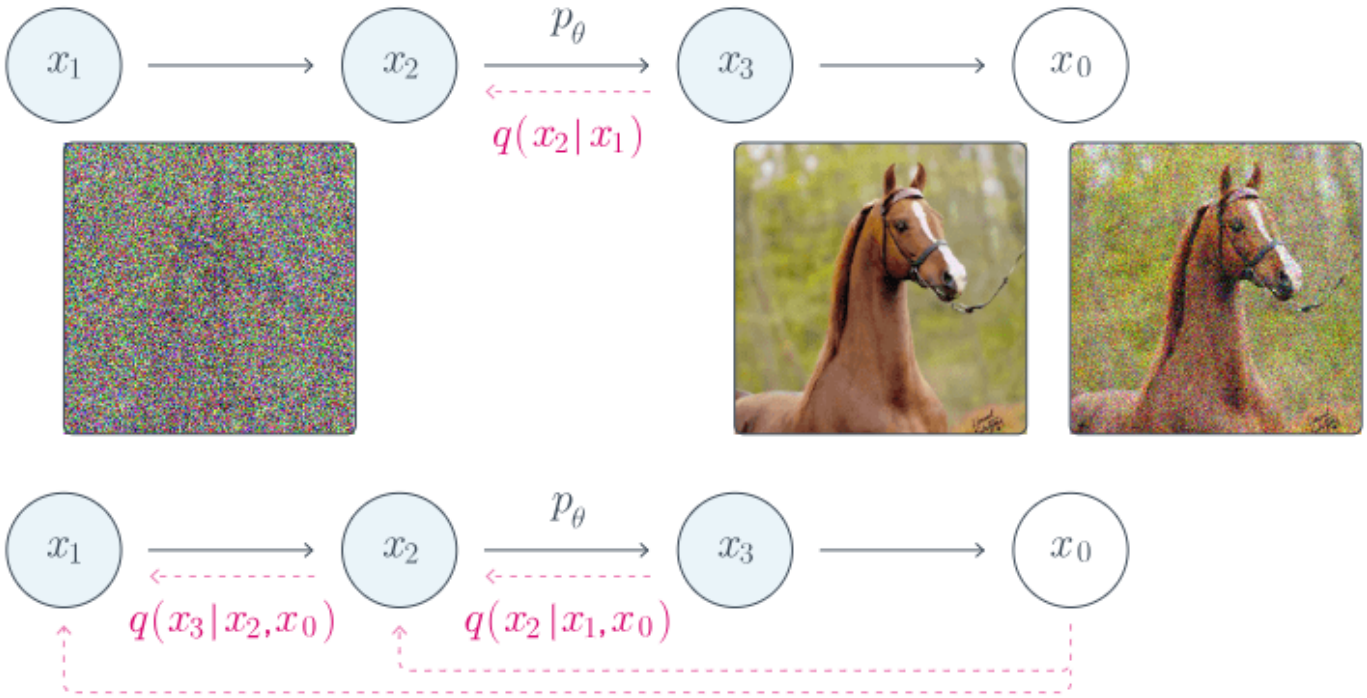


Рис. 13. Пример применения зашумления и расшумления данных

Диффузионные модели VI

Проблема ДМ:

скорость.
Классической ДМ требуются сотни или тысячи шагов для генерации одного изображения, в то время как GAN делает это за один шаг.

1 Архитектура Seq2Seq
2 Механизмы внимания
3 Механизм внимания «Scaled Dot-Product Attention»:
4 Архитектура трансформера
5 Архитектура GPT
6 Архитектура BERT
7 Генеративно-состязательные сети (GAN)
8 Диффузионные модели
9 Заключение
10 Итоги курса «ППНСГА»
11 Литература

Заключение

1. Механизм внимания в НС – способ поиска взаимосвязей в динамическом (распределенном) процессе.
2. Выделяют два механизма внимания – обобщенный и самовнимание.
3. Преимущество механизма самовнимания – уход от НС с обратными связями.
4. Основные архитектуры, которые используют механизм самовнимания: BERT и трансформеры.
5. Современные архитектуры НС – генеративно-состязательные сети (GAN) и диффузионные модели.
6. Обучение GAN хорошо иллюстрируется как взаимодействие фальшивомонетчика и полицейского.
7. Обучение диффузионных моделей заключается в применении двух этапов – зашумлении и расшумлении.

1 Архитектура Seq2Seq
2 Механизмы внимания
3 Механизм внимания «Scaled Dot-Product Attention»:
4 Архитектура трансформера
5 Архитектура GPT
6 Архитектура BERT
7 Генеративно-состязательные сети (GAN)
8 Диффузионные модели
9 Заключение
10 Итоги курса «ППНСГА»
11 Литература

Итоги курса «ППНСГА» I

N	Тема занятия
1	**Основные понятия и история развития теории нейронных сетей**. Содержание курса. Понятие нейронных сетей. История развития теории нейронных сетей. Задачи, решаемые нейронными сетями. Искусственный нейрон. Простейший перцептрон. Понятие обучения. Обучение перцептрона. Метод Монте-Карло для обучения нейронных сетей.
2	**Многослойные перцептроны и процедура обратного распространения ошибки**. Обзор архитектур нейронных сетей. Многослойный персептрон (нейронные сети прямого распространения). Универсальные аппроксимационные способности нейронных сетей. Теорема Фунахаши. Архитектура нейронных сетей с радиальными базисными функциями. Методика применения нейронных сетей. Обучение нейронных сетей как задача оптимизации. Суперпозиционная структура нейронных сетей. Процедура обратного распространения ошибки.

Итоги курса «ППНСГА» II

3	**Генетические алгоритмы**. Обучение как задача глобальной оптимизации. Генетические алгоритмы для решения задач глобальной оптимизации. Особенности применения генетических алгоритмов.
4	**Интервальные алгоритмы для решения задач глобальной оптимизации и обучения нейронных сетей**. Функционалы качества обучения, отличные от квадратичного. Основы интервального анализа. Интервальные алгоритмы в обучении нейронных сетей. Интервальные генетические алгоритмы.
5	**Самоорганизующиеся карты Кохонена и их обучение**. Самоорганизующиеся карты Кохонена SOM. Обучение без учителя. Алгоритм обучения самоорганизующихся карт Кохонена «Победитель забирает всё» и его модификации.

Итоги курса «ППНСГА» III

6	**Свёрточные и глубокие нейронные сети**. Свёрточные нейронные сети ЛеКуна. Глубокие нейронные сети. Архитектура BERT. Особенности архитектур глубоких нейронных сетей. Глубокое обучение и его особенности. Метод стохастического градиента и mini-batch метод.
7	**Нейронные сети с обратными связями**. Понятие нейронных сетей с обратными связями. Архитектура сетей Хопфилда. Архитектура сетей Хэмминга. Временная обработка с использованием многослойных перцептронов. Алгоритм BPTT.
8	**Современные архитектуры нейронных сетей. НС для решения задач компьютерного зрения и обработки текста**. Генеративные и диффузионные модели. Решение прикладных задач с помощью НС. НС для задач компьютерного зрения. Нейронные сети для решения задач обработки естественного языка.

Итоги курса «ППНСГА» IV

1 Архитектура Seq2Seq
2 Механизмы внимания
3 Механизм внимания «Scaled Dot-Product Attention»:
4 Архитектура трансформера
5 Архитектура GPT
6 Архитектура BERT
7 Генеративно-состязательные сети (GAN)
8 Диффузионные модели
9 Заключение
10 Итоги курса «ППНСГА»
11 Литература

Литература

1. Nikolas Adaloglou. How Attention works in Deep Learning: understanding the attention mechanism in sequence models.– https://theaisummer.com/attention/
2. Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A. N., Kaiser ?., Polosukhin I. Attention is All you Need (англ.) // Advances in Neural Information Processing Systems 30 / I. Guyon, U. v. Luxburg, S. Bengio, H. Wallach, R. Fergus, S.V.N. Vishwanathan, R. Garnett — 2017. — P. 15. — arXiv:1706.03762.
3. GAN https://education.yandex.ru/handbook/ml/article/generativno-sostyazatelnye-seti-(gan)
4. Диффузионные модели https://education.yandex.ru/handbook/ml/article/diffuzionnye-modeli

1 Архитектура Seq2Seq

2 Механизмы внимания

3 Механизм внимания «Scaled Dot-Product Attention»:

4 Архитектура трансформера

5 Архитектура GPT

6 Архитектура BERT

7 Генеративно-состязательные сети (GAN)

8 Диффузионные модели

9 Заключение

10 Итоги курса «ППНСГА»

11 Литература

СПАСИБО ЗА ВНИМАНИЕ!